

**ĐẠI HỌC QUỐC GIA TP. HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH**



**BÁO CÁO
ĐỒ ÁN TỐT NGHIỆP**

**NGHIÊN CỨU VÀ PHÁT TRIỂN ROBOT TRỢ LÝ ẢO
ỨNG DỤNG TRÍ TUỆ NHÂN TẠO HỖ TRỢ CHĂM SÓC
VÀ GIÁM SÁT NGƯỜI CAO TUỔI TẠI NHÀ**

Ngành: Kỹ thuật Máy tính

HỘI ĐỒNG: 10L

GVHD: TS. Lê Trọng Nhân

GVPB: PGS. TS. Trần Văn Hoài

—o0o—

SVTH 1: Lê Đăng Khoa (2211601)

SVTH 2: Vũ Trọng Khánh (2211541)

SVTH 3: Lương Văn Khanh (2211486)

Tp. Hồ Chí Minh, tháng 12/2025

LỜI CAM ĐOAN

Chúng em xin cam đoan rằng đề án với đề tài “*Nghiên cứu và phát triển robot trợ lý ảo ứng dụng trí tuệ nhân tạo hỗ trợ chăm sóc và giám sát người cao tuổi tại nhà*” là công trình nghiên cứu do chính chúng em thực hiện dưới sự hướng dẫn khoa học của TS. Lê Trọng Nhân. Toàn bộ nội dung của đề án được xây dựng trên cơ sở quá trình học tập, nghiên cứu nghiêm túc, có sự tìm hiểu, phân tích và tổng hợp từ nhiều nguồn tài liệu khác nhau.

Chúng em xin cam đoan rằng các số liệu, kết quả thực nghiệm, mô hình đề xuất, hình ảnh minh họa và các nội dung trình bày trong đề án là trung thực, phản ánh đúng quá trình nghiên cứu và thực hiện đề tài.

Trong quá trình thực hiện đề án, chúng em có tham khảo và sử dụng các tài liệu, công trình nghiên cứu, sách, bài báo khoa học, cũng như các nguồn thông tin hợp pháp khác. Tất cả các tài liệu tham khảo đều đã được trích dẫn đầy đủ, chính xác và tuân thủ đúng các quy định về trích dẫn học thuật. Chúng em xin cam đoan không sao chép, không đạo văn, không sử dụng trái phép kết quả nghiên cứu của các tác giả khác dưới bất kỳ hình thức nào.

Chúng em xin hoàn toàn chịu trách nhiệm trước Khoa, Nhà trường và Hội đồng chấm đề án về tính trung thực, chính xác và tính pháp lý của toàn bộ nội dung được trình bày trong đề án này. Nếu có bất kỳ sai sót hoặc vi phạm nào liên quan đến bản quyền, đạo đức học thuật hoặc tính xác thực của dữ liệu, chúng em xin hoàn toàn chịu trách nhiệm theo các quy định hiện hành.

Nhóm tác giả

Lê Đăng Khoa

Vũ Trọng Khánh

Lương Văn Khanh

LỜI CẢM ƠN

Trước hết, chúng em xin bày tỏ lòng biết ơn sâu sắc đến TS. Lê Trọng Nhân, người đã trực tiếp hướng dẫn, định hướng và đồng hành cùng chúng em trong suốt quá trình thực hiện đề tài “*Nghiên cứu và phát triển robot trợ lý ảo ứng dụng trí tuệ nhân tạo hỗ trợ chăm sóc và giám sát người cao tuổi tại nhà*”. Trong quá trình thực hiện đồ án, thầy đã tận tình chỉ dẫn về phương pháp nghiên cứu, định hướng triển khai nội dung, cũng như thường xuyên góp ý, chỉnh sửa và giải đáp những khó khăn mà chúng em gặp phải. Sự hướng dẫn tận tâm và những ý kiến chuyên môn quý báu của thầy đã giúp chúng em từng bước hoàn thiện đề tài theo hướng khoa học, thực tiễn và có giá trị ứng dụng.

Chúng em xin chân thành cảm ơn quý thầy cô trong Khoa/Viện đã giảng dạy và truyền đạt cho chúng em những kiến thức nền tảng và chuyên môn trong suốt quá trình học tập. Những kiến thức và kinh nghiệm mà quý thầy cô đã chia sẻ không chỉ giúp chúng em hoàn thành đồ án này mà còn là hành trang quan trọng cho quá trình học tập và công tác sau này.

Bên cạnh đó, chúng em xin gửi lời cảm ơn đến Khoa và Nhà trường đã tạo điều kiện thuận lợi về cơ sở vật chất, môi trường học tập và nghiên cứu, giúp chúng em có thể triển khai và hoàn thành đề tài đúng tiến độ.

Chúng em cũng xin chân thành cảm ơn gia đình và bạn bè đã luôn quan tâm, động viên, chia sẻ và hỗ trợ chúng em cả về tinh thần lẫn thời gian trong suốt quá trình học tập và thực hiện đồ án. Sự động viên đó là nguồn khích lệ to lớn giúp chúng em vượt qua những khó khăn và áp lực trong quá trình nghiên cứu.

Mặc dù đã có nhiều cố gắng và nỗ lực trong quá trình thực hiện, song do hạn chế về thời gian, kiến thức và kinh nghiệm nghiên cứu, đồ án không tránh khỏi những thiếu sót. Chúng em rất mong nhận được sự góp ý và chỉ dẫn từ quý thầy cô để đề tài được hoàn thiện hơn.

Chúng em xin chân thành cảm ơn.

Nhóm tác giả

Lê Đăng Khoa

Vũ Trọng Khánh

Lương Văn Khanh

TÓM TẮT

Già hóa dân số làm tăng nhu cầu hỗ trợ người cao tuổi tại nhà, trong khi người thân không phải lúc nào cũng có thể hiện diện trực tiếp. Đồ án nghiên cứu và xây dựng một robot trợ lý ảo sử dụng thiết bị Android gắn trên robot làm nền tảng cảm biến, tính toán và tương tác. Mục tiêu là kết hợp giám sát an toàn, nhắc lịch, hội thoại, liên lạc và điều khiển từ xa trong một nguyên mẫu có chi phí phần cứng hợp lý.

Hệ thống gồm RobotApp, CaregiverApp, khối mô hình AI và firmware ESP32. RobotApp xử lý camera tại biên bằng mô hình ước lượng tư thế TensorFlow Lite, theo dõi người bằng SORT/Kalman/Hungarian và tái định danh nhẹ, sau đó đưa chuỗi 15 khung hình với 69 đặc trưng vào LSTM. Bộ hậu kiểm sử dụng vùng bao, keypoint, tỷ lệ tư thế, vị trí thân trên và trạng thái mắt dấu để quyết định có hỏi xác nhận hay không. Khi người dùng cần trợ giúp hoặc không phản hồi, ứng dụng ghi cảnh báo và khởi tạo cuộc gọi khẩn cấp. RobotApp còn tích hợp nhận dạng giọng nói, Groq Chat Completions, Android Text-to-Speech và dịch vụ nhắc lịch.

CaregiverApp cho phép quản lý hồ sơ, lịch nhắc, cờ theo dõi/phát hiện té ngã, nhật ký cảnh báo–nhắc việc–cuộc gọi, gọi video hai chiều và điều khiển robot bằng joystick. Firebase Authentication và Realtime Database được dùng cho danh tính, dữ liệu, lệnh và signaling; âm thanh/hình ảnh truyền bằng WebRTC. RobotApp kiểm tra timestamp và miền lệnh trước khi chuyển thành tốc độ hai bánh rồi gửi qua BLE GATT. Firmware ESP32 sử dụng NimBLE, SimpleFOC, hai encoder AS5600 và hai động cơ BLDC; watchdog 800 ms đưa robot về dừng khi mất lệnh.

Kết quả là nguyên mẫu tích hợp xuyên suốt từ giao diện người chăm sóc tới phần thân robot, kèm các artifact mô hình H5/PT/ONNX/TFLite và quy trình chuyển đổi cho Android. Báo cáo đặc tả các cơ chế an toàn, ma trận truy vết và phương pháp đánh giá; đồng thời công bố giới hạn về dữ liệu thực địa, metric AI, node Firebase toàn cục, phụ thuộc dịch vụ Internet và bảo mật trước khi phát triển thành sản phẩm.

Từ khóa: *robot trợ lý, người cao tuổi, phát hiện té ngã, pose estimation, LSTM, Android, WebRTC, BLE, điều khiển FOC.*

ABSTRACT

Population ageing increases the demand for in-home assistance while family caregivers cannot always be physically present. This thesis investigates and develops a virtual-assistant robot in which an Android device mounted on the robot provides sensing, computation, communication, and interaction. The objective is to integrate safety monitoring, reminders, voice dialogue, live communication, and remote motion control into a cost-conscious prototype.

The system consists of RobotApp, CaregiverApp, an AI model toolchain, and ESP32 firmware. RobotApp performs on-device pose inference with TensorFlow Lite, tracks the primary person using SORT, Kalman filtering, Hungarian assignment, and lightweight re-identification, and feeds a 15-frame sequence of 69 features to an LSTM. A verification layer examines bounding-box integrity, keypoint visibility, body geometry, upper-body position, and target loss before requesting confirmation. If the person asks for help or does not respond, the system records an alert and starts an emergency call. RobotApp also integrates speech recognition, Groq Chat Completions, Android Text-to-Speech, and reminder services.

CaregiverApp supports profile and schedule management, tracking and fall-detection controls, reminder/alert/call diaries, two-way video calls, and a virtual joystick. Firebase Authentication and Realtime Database provide identity, shared data, commands, and signaling, while WebRTC transports audio and video. RobotApp validates command ranges and timestamps before mapping commands to two wheel targets and sending them over BLE GATT. The ESP32 firmware uses NimBLE, SimpleFOC, two AS5600 encoders, and two BLDC motors; an 800-ms watchdog stops the robot when commands are lost.

The outcome is an end-to-end prototype spanning caregiver interaction, edge AI, real-time services, and the physical robot body, together with H5/PT/ONNX/TFLite artifacts and a model-conversion workflow. The thesis specifies safety mechanisms, requirement traceability, and an evaluation protocol, while explicitly identifying limitations in field data, quantitative AI metrics, globally scoped Firebase nodes, Internet dependencies, and security hardening required before product deployment.

Keywords: *assistive robot, older adults, fall detection, pose estimation, LSTM, Android, WebRTC, BLE, field-oriented control.*

MỤC LỤC

Lời cam đoan	i
Lời cảm ơn	ii
Tóm tắt	iii
Abstract	iv
Mục lục	v
Danh mục hình ảnh	xii
Danh mục bảng biểu	xiv
Danh mục từ viết tắt	xv
1 Giới thiệu	1
1.1 Bối cảnh và tính cấp thiết của đề tài	1
1.2 Phát biểu bài toán	2
1.3 Câu hỏi nghiên cứu	3
1.4 Mục tiêu đề tài	3
1.4.1 Mục tiêu tổng quát	3
1.4.2 Mục tiêu cụ thể	3
1.5 Đối tượng và phạm vi nghiên cứu	4
1.5.1 Đối tượng nghiên cứu	4
1.5.2 Phạm vi chức năng	4
1.5.3 Giới hạn nghiên cứu	5
1.6 Phương pháp nghiên cứu	5
1.6.1 Khảo cứu và phân tích tài liệu	5
1.6.2 Phân tích yêu cầu và đồng thiết kế hệ thống	5
1.6.3 Phát triển lập và kiểm chứng theo mã nguồn	5
1.6.4 Kiểm thử và đánh giá	6
1.7 Các công trình và giải pháp liên quan	6
1.7.1 Robot xã hội và robot hỗ trợ người cao tuổi	6
1.7.2 Robot dựa trên smartphone	7

1.7.3	Phát hiện té ngã dựa trên thị giác	7
1.7.4	Khoảng trống nghiên cứu và định vị của đề tài	7
1.8	Đóng góp của đề tài	7
1.9	Cấu trúc báo cáo	9
1.10	Kết luận chương	9
2	Cơ sở lý thuyết	10
2.1	Nền tảng Intel OpenBot	10
2.1.1	Giới thiệu Intel OpenBot	10
2.1.2	Kiến trúc hệ thống OpenBot	10
2.1.3	Giao tiếp giữa Smartphone và Robot	11
2.2	Tổng quan về Mạng Nơ-ron Nhân tạo (Neural Network)	12
2.2.1	Khái niệm và nguồn gốc sinh học	12
2.2.2	Cấu trúc neuron nhân tạo	12
2.2.3	Lan truyền ngược (Backpropagation)	14
2.2.4	Vai trò trong Thị giác Máy tính	18
2.3	Mạng Perceptron Đa tầng (MLP)	21
2.3.1	Định nghĩa MLP	21
2.3.2	Cấu trúc nhiều lớp	21
2.3.3	Ưu điểm, hạn chế của MLP và giới hạn trong xử lý ảnh	21
2.4	Mạng Nơ-ron Tích chập (CNN)	23
2.4.1	Động cơ ra đời của CNN	23
2.4.2	Các thành phần chính của CNN	25
2.4.3	Ưu điểm của CNN trong xử lý ảnh và video	26
2.4.4	Các kiến trúc CNN tiêu biểu	26
2.5	Giới thiệu YOLO (You Only Look Once)	28
2.5.1	Bài toán Object Detection	28
2.5.2	Lịch sử phát triển Object Detection	28
2.5.3	Khái niệm Bounding Box	29
2.5.4	Intersection over Union (IoU)	31
2.5.5	Các khái niệm True Positive, False Positive, False Negative	31
2.5.6	Đánh giá hiệu suất: Mean Average Precision (mAP)	32
2.5.7	Ý tưởng YOLO: Phát hiện trong một lần suy luận	33
2.5.8	So sánh YOLO với phương pháp truyền thống	34

2.6	Khảo sát YOLOv11 cho bài toán Fall Detection	35
2.6.1	Giới thiệu YOLOv11	35
2.6.2	Các phiên bản mô hình YOLOv11	36
2.7	Theo dõi và Nhận diện Người (Person Tracking)	36
2.7.1	Giới thiệu Person Tracking	36
2.7.2	Tracking-by-Detection Paradigm	37
2.8	Giới thiệu Dataset Fall Detection – CauCaFall	38
2.8.1	Mục tiêu và đặc điểm của dataset CauCaFall	38
2.8.2	Dữ liệu video trong nhà	38
2.8.3	Annotation ban đầu	39
2.8.4	Ưu điểm và hạn chế của dataset CauCaFall	39
2.8.5	Nghiên cứu liên quan: Dataset DiverseFALL10500	40
2.9	Hệ thống AI Agent và Mô hình Ngôn ngữ Lớn (LLM)	41
2.9.1	Khái niệm AI Agent	41
2.9.2	Mô hình Ngôn ngữ Lớn (Large Language Models)	41
2.9.3	Retrieval-Augmented Generation (RAG)	49
2.10	Hệ thống Nhận dạng Giọng nói và Xử lý Ngôn ngữ Tự nhiên	50
2.10.1	Một số giải pháp Nhận dạng Giọng nói:	50
2.10.2	So sánh các phương pháp Xử lý Ngôn ngữ Tự nhiên (NLP)	50
2.10.3	Phương thức triển khai: Cloud Computing và Edge Computing	51
2.11	Mô hình chuỗi LSTM cho nhận dạng hành động	51
2.12	Tracking-by-detection và duy trì danh tính	52
2.13	Suy luận AI trên thiết bị và TensorFlow Lite	53
2.14	Firebase Authentication và Realtime Database	53
2.15	WebRTC, signaling và ICE	54
2.16	Bluetooth Low Energy và GATT	54
2.17	Động cơ BLDC và điều khiển FOC	55
2.18	An toàn và quản trị rủi ro AI	55
2.19	Kết luận chương	56
3	Phân tích và thiết kế hệ thống	57
3.1	Mục tiêu và nguyên tắc thiết kế	57
3.2	Mô hình bối cảnh và tác nhân	57
3.3	Phân tích yêu cầu	59

3.3.1	Yêu cầu chức năng	59
3.3.2	Yêu cầu phi chức năng	60
3.4	Mô hình use case	61
3.5	Kiến trúc triển khai	62
3.6	Thiết kế dữ liệu Firebase	63
3.7	Thiết kế kênh điều khiển chuyển động	64
3.7.1	Hợp đồng lệnh trên RTDB	65
3.7.2	Ánh xạ joystick sang hai bánh	65
3.7.3	Hợp đồng BLE và chuỗi dừng an toàn	66
3.8	Thiết kế pipeline thị giác và té ngã	66
3.8.1	Luồng dữ liệu	66
3.8.2	Khóa người mục tiêu ID0	67
3.8.3	Đặc trưng LSTM và máy trạng thái	67
3.8.4	Hậu kiểm và quản lý một sự cố	67
3.9	Thiết kế cuộc gọi WebRTC và quyền sở hữu camera	68
3.10	Thiết kế hội thoại và lịch nhắc	68
3.11	Thiết kế firmware và điều khiển động cơ	69
3.12	An toàn, bảo mật và quyền riêng tư	69
3.13	Ma trận truy vết yêu cầu	70
3.14	Đánh đổi và quyết định kiến trúc	71
3.15	Kết luận chương	71
4	Hiện thực hệ thống	72
4.1	Môi trường và tổ chức mã nguồn	72
4.2	Hiện thực RobotApp	72
4.2.1	Khởi động, điều hướng và vòng đời dịch vụ	72
4.2.2	TrackingFragment và quyền sở hữu camera	73
4.2.3	Nạp và suy luận mô hình pose	73
4.2.4	SORT, Kalman, Hungarian và Re-ID	74
4.2.5	Khóa mục tiêu ID0 và sinh lệnh auto-follow	74
4.2.6	LSTM phát hiện té ngã	75
4.2.7	Hậu kiểm té ngã	75
4.2.8	Luồng xác nhận và cảnh báo	75
4.2.9	Chatbot giọng nói	76

4.2.10	Lịch nhắc	77
4.2.11	WebRTC trong RobotApp	77
4.2.12	RobotControlManager và BLE	77
4.3	Hiện thực CaregiverApp	78
4.3.1	Khởi động, xác thực và hồ sơ	78
4.3.2	Bật/tắt chức năng theo dõi	78
4.3.3	Lịch và nhật ký	78
4.3.4	Cuộc gọi đến nền	78
4.3.5	Video call và joystick	78
4.4	Hiện thực dữ liệu dùng chung	79
4.5	Huấn luyện và chuyển đổi mô hình AI	79
4.5.1	Dữ liệu và bài toán	79
4.5.2	Xây dựng vector đặc trưng chuỗi	81
4.5.3	Chuyển Keras LSTM sang TFLite	81
4.5.4	Xuất mô hình pose qua ONNX	81
4.5.5	Chuyển ONNX sang TFLite và FP16	81
4.5.6	Kiểm tra tương đương sau chuyển đổi	82
4.6	Hiện thực firmware phần thân robot	82
4.6.1	Khởi tạo phần cứng	82
4.6.2	BLE GATT server	83
4.6.3	Deadzone và watchdog	83
4.7	Cấu hình và triển khai	83
4.8	Những điểm hiện thực cần lưu ý	84
4.9	Kết luận chương	84
5	Kết quả và thảo luận	85
5.1	Mục tiêu đánh giá	85
5.2	Cấu hình và tiêu chí đánh giá	85
5.3	Kết quả hiện thực và artifact	86
5.3.1	Mức độ hoàn thành thành phần	86
5.3.2	Kết quả đóng gói Android	87
5.3.3	Kết quả artifact mô hình	87
5.4	Kiểm thử tự động ở mức logic	88
5.4.1	Ánh xạ điều khiển	88

5.4.2	Hậu kiểm té ngã	88
5.5	Đánh giá hợp đồng tích hợp bằng kiểm tra tĩnh	89
5.5.1	Nhất quán Firebase	89
5.5.2	Nhất quán BLE	89
5.5.3	Chuỗi timeout	89
5.5.4	Quyền sở hữu tài nguyên	90
5.6	Kịch bản kiểm thử chức năng đầu cuối	90
5.7	Thiết kế phép đo hiệu năng	92
5.7.1	Độ trễ pipeline AI	92
5.7.2	Độ trễ điều khiển	92
5.7.3	WebRTC	92
5.7.4	Độ chính xác té ngã	92
5.8	Thảo luận kết quả	92
5.8.1	Điểm mạnh	93
5.8.2	Hạn chế	93
5.8.3	Mối đe dọa tới tính hợp lệ	93
5.9	Mức đáp ứng mục tiêu	94
5.10	Khuyến nghị trước nghiệm thu và trình diễn	94
5.11	Kết luận chương	94
6	Kết luận	96
6.1	Tổng kết	96
6.2	Kết quả đạt được	97
6.3	Hạn chế	97
6.4	Hướng phát triển	98
6.4.1	Đánh giá AI có thể tái lập	98
6.4.2	Tăng độ bền theo dõi và phát hiện	98
6.4.3	An toàn và điều khiển robot	98
6.4.4	Backend và bảo mật nhiều người dùng	98
6.4.5	Khả năng hoạt động khi mất mạng	99
6.4.6	Cải thiện trải nghiệm người cao tuổi	99
6.4.7	Tối ưu triển khai	99
6.5	Kết luận cuối cùng	99

Phụ lục	105
Phụ lục A – Đặc tả use case	105
Phụ lục B – Hợp đồng dữ liệu và giao thức	109
Phụ lục C – Danh mục module mã nguồn	110
Phụ lục D – Manifest artifact AI	112
Phụ lục E – Quy trình build và triển khai	113
Phụ lục F – Phiếu ghi kết quả kiểm thử	114
Phụ lục G – Checklist an toàn trình diễn	114

DANH SÁCH HÌNH VẼ

2.1	So sánh giữa neuron sinh học (trái) và neuron nhân tạo (phải). Các thành phần tương ứng: dendrite $\leftrightarrow$ input, synapse $\leftrightarrow$ weight, soma $\leftrightarrow$ summation, axon $\leftrightarrow$ activation function và output.	13
2.2	Computational Graph minh họa quá trình Forward Pass (mũi tên xanh) và Backward Pass (mũi tên đỏ) với các gradient được tính ngược từ Loss về các tham số.	15
2.3	Bề mặt hàm mất mát $J(\theta_0, \theta_1)$ và quỹ đạo của Gradient Descent (đường đen) tìm điểm cực tiểu.	17
2.4	Bốn bài toán chính trong Thị giác Máy tính: Phân loại ảnh, Phát hiện đối tượng, Phân vùng ngữ nghĩa, và Nhận dạng hành động.	18
2.5	Kiến trúc mạng Perceptron Đa tầng (MLP) với lớp đầu vào, hai lớp ẩn và lớp đầu ra. Mỗi neuron trong một lớp kết nối đầy đủ với tất cả neuron của lớp liền kề.	22
2.6	Minh họa phép tích chập (Convolution) và Max Pooling trong CNN: bộ lọc 3×3 trượt qua ảnh đầu vào tạo ra feature map, sau đó Max Pooling 2×2 giảm kích thước không gian.	25
2.7	AlexNet	27
2.8	VGG	27
2.9	Ví dụ minh họa Bounding Box trong Object Detection.	30
2.10	Minh họa chia lưới 7×7 trong YOLO: mỗi ô chịu trách nhiệm phát hiện đối tượng có tâm nằm trong ô đó.	34
2.11	Flowchart pipeline Tracking-by-Detection: Detection $\rightarrow$ Feature Extraction $\rightarrow$ Association $\rightarrow$ State Update, lặp lại cho mỗi frame.	37
2.12	Minh họa không gian vector embedding và độ tương đồng cosine giữa các từ	42
2.13	Kiến trúc Transformer với các khối Encoder và Decoder	43
2.14	Cơ chế Self-Attention: Query, Key, Value và Scaled Dot-Product Attention	44
2.15	Ví dụ Chain-of-Thought: thay vì trả lời trực tiếp, mô hình giải thích từng bước suy luận để đi đến kết quả cuối cùng.	49
3.1	Kiến trúc mức cao của hệ thống robot chăm sóc người cao tuổi	58
3.2	Kiến trúc hiện thực của hệ thống	62
4.1	Ví dụ khung hình té ngã trong dữ liệu tham khảo	76

4.2	Ví dụ phân bố số lượng thể hiện sự mất cân bằng lớp trong dữ liệu	80
-----	---	----

DANH SÁCH BẢNG

1.1	So sánh định tính với các hướng giải pháp liên quan	8
2.1	Ưu điểm và hạn chế của mạng MLP	22
2.2	So sánh YOLO với các phương pháp Object Detection khác	35
2.3	So sánh Transformer và CNN	45
2.4	So sánh các công nghệ Speech-to-Text.	50
2.5	So sánh các phương pháp NLP.	50
3.1	Tác nhân và trách nhiệm	58
3.2	Danh sách yêu cầu chức năng	59
3.3	Yêu cầu phi chức năng	60
3.4	Tóm tắt use case chính	62
3.5	Phân rã thành phần và trách nhiệm	63
3.6	Cây dữ liệu Realtime Database	63
3.7	Các điểm chuẩn ánh xạ lệnh	66
3.8	Phân tích mối đe dọa và biện pháp	69
3.9	Truy vết từ yêu cầu đến module và kiểm thử	70
4.1	Các khối mã nguồn của đồ án	72
4.2	Các artifact mô hình chính	82
5.1	Cấu hình phần mềm được đánh giá	85
5.2	Kết quả hiện thực theo thành phần	86
5.3	Artifact đóng gói hiện có	87
5.4	Đánh giá định tính các biến thể mô hình	88
5.5	Bộ unit test ánh xạ điều khiển	88
5.6	Bộ unit test hậu kiểm té ngã	89
5.7	Danh mục kiểm thử đầu cuối	90
5.8	Đối chiếu mục tiêu và kết quả	94
6.11	Thông số BLE triển khai	111
6.12	Module chính của RobotApp	111
6.13	Module chính của CaregiverApp	112
6.15	Mẫu tổng hợp benchmark AI	115
6.16	Mẫu tổng hợp phát hiện té ngã theo sự cố	115
6.17	Mẫu tổng hợp độ trễ và an toàn điều khiển	115

DANH MỤC TỪ VIẾT TẮT

Từ viết tắt	Thuật ngữ tiếng Anh	Ý nghĩa tiếng Việt
AI	Artificial Intelligence	Trí tuệ nhân tạo
AP	Average Precision	Độ chính xác trung bình
API	Application Programming Interface	Giao diện lập trình ứng dụng
BERT	Bidirectional Encoder Representations from Transformers	Biểu diễn mã hóa hai chiều từ Transformer
BLE	Bluetooth Low Energy	Bluetooth năng lượng thấp
CNN	Convolutional Neural Network	Mạng nơ-ron tích chập
COCO	Common Objects in Context	Bộ dữ liệu đối tượng phổ biến trong ngữ cảnh
CPU	Central Processing Unit	Bộ xử lý trung tâm
FFN	Feed-Forward Network	Mạng nơ-ron truyền thẳng
FOC	Field-Oriented Control	Điều khiển định hướng từ trường
FPS	Frames Per Second	Số khung hình trên giây
GATT	Generic Attribute Profile	Hồ sơ thuộc tính của Bluetooth Low Energy
GPU	Graphics Processing Unit	Bộ xử lý đồ họa
GPT	Generative Pre-trained Transformer	Mô hình Transformer sinh được tiền huấn luyện
IoT	Internet of Things	Internet vạn vật
IoU	Intersection over Union	Tỷ lệ phần giao trên phần hợp
ICE	Interactive Connectivity Establishment	Cơ chế thiết lập đường kết nối qua NAT
LLM	Large Language Model	Mô hình ngôn ngữ lớn
LSTM	Long Short-Term Memory	Mạng bộ nhớ dài-ngắn hạn
MLP	Multilayer Perceptron	Mạng perceptron đa lớp
MOTA	Multiple Object Tracking Accuracy	Độ chính xác theo dõi đa đối tượng
NAT	Network Address Translation	Biên dịch địa chỉ mạng

Từ viết tắt	Thuật ngữ tiếng Anh	Ý nghĩa tiếng Việt
NLP	Natural Language Processing	Xử lý ngôn ngữ tự nhiên
NLU	Natural Language Understanding	Hiểu ngôn ngữ tự nhiên
NMS	Non-Maximum Suppression	Úc chế không cực đại
P2P	Peer-to-Peer	Kết nối ngang hàng
R-CNN	Region-based Convolutional Neural Network	Mạng nơ-ron tích chập dựa trên vùng
RAG	Retrieval-Augmented Generation	Sinh nội dung có tăng cường truy xuất
Re-ID	Person Re-identification	Tái định danh người
RGB	Red, Green, Blue	Hệ màu đỏ, xanh lá và xanh dương
RTDB	Realtime Database	Cơ sở dữ liệu thời gian thực Firebase
SDP	Session Description Protocol	Giao thức mô tả phiên truyền thông
SORT	Simple Online and Realtime Tracking	Thuật toán theo dõi trực tuyến thời gian thực
SSD	Single Shot MultiBox Detector	Mô hình phát hiện đối tượng một giai đoạn
SOTA	State of the Art	Trình độ tiên tiến nhất tại thời điểm xem xét
STT	Speech-to-Text	Chuyển giọng nói thành văn bản
TTS	Text-to-Speech	Chuyển văn bản thành giọng nói
UID	User Identifier	Định danh người dùng
WebRTC	Web Real-Time Communication	Giao tiếp thời gian thực trên nền web
WHO	World Health Organization	Tổ chức Y tế Thế giới
YOLO	You Only Look Once	Họ mô hình thị giác máy tính thời gian thực

Chương 1 – Giới thiệu

1.1 Bối cảnh và tính cấp thiết của đề tài

Già hóa dân số đã trở thành bài toán hiện hữu đối với hệ thống y tế, an sinh xã hội và từng gia đình. Tổ chức Y tế Thế giới (WHO) dự báo số người từ 60 tuổi trở lên trên toàn cầu tăng từ khoảng 1 tỷ người năm 2020 lên 1,4 tỷ người vào năm 2030 và 2,1 tỷ người vào năm 2050; khoảng hai phần ba số người cao tuổi sẽ sống tại các quốc gia thu nhập thấp và trung bình [1]. Quá trình chuyển đổi này diễn ra nhanh hơn trước, trong khi nguồn nhân lực chăm sóc dài hạn, cơ sở hạ tầng và khả năng chi trả không tăng với cùng tốc độ.

Tại Việt Nam, áp lực còn rõ rệt hơn vì thời gian chuyển từ cơ cấu dân số “già hóa” sang “dân số già” tương đối ngắn. Dự báo dựa trên Tổng điều tra dân số và nhà ở năm 2019 cho thấy Việt Nam có thể trở thành quốc gia có dân số già vào khoảng năm 2036, khi nhóm từ 65 tuổi trở lên đạt xấp xỉ 14% dân số; đến năm 2069, người cao tuổi có thể chiếm 27,1% dân số [2]. Nghiên cứu về kinh tế chăm sóc người cao tuổi tại Việt Nam còn dự báo số người cần hỗ trợ thực hiện các hoạt động sống hằng ngày tăng từ khoảng 4,7 triệu năm 2025 lên 6,5 triệu năm 2035 [3]. Mô hình chỉ dựa vào chăm sóc trực tiếp bởi thành viên gia đình vì vậy sẽ ngày càng khó duy trì, đặc biệt tại đô thị khi người thân thường làm việc xa nhà.

Té ngã là một rủi ro tiêu biểu cần được phát hiện sớm. WHO định nghĩa té ngã là sự kiện khiến một người ngoài ý muốn nằm lại trên mặt đất, sàn hoặc một mức thấp hơn. Mỗi năm thế giới ghi nhận khoảng 684.000 ca tử vong do té ngã và 37,3 triệu trường hợp đủ nghiêm trọng để cần chăm sóc y tế; nhóm trên 60 tuổi chịu số ca tử vong cao nhất [4]. Trong nhà, sự cố có thể xảy ra tại thời điểm không có người chăm sóc. Khoảng thời gian từ khi xảy ra sự cố đến khi người thân biết và can thiệp vì vậy có ý nghĩa trực tiếp đối với mức độ hậu quả.

Bên cạnh rủi ro té ngã, người cao tuổi thường phải quản lý nhiều thuốc và lịch sinh hoạt. Tổng quan hệ thống về tuân thủ thuốc ở người cao tuổi chỉ ra rằng bệnh đồng mắc, đa thuốc, suy giảm chức năng và độ phức tạp của phác đồ là những yếu tố làm tăng khả năng không tuân thủ [5]. Cô đơn và cô lập xã hội cũng là các yếu tố nguy cơ quan trọng đối với sức khỏe tinh thần ở tuổi già [6]. Do đó, một giải pháp hỗ trợ tại nhà không nên chỉ là camera giám sát thụ động; hệ thống cần kết hợp nhắc việc, tương tác tự nhiên, liên lạc trực tiếp và cơ chế cảnh báo có thể kiểm chứng.

Sự phổ biến của điện thoại thông minh tạo điều kiện xây dựng robot chi phí thấp. Điện thoại cung cấp camera, microphone, loa, kết nối mạng và năng lực tính toán đủ cho nhiều tác vụ thị giác máy tính. Công trình OpenBot chứng minh một điện thoại Android có thể đóng vai trò bộ não và hệ cảm biến cho thân xe robot giá thấp, đồng thời hỗ trợ các tác vụ như bám theo người và điều hướng [10]. Cách tiếp cận này phù hợp với bối cảnh của đề tài: tái sử dụng phần cứng phổ biến, đưa suy luận AI đến gần nguồn dữ liệu và chỉ dùng vi điều khiển cho vòng điều khiển động cơ.

Từ các nhu cầu trên, đề tài “**Nghiên cứu và phát triển robot trợ lý ảo ứng dụng trí tuệ nhân tạo hỗ trợ chăm sóc và giám sát người cao tuổi tại nhà**” xây dựng một nguyên mẫu tích hợp thay vì các chức năng rời rạc. Hệ thống gồm ứng dụng RobotApp trên thiết bị Android gắn với robot, ứng dụng CaregiverApp cho người chăm sóc, firmware ESP32 điều khiển hai động cơ BLDC và chuỗi mô hình AI được chuyển đổi để suy luận trên thiết bị. Giá trị cốt lõi nằm ở việc kết nối chu trình *nhận biết – đánh giá – tương tác xác nhận – thông báo – liên lạc – can thiệp từ xa* trong cùng một kiến trúc.

1.2 Phát biểu bài toán

Bài toán đặt ra là thiết kế một robot di động trong nhà có thể hỗ trợ người cao tuổi mà vẫn cho phép người chăm sóc theo dõi và can thiệp từ xa. Hệ thống phải giải quyết đồng thời bốn nhóm vấn đề:

1. **Nhận biết an toàn:** khai thác camera để phát hiện và theo dõi đúng người mục tiêu, nhận biết chuỗi tư thế nghi té ngã trong điều kiện ảnh thay đổi và hạn chế cảnh báo giả từ hành vi nằm/ngồi bình thường.
2. **Tương tác và chăm sóc:** hỗ trợ hội thoại giọng nói tiếng Việt, phát lời nhắc theo lịch và hỏi xác nhận khi phát hiện sự cố.
3. **Liên lạc và điều khiển:** thiết lập cuộc gọi âm thanh/hình ảnh thời gian thực; truyền lệnh joystick từ CaregiverApp qua hạ tầng đám mây tới RobotApp rồi qua BLE tới ESP32.
4. **An toàn hệ thống:** loại bỏ lệnh cũ hoặc không hợp lệ, dừng robot khi mất kết nối, tránh tranh chấp nguồn điều khiển và bảo vệ dữ liệu người dùng.

Các yêu cầu trên có quan hệ chặt chẽ. Chẳng hạn, camera vừa phục vụ cuộc gọi

WebRTC vừa cung cấp ảnh cho AI; nếu hai thành phần đồng thời mở camera sẽ gây xung đột tài nguyên. Tương tự, joystick từ xa và chế độ tự bấm đều có thể phát sinh lệnh chuyển động; nếu không xác định quyền sở hữu, robot có thể nhận hai chỉ thị trái ngược. Vì vậy, đề tài được tiếp cận như một bài toán đồng thiết kế phần mềm di động, AI biên, giao tiếp thời gian thực và điều khiển nhúng.

1.3 Câu hỏi nghiên cứu

- Có thể tổ chức kiến trúc nào để một thiết bị Android đồng thời đảm nhiệm camera, suy luận AI, hội thoại, liên lạc và cầu nối điều khiển phần thân robot?
- Làm thế nào duy trì danh tính người cần theo dõi khi vùng bao thay đổi mạnh, bị che khuất ngắn hạn hoặc xuất hiện thêm người trong khung hình?
- Làm thế nào kết hợp thông tin tư thế theo thời gian với luật hậu kiểm để giảm cảnh báo giả và vẫn giữ nguyên tắc an toàn khi người dùng không phản hồi?
- Hợp đồng dữ liệu nào bảo đảm lệnh điều khiển từ xa có thể được phát hiện là mới, hợp lệ và được dừng tự động khi đường truyền gián đoạn?
- Những giới hạn nào cần được công bố để phân biệt một hệ thống hỗ trợ nghiên cứu với thiết bị y tế hoặc sản phẩm thương mại hoàn chỉnh?

1.4 Mục tiêu đề tài

1.4.1 Mục tiêu tổng quát

Nghiên cứu, thiết kế và hiện thực nguyên mẫu robot trợ lý ảo có khả năng tương tác bằng giọng nói, nhắc lịch, theo dõi người, phát hiện tình huống nghi ngờ, gọi video và cho phép người chăm sóc điều khiển từ xa; trong đó điện thoại thông minh là nền tảng tính toán chính và ESP32 đảm nhiệm điều khiển hai động cơ.

1.4.2 Mục tiêu cụ thể

- Xây dựng RobotApp bằng Android/Kotlin để quản lý camera, suy luận mô hình TFLite, hội thoại, nhắc lịch, WebRTC và kết nối BLE.
- Xây dựng CaregiverApp để quản lý hồ sơ, lịch nhắc, trạng thái theo dõi, cảnh báo, lịch sử, cuộc gọi và joystick điều khiển robot.

- Xây dựng firmware ESP32 dùng BLE GATT, SimpleFOC và phản hồi encoder AS5600 để điều khiển vận tốc hai động cơ BLDC.
- Xây dựng chuỗi xử lý thị giác gồm mô hình pose, giải mã/NMS, SORT, Kalman, Hungarian, Re-ID, khóa người mục tiêu ID0 và LSTM phát hiện té ngã.
- Chuyển đổi mô hình từ định dạng huấn luyện sang ONNX/TensorFlow Lite, tạo biến thể đầu vào 320×320 và FP16/GPU để triển khai trên Android.
- Thiết kế cơ chế xác nhận sự cố bằng giọng nói, ghi nhận cảnh báo và tự khởi tạo cuộc gọi khi phản hồi cho thấy nguy hiểm hoặc không có phản hồi.
- Xác định các yêu cầu an toàn, bảo mật, khả năng bảo trì và giới hạn của nguyên mẫu thông qua kiểm tra mã nguồn và kiểm thử có thể tái lập.

1.5 Đối tượng và phạm vi nghiên cứu

1.5.1 Đối tượng nghiên cứu

Đối tượng kỹ thuật gồm nền tảng robot dựa trên smartphone; hai ứng dụng Android; thị giác máy tính và học sâu trên thiết bị; theo dõi đa đối tượng; xử lý chuỗi tư thế; nhận dạng và tổng hợp tiếng nói; mô hình ngôn ngữ lớn; Firebase Realtime Database; WebRTC; BLE GATT; điều khiển vận tốc BLDC bằng SimpleFOC; và quy trình chuyển đổi mô hình sang TFLite.

Đối tượng sử dụng gồm người cao tuổi sinh hoạt tại nhà và người thân/người chăm sóc từ xa. Trong nguyên mẫu, hệ thống giả định một cặp người dùng–robot chính; một số nút dữ liệu thời gian thực vẫn ở phạm vi toàn cục và chưa hỗ trợ cách ly nhiều gia đình/robot.

1.5.2 Phạm vi chức năng

- Theo dõi một người mục tiêu, tự bám theo ở mức điều khiển chuyển động cơ bản và phát hiện tình huống nghi té ngã từ camera RGB.
- Hội thoại bằng giọng nói thông qua SpeechRecognizer, Groq API và Android TextToSpeech; không huấn luyện một LLM riêng.
- Tạo và thực hiện lịch nhắc; ghi nhật ký phản hồi; hiển thị lịch sử cảnh báo và cuộc gọi.

- Gọi video hai chiều bằng WebRTC, sử dụng Firebase RTDB làm kênh trao đổi offer, answer, ICE candidate và trạng thái; không xây dựng signaling server riêng.
- Điều khiển robot qua chuỗi CaregiverApp–Firebase–RobotApp–BLE–ESP32; CaregiverApp không kết nối trực tiếp tới ESP32.

1.5.3 Giới hạn nghiên cứu

Nguyên mẫu không được định vị là thiết bị chẩn đoán hay thiết bị y tế. Dữ liệu té ngã chủ yếu là tình huống dừng lại; khả năng tổng quát với người cao tuổi thật, ánh sáng yếu, vật cản, camera rung và nhiều người cần nghiên cứu thực địa có phê duyệt đạo đức. Hệ thống phụ thuộc Internet cho Firebase, Groq và thiết lập cuộc gọi; khi mất Internet, vòng điều khiển cục bộ BLE có thể dừng an toàn nhưng các chức năng từ xa không hoạt động. Việc đánh giá lâm sàng, chứng nhận an toàn điện/cơ khí, khả năng chống nước, độ bền dài hạn và tuân thủ quy định thiết bị y tế nằm ngoài phạm vi đồ án.

1.6 Phương pháp nghiên cứu

1.6.1 Khảo cứu và phân tích tài liệu

Nhóm tổng hợp nguồn từ tổ chức quốc tế, tiêu chuẩn kỹ thuật, bài báo bình duyệt và tài liệu chính thức. Các nhóm tài liệu bao gồm già hóa và té ngã, robot hỗ trợ, phát hiện đối tượng/tư thế, theo dõi, LSTM, giao tiếp thời gian thực, cơ sở dữ liệu thời gian thực, BLE và điều khiển FOC. Kết quả khảo cứu được dùng để xác lập yêu cầu và lựa chọn kiến trúc, không dùng thay thế đánh giá trên nguyên mẫu.

1.6.2 Phân tích yêu cầu và đồng thiết kế hệ thống

Từ kịch bản chăm sóc tại nhà, nhóm xác định tác nhân, use case, yêu cầu chức năng/phi chức năng và tình huống lỗi. Thiết kế được chia theo ranh giới triển khai: CaregiverApp, dịch vụ Internet, RobotApp và firmware. Các hợp đồng giao tiếp được mô tả bằng cấu trúc dữ liệu, đơn vị, miền giá trị, timestamp, timeout và hành vi khi lỗi.

1.6.3 Phát triển lập và kiểm chứng theo mã nguồn

Các chức năng được phát triển theo lát cắt dọc: từ giao diện người chăm sóc, bản ghi RTDB, xử lý tại RobotApp đến payload BLE và chuyển động động cơ. Đối với AI, quy trình đi từ mô hình huấn luyện, xuất ONNX/TFLite, kiểm tra tensor, tích hợp Android, theo dõi mục tiêu, xử lý chuỗi và hậu kiểm. Mã nguồn đang triển khai được

xem là nguồn sự thật khi có khác biệt với tài liệu cũ.

1.6.4 Kiểm thử và đánh giá

Đánh giá gồm bốn mức: kiểm tra tĩnh hợp đồng và cấu hình; unit test cho ánh xạ lệnh và luật hậu kiểm; build ứng dụng; thử nghiệm tích hợp trên thiết bị với camera, Firebase, WebRTC, BLE và robot. Chỉ số gồm khả năng build, tỷ lệ test đạt, thời gian phản hồi từng chặng, số khung hình xử lý mỗi giây, precision/recall/F1 của cảnh báo té ngã, độ ổn định ID và tỷ lệ dừng an toàn. Khi chưa có log đo đáng tin cậy, báo cáo ghi rõ trạng thái “chưa đo” thay vì suy diễn số liệu.

1.7 Các công trình và giải pháp liên quan

1.7.1 Robot xã hội và robot hỗ trợ người cao tuổi

Các nghiên cứu robot chăm sóc thường phân thành robot đồng hành, robot hiện diện từ xa, robot hỗ trợ thao tác, robot phục hồi chức năng, robot giám sát và robot nhắc việc. Một tổng quan hệ thống về công nghệ robot cho người trên 65 tuổi ghi nhận các mục đích như hỗ trợ cảm xúc, rèn luyện nhận thức, giao tiếp, theo dõi sức khỏe và phát hiện/ngăn ngừa té ngã [7]. Tuy nhiên, mức bằng chứng khác nhau đáng kể theo thiết kế nghiên cứu và loại robot.

PARO là ví dụ nổi bật của robot đồng hành trị liệu. Tổng quan các thử nghiệm ngẫu nhiên cho thấy tín hiệu tích cực về chất lượng sống và trạng thái tâm-sinh lý, nhưng cũng nhấn mạnh chất lượng bằng chứng thấp đến trung bình và cần thận trọng khi khái quát [8]. Tổng quan phạm vi về PARO còn chỉ ra rào cản chi phí, khối lượng công việc, kiểm soát nhiễm khuẩn, kỳ thị và đạo đức [9]. Robot vì vậy không tự động thay thế người chăm sóc; giá trị thực tế phụ thuộc thiết kế lấy người dùng làm trung tâm và quy trình vận hành.

Pepper đại diện cho robot hình người có khả năng tương tác xã hội, trong khi ElliQ đại diện cho thiết bị đồng hành đặt bàn. Các nền tảng thương mại cho thấy tiềm năng của hội thoại, nhắc việc và kết nối xã hội nhưng thường có chi phí, khả năng tùy biến và mức độ sẵn có theo vùng khác nhau. Đề tài không cố tái tạo hình thái robot hình người; thay vào đó sử dụng thân robot hai bánh và smartphone để ưu tiên chi phí, khả năng phát triển và tính mô-đun.

1.7.2 Robot dựa trên smartphone

OpenBot chuyển phần cảm biến, tính toán và giao tiếp sang điện thoại Android, còn thân robot đảm nhiệm chấp hành. Công trình gốc báo cáo thân xe chi phí thấp và chứng minh các tác vụ robot học được trên phần cứng phổ thông [10]. Đề tài kế thừa tư tưởng này nhưng mở rộng theo hướng chăm sóc: tách hai ứng dụng theo vai trò, bổ sung lịch nhắc, cảnh báo té ngã, hội thoại, WebRTC và điều khiển từ xa. Phần thân cũng khác cấu hình OpenBot tham chiếu vì dùng ESP32, hai BLDC, AS5600 và SimpleFOC.

1.7.3 Phát hiện té ngã dựa trên thị giác

Các hệ thống phát hiện té ngã có thể dùng thiết bị đeo, cảm biến môi trường, radar/depth camera hoặc camera RGB. Camera RGB không yêu cầu người dùng nhớ đeo thiết bị nhưng gây quan ngại riêng tư và dễ bị ảnh hưởng bởi che khuất, ánh sáng, góc nhìn. Tổng quan về phát hiện té ngã bằng học sâu cho thấy các hướng phổ biến gồm phân loại ảnh/đoạn video, phát hiện người ngã và mô hình dựa trên pose; thách thức chung là dữ liệu dựng lại, khác biệt miền và đánh đổi giữa độ chính xác với thời gian thực [24].

CAUCAFall cung cấp video trong môi trường ít kiểm soát, có nhãn fall/no-fall và mô tả khoảng cách, góc rơi, độ chiếu sáng; dữ liệu phù hợp cho cả detector và phương pháp dựa trên bản đồ xương [25]. DiverseFALL10500 bổ sung khoảng 10.500 ảnh có chú thích với đa dạng môi trường, ánh sáng, trang phục và độ tuổi [26]. So với detector chỉ phân lớp một khung hình, đề tài sử dụng pose theo thời gian, theo dõi một người mục tiêu và hậu kiểm sự cố để giảm nhầm lẫn giữa nằm chủ động, ngồi thấp, che khuất và té ngã.

1.7.4 Khoảng trống nghiên cứu và định vị của đề tài

Các công trình liên quan thường tối ưu một lớp riêng lẻ: tương tác xã hội, phát hiện té ngã, hiện diện từ xa hoặc thân robot giá thấp. Khoảng trống mà đề án hướng tới là tích hợp toàn chuỗi trên nền tảng Android phổ biến, có hai vai trò sử dụng rõ ràng và có đường điều khiển tới phần cứng thật. Bảng 1.1 tóm tắt định vị này.

1.8 Đóng góp của đề tài

- Kiến trúc nguyên mẫu gồm hai ứng dụng Android, Firebase, WebRTC, dịch vụ LLM, BLE và firmware điều khiển động cơ.

Bảng 1.1: So sánh định tính với các hướng giải pháp liên quan

Hướng	Điểm mạnh	Giới hạn điển hình	Liên hệ với đề tài
Robot xã hội PARO/Pepper	Tương tác, hỗ trợ cảm xúc, hoạt động có hướng dẫn	Chi phí, bằng chứng dài hạn còn hạn chế; không tập trung điều khiển di động từ xa	Kế thừa nhu cầu tương tác nhưng dùng smartphone và thân xe đơn giản
Thiết bị đồng hành ElliQ	Nhắc việc, trò chuyện, kết nối người thân	Nền tảng đóng, phụ thuộc dịch vụ, không phải robot giám sát di động tổng quát	Tương đồng ở nhắc việc/hội thoại; đề tài bổ sung camera, robot và AI biên
OpenBot	Chi phí thấp, tận dụng smartphone, hỗ trợ robot học	Kiến trúc tham chiếu không chuyên cho chăm sóc người cao tuổi	Là nền tảng tư tưởng cho phân chia smartphone–thân robot
Detector té ngã YOLO	Suy luận nhanh, định vị trực tiếp người/tư thế	Dễ nhầm hành vi tĩnh; phụ thuộc dữ liệu và góc nhìn	Được đặt trong pipeline pose–tracking–LSTM–hậu kiểm
Hệ thống thiết bị đeo	Đo gia tốc trực tiếp, riêng tư hơn camera trong một số bối cảnh	Người dùng phải nhớ đeo/sạc; có thể bỏ quên	Đề tài chọn camera RGB, đồng thời công bố giới hạn riêng tư
Nguyên mẫu đề tài	Tích hợp theo dõi, cảnh báo, nhắc lịch, hội thoại, video call và điều khiển	Chưa phải thiết bị y tế; cần đánh giá thực địa và hoàn thiện bảo mật đa người dùng	Đóng góp tích hợp và hợp đồng an toàn xuyên suốt

- Pipeline AI biên kết hợp pose, SORT/Kalman/Hungarian, Re-ID, người mục tiêu ID0, LSTM 15 khung hình với 69 đặc trưng và bộ hậu kiểm theo bằng chứng.
- Cơ chế chuyển quyền camera: CameraX sở hữu camera khi hoạt động bình thường; trong cuộc gọi, WebRTC sở hữu camera và chia sẻ khung hình cục bộ cho pipeline AI.
- Chuỗi an toàn chuyển động gồm giới hạn miền lệnh, timestamp chống lệnh cũ, watchdog ở Android và watchdog 800 ms trong firmware.
- Quy trình phản ứng sự cố: dừng robot, tạo đúng một cảnh báo, hỏi xác nhận bằng giọng nói, phân loại phản hồi và gọi người chăm sóc theo nguyên tắc mặc định an toàn.
- Tài liệu hóa các điểm còn hạn chế: node RTDB toàn cục, phụ thuộc dịch vụ đám mây, quản lý khóa API, phân tách nhiều robot và nhu cầu kiểm thử thực địa.

1.9 Cấu trúc báo cáo

Báo cáo gồm sáu chương. Chương 1 trình bày bối cảnh, bài toán, mục tiêu, phạm vi, phương pháp và nghiên cứu liên quan. Chương 2 hệ thống hóa cơ sở lý thuyết về robot smartphone, AI, theo dõi, phát hiện té ngã, LLM, giao tiếp và điều khiển. Chương 3 phân tích yêu cầu và thiết kế kiến trúc, dữ liệu, luồng xử lý và cơ chế an toàn. Chương 4 mô tả hiện thực hai ứng dụng, pipeline AI, chuyển đổi mô hình và firmware. Chương 5 trình bày phương pháp kiểm thử, kết quả, thảo luận giới hạn và mối đe dọa tới tính hợp lệ. Chương 6 tổng kết đóng góp và đề xuất hướng phát triển. Phụ lục cung cấp đặc tả use case, hợp đồng dữ liệu, quy trình build và danh mục kiểm thử.

1.10 Kết luận chương

Chương này đã xác lập tính cấp thiết từ xu hướng già hóa, rủi ro té ngã, nhu cầu nhắc việc và liên lạc tại nhà; đồng thời định vị nguyên mẫu trong bối cảnh robot hỗ trợ và phát hiện té ngã hiện có. Bài toán của đề tài không chỉ là huấn luyện một mô hình AI mà là bảo đảm nhiều thành phần hoạt động như một hệ thống có hành vi dự đoán được. Các kiến thức nền cho những quyết định đó được trình bày ở chương tiếp theo.

Chương 2 – Cơ sở lý thuyết

Phần này trình bày nền tảng lý thuyết cần thiết cho hệ thống robot thông minh tích hợp trí tuệ nhân tạo phục vụ giám sát và phát hiện té ngã. Nội dung được tổ chức theo thứ tự: tìm hiểu nền tảng Intel OpenBot; hệ thống nhận dạng giọng nói và xử lý ngôn ngữ tự nhiên; công nghệ Internet vạn vật (IoT) cho kết nối và điều khiển; các kiến trúc mạng nơ-ron nhân tạo (MLP, CNN); mô hình YOLO và lý do lựa chọn YOLOv11; cùng phương pháp xây dựng tập dữ liệu đa nhiệm phục vụ huấn luyện mô hình phát hiện ngã.

2.1 Nền tảng Intel OpenBot

2.1.1 Giới thiệu Intel OpenBot

Intel OpenBot là một nền tảng robot mã nguồn mở được phát triển cho nghiên cứu và giáo dục trong lĩnh vực robotics và trí tuệ nhân tạo. Điểm đặc biệt của OpenBot là sử dụng điện thoại thông minh (smartphone) làm “bộ não” chính của robot, tận dụng sức mạnh tính toán, camera và các cảm biến có sẵn trên điện thoại [10].

Các đặc điểm nổi bật của Intel OpenBot:

- **Chi phí thấp:** Phần cứng cơ bản chỉ khoảng 50-100 USD, bao gồm khung xe, động cơ, và mạch điều khiển đơn giản
- **Mã nguồn mở:** Toàn bộ thiết kế phần cứng (3D printable) và phần mềm (Android app, firmware) đều công khai
- **Tận dụng smartphone:** Sử dụng camera và khả năng kết nối của điện thoại để chạy các mô hình AI
- **Hỗ trợ Machine Learning:** Tích hợp TensorFlow Lite cho việc triển khai mô hình học sâu trực tiếp trên thiết bị

2.1.2 Kiến trúc hệ thống OpenBot

Hệ thống OpenBot bao gồm ba thành phần chính:

a) Phần cứng Robot (Hardware)

- **Khung xe:** Có thể in 3D hoặc sử dụng các kit robot có sẵn
- **Động cơ DC:** Thường sử dụng động cơ DC với encoder để điều khiển chính xác

- **Board điều khiển:** Arduino Nano hoặc ESP32 làm vi điều khiển
- **Cảm biến phụ trợ:** Có thể tích hợp thêm cảm biến siêu âm, IR, hoặc IMU
- **Pin:** Pin lithium để cấp nguồn cho robot và sạc điện thoại

b) Ứng dụng Android (Mobile App)

Ứng dụng Android là “bộ não” của robot, chịu trách nhiệm:

- Thu thập dữ liệu từ camera và các cảm biến của điện thoại
- Chạy các mô hình AI (object detection, navigation, policy learning)
- Gửi lệnh điều khiển đến phần cứng robot qua USB hoặc Bluetooth
- Giao diện người dùng cho việc điều khiển và giám sát

c) Firmware vi điều khiển

Firmware trên Arduino/ESP32 thực hiện các tác vụ cấp thấp:

- Nhận lệnh từ điện thoại (tốc độ, hướng đi)
- Điều khiển PWM cho động cơ
- Đọc encoder và cảm biến
- Gửi phản hồi trạng thái về điện thoại

2.1.3 Giao tiếp giữa Smartphone và Robot

Giao tiếp giữa điện thoại và phần cứng robot được thực hiện qua hai phương thức chính:

a) Kết nối USB Serial

- Sử dụng cáp USB OTG kết nối điện thoại với Arduino/ESP32
- Giao thức Serial với baudrate thường là 115200
- Ưu điểm: Độ trễ thấp, ổn định, có thể sạc pin cho điện thoại
- Nhược điểm: Dây cáp hạn chế tính linh hoạt

b) Kết nối Bluetooth

- Sử dụng module Bluetooth (HC-05/HC-06) hoặc Bluetooth tích hợp trên ESP32
- Giao thức Bluetooth Serial Port Profile (SPP)
- Ưu điểm: Không dây, linh hoạt
- Nhược điểm: Độ trễ cao hơn, cần quản lý pin riêng

2.2 Tổng quan về Mạng Nơ-ron Nhân tạo (Neural Network)

2.2.1 Khái niệm và nguồn gốc sinh học

Mạng nơ-ron nhân tạo (Artificial Neural Network - ANN) là một mô hình tính toán lấy cảm hứng từ cấu trúc và cách thức hoạt động của hệ thần kinh sinh học. Trong não bộ con người, hàng tỷ tế bào thần kinh (neuron) liên kết với nhau thông qua các khớp thần kinh (synapse) để xử lý và truyền tải thông tin. Mỗi neuron sinh học nhận tín hiệu đầu vào từ các neuron khác thông qua các sợi nhánh (dendrite), xử lý tín hiệu tại thân tế bào (soma), và truyền tín hiệu đầu ra qua sợi trục (axon) đến các neuron tiếp theo.

Dựa trên nguyên lý này, các nhà khoa học đã phát triển neuron nhân tạo như một đơn vị tính toán cơ bản. Ý tưởng đầu tiên về neuron nhân tạo được Warren McCulloch và Walter Pitts đề xuất vào năm 1943, mở đường cho sự phát triển của lĩnh vực học máy và trí tuệ nhân tạo hiện đại.

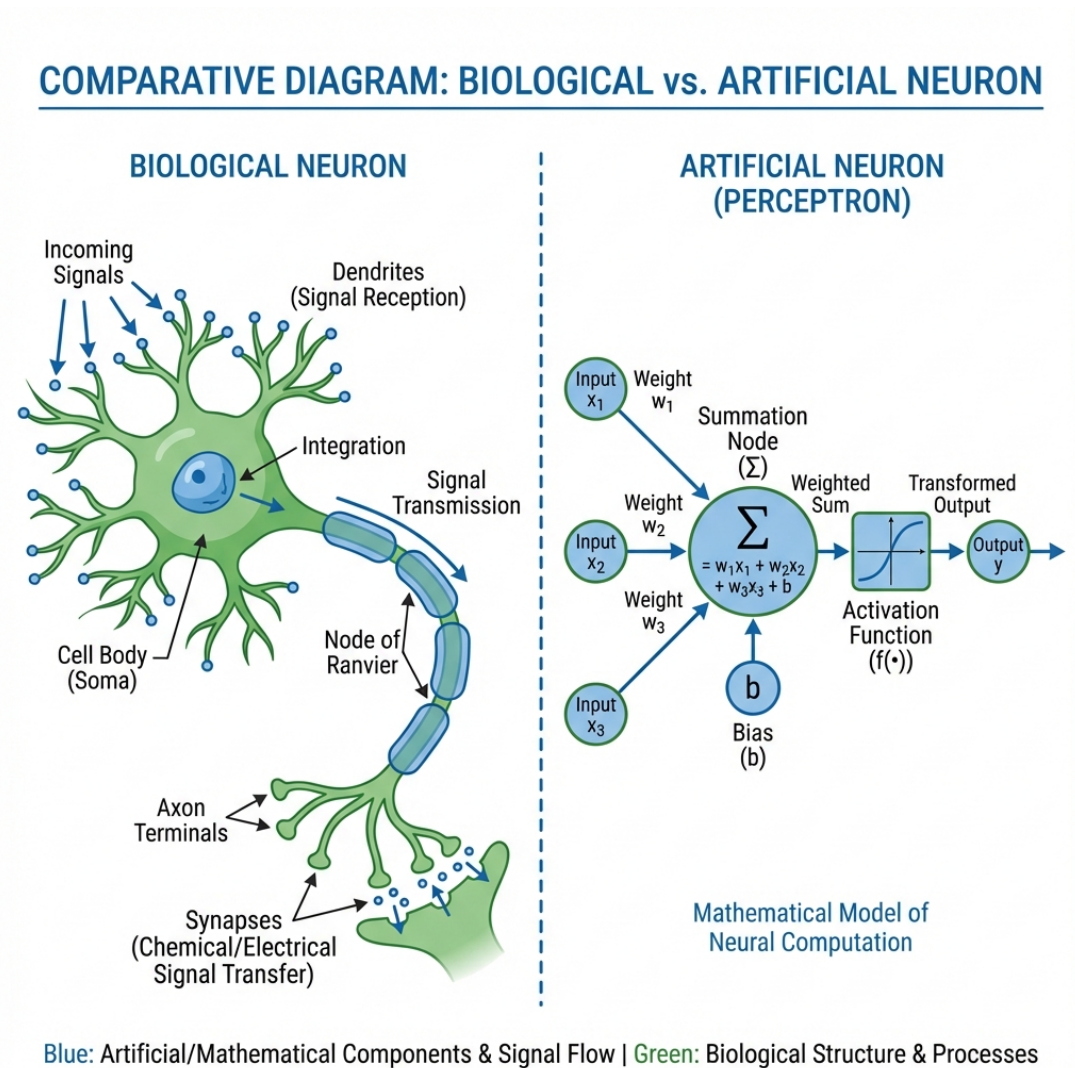
2.2.2 Cấu trúc neuron nhân tạo

Nguồn gốc sinh học và cảm hứng từ não bộ

Neuron nhân tạo được thiết kế dựa trên việc mô phỏng cấu trúc và chức năng của neuron sinh học trong hệ thần kinh con người. Một neuron sinh học hoạt động theo cơ chế:

1. **Tiếp nhận tín hiệu:** Các sợi nhánh (dendrite) tiếp nhận tín hiệu điện-hóa học từ hàng nghìn neuron khác thông qua các khớp thần kinh (synapse).
2. **Tích hợp thông tin:** Thân tế bào (soma) tổng hợp tất cả tín hiệu đầu vào. Mỗi synapse có “sức mạnh” khác nhau - một số kết nối có ảnh hưởng mạnh hơn (tương tự như trọng số trong mạng nhân tạo).

3. **Quyết định kích hoạt:** Nếu tổng tín hiệu vượt quá ngưỡng kích hoạt (threshold), neuron sẽ “bắn” (fire) một xung điện.
4. **Truyền tín hiệu:** Xung điện truyền dọc theo sợi trục (axon) đến các neuron tiếp theo.



Hình 2.1: So sánh giữa neuron sinh học (trái) và neuron nhân tạo (phải). Các thành phần tương ứng: dendrite ↔ input, synapse ↔ weight, soma ↔ summation, axon ↔ activation function và output.

Năm 1943, Warren McCulloch (nhà thần kinh học) và Walter Pitts (nhà logic học) đã công bố bài báo “A Logical Calculus of Ideas Immanent in Nervous Activity”, đề xuất mô hình toán học đầu tiên của neuron - đặt nền móng cho lĩnh vực mạng nơ-ron nhân tạo. Họ chứng minh rằng một mạng các neuron đơn giản có thể thực hiện bất kỳ phép tính logic nào.

Các thành phần của neuron nhân tạo

Ánh xạ từ neuron sinh học sang nhân tạo, một neuron nhân tạo bao gồm các thành phần chính:

- **Đầu vào (Input):** Tập hợp các giá trị $x_1, x_2, \dots, x_n$ đại diện cho dữ liệu hoặc tín hiệu từ các neuron trước đó. *(Tương ứng với dendrite trong sinh học)*
- **Trọng số (Weight):** Mỗi đầu vào x_i được gán với một trọng số w_i thể hiện mức độ quan trọng của đầu vào đó. Trọng số là tham số có thể học được trong quá trình huấn luyện. *(Tương ứng với sức mạnh synapse - sức mạnh kết nối giữa các neuron)*
- **Độ lệch (Bias):** Giá trị b được cộng thêm vào tổng có trọng số, cho phép neuron điều chỉnh ngưỡng kích hoạt một cách linh hoạt. *(Tương ứng với ngưỡng kích hoạt trong neuron sinh học)*
- **Hàm kích hoạt (Activation Function):** Hàm phi tuyến σ áp dụng lên tổng có trọng số để tạo ra đầu ra của neuron. *(Mô phỏng cơ chế “bắn” hay không “bắn” của neuron sinh học)*

Đầu ra của một neuron được tính theo công thức:

$$y = \sigma \left(\sum_{i=1}^n w_i x_i + b \right) = \sigma(W^T X + b) \quad (2.1)$$

Trong đó $W = [w_1, w_2, \dots, w_n]^T$ là vector trọng số, $X = [x_1, x_2, \dots, x_n]^T$ là vector đầu vào, và σ là hàm kích hoạt.

Các hàm kích hoạt phổ biến bao gồm:

- **Hàm Sigmoid:** $\sigma(z) = \frac{1}{1+e^{-z}}$, cho đầu ra trong khoảng $(0, 1)$
- **Hàm Tanh:** $\sigma(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$, cho đầu ra trong khoảng $(-1, 1)$
- **Hàm ReLU:** $\sigma(z) = \max(0, z)$, được sử dụng rộng rãi trong học sâu nhờ tính đơn giản và hiệu quả

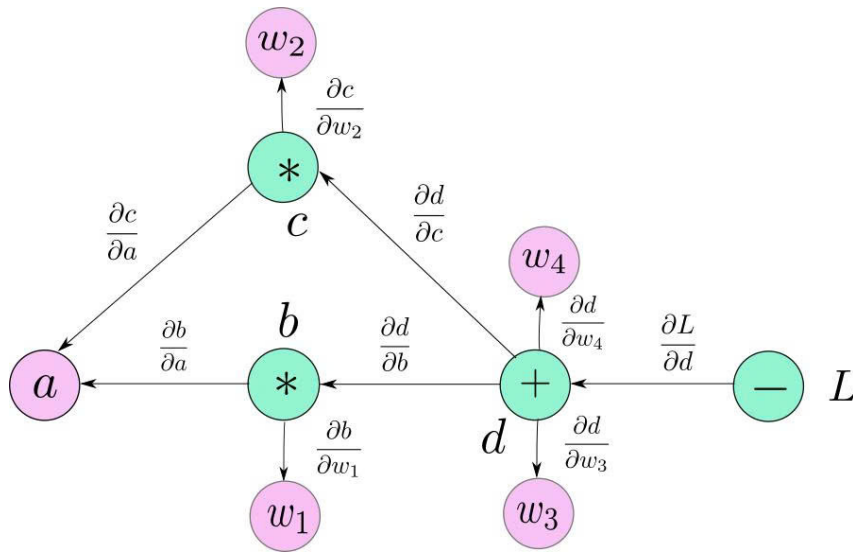
2.2.3 Lan truyền ngược (Backpropagation)

Lan truyền ngược là thuật toán cốt lõi để huấn luyện mạng nơ-ron, cho phép tính gradient của hàm mất mát theo từng tham số trong mạng. Thuật toán dựa trên quy tắc chuỗi (chain rule) trong giải tích.

Đồ thị Tính toán (Computational Graph)

Computational Graph là cách biểu diễn các phép tính toán học dưới dạng đồ thị có hướng, trong đó:

- **Các nút (nodes):** Đại diện cho các biến (inputs, outputs, intermediate values) hoặc các phép toán (cộng, nhân, activation function)
- **Các cạnh (edges):** Thể hiện sự phụ thuộc dữ liệu giữa các nút



Hình 2.2: Computational Graph minh họa quá trình Forward Pass (mũi tên xanh) và Backward Pass (mũi tên đỏ) với các gradient được tính ngược từ Loss về các tham số.

Computational Graph là nền tảng cho cơ chế **Automatic Differentiation** (vi phân tự động) trong các framework deep learning hiện đại như PyTorch, TensorFlow:

1. Forward Pass: Tính toán giá trị đầu ra theo chiều xuôi từ input đến loss

- Mỗi node lưu lại giá trị tính được (activation cache)
- Đồng thời xây dựng đồ thị các phép tính đã thực hiện

2. Backward Pass: Tính gradient theo chiều ngược từ loss về các tham số

- Bắt đầu từ $\frac{\partial \mathcal{L}}{\partial \mathcal{L}} = 1$
- Áp dụng chain rule tại mỗi node để lan truyền gradient ngược
- Mỗi node nhận gradient từ phía sau và tính gradient cho các input của nó

Ví dụ minh họa Chain Rule trên Computational Graph

Xét phép tính đơn giản: $\mathcal{L} = (w \cdot x + b)^2$ với $w = 2, x = 3, b = 1$

Forward Pass:

$$\begin{aligned} z1 &= w * x = 2 * 3 = 6 \\ z2 &= z1 + b = 6 + 1 = 7 \\ L &= z2^2 = 49 \end{aligned}$$

Backward Pass:

$$\begin{aligned} dL/dL &= 1 \\ dL/dz2 &= 2 * z2 = 14 \\ dL/dz1 &= dL/dz2 * dz2/dz1 = 14 * 1 = 14 \\ dL/db &= dL/dz2 * dz2/db = 14 * 1 = 14 \\ dL/dw &= dL/dz1 * dz1/dw = 14 * x = 14 * 3 = 42 \\ dL/dx &= dL/dz1 * dz1/dx = 14 * w = 14 * 2 = 28 \end{aligned}$$

Ưu điểm của Computational Graph:

- **Tự động hóa:** Không cần đạo hàm thủ công, framework tự động tính gradient
- **Hiệu quả:** Gradient được tính một lần backward pass, không cần tính lại nhiều lần
- **Linh hoạt:** Hỗ trợ dynamic computational graph (PyTorch) cho kiến trúc thay đổi
- **Tối ưu bộ nhớ:** Các framework áp dụng gradient checkpointing để tiết kiệm memory

Công thức Backpropagation

Cho hàm mất mát $\mathcal{L}$, gradient theo trọng số của lớp l được tính:

$$\frac{\partial \mathcal{L}}{\partial W^{(l)}} = \frac{\partial \mathcal{L}}{\partial a^{(l)}} \cdot \frac{\partial a^{(l)}}{\partial z^{(l)}} \cdot \frac{\partial z^{(l)}}{\partial W^{(l)}} \quad (2.2)$$

Đặt $\delta^{(l)} = \frac{\partial \mathcal{L}}{\partial z^{(l)}}$ là “error signal” của lớp l :

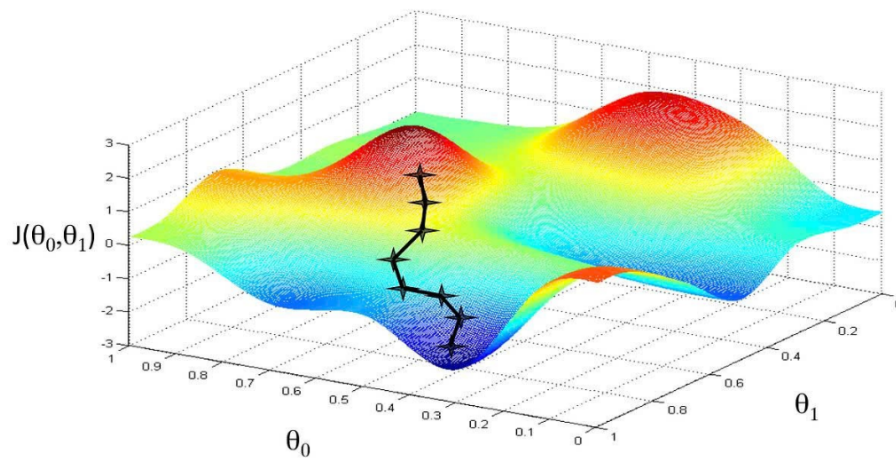
$$\delta^{(l)} = (W^{(l+1)})^T \delta^{(l+1)} \odot \sigma'^{(l)}(z^{(l)}) \quad (2.3)$$

Trong đó $\odot$ là phép nhân element-wise và σ' là đạo hàm của hàm kích hoạt. Sau khi tính được gradient, các tham số được cập nhật theo thuật toán tối ưu như Stochastic Gradient Descent (SGD):

$$W^{(l)} \leftarrow W^{(l)} - \eta \frac{\partial \mathcal{L}}{\partial W^{(l)}} \quad (2.4)$$

với η là tốc độ học (learning rate).

Ý nghĩa trực quan của Gradient Descent



Hình 2.3: Bề mặt hàm mất mát $J(\theta_0, \theta_1)$ và quỹ đạo của Gradient Descent (đường đen) tìm điểm cực tiểu.

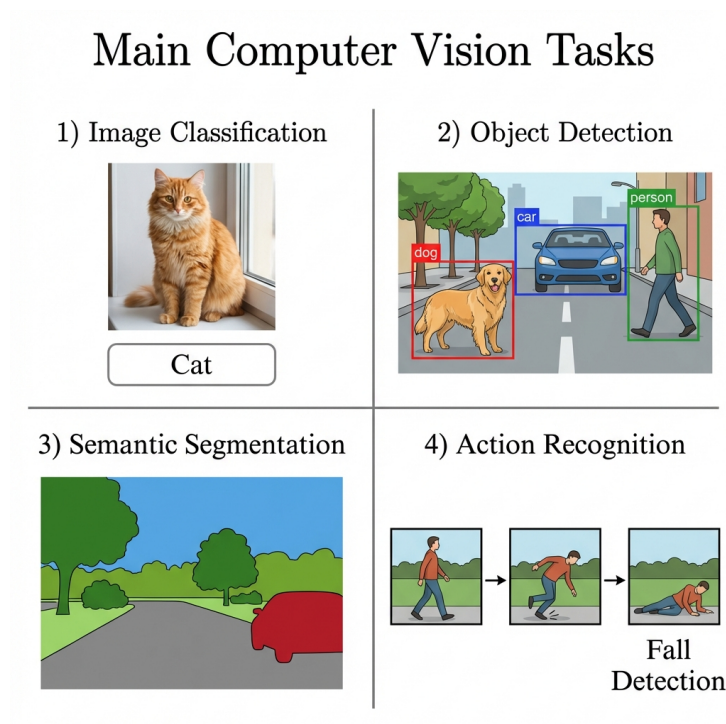
Công thức cập nhật trọng số có ý nghĩa hình học trực quan:

- **Bề mặt Loss:** Hàm mất mát $\mathcal{L}$ tạo thành một “bề mặt” trong không gian nhiều chiều, với mỗi điểm tương ứng với một tập giá trị trọng số W . Mục tiêu huấn luyện là tìm điểm thấp nhất trên bề mặt này (điểm cực tiểu).
- **Gradient:** Vector gradient $\nabla_W \mathcal{L} = \frac{\partial \mathcal{L}}{\partial W}$ chỉ hướng tăng nhanh nhất của hàm mất mát. Do đó, hướng ngược lại ($-\nabla_W \mathcal{L}$) chính là hướng giảm nhanh nhất.
- **Bước cập nhật:** Mỗi lần cập nhật, ta “đi” một bước theo hướng giảm nhanh nhất với độ lớn được kiểm soát bởi learning rate η :
 - η quá lớn: Bước đi quá dài, có thể “nhảy” qua điểm cực tiểu hoặc dao động không hội tụ
 - η quá nhỏ: Hội tụ rất chậm, tốn nhiều thời gian huấn luyện
 - η phù hợp: Hội tụ ổn định và nhanh đến điểm cực tiểu

- **Quỹ đạo tối ưu:** Qua nhiều iterations, các tham số được điều chỉnh dần để “lăn” xuống đáy của bề mặt loss, như hình minh họa với đường đen trên Hình 2.3.

2.2.4 Vai trò trong Thị giác Máy tính

Mạng nơ-ron đóng vai trò then chốt trong lĩnh vực thị giác máy tính (Computer Vision), cho phép máy tính “nhìn” và hiểu nội dung hình ảnh. Các ứng dụng tiêu biểu được minh họa trong Hình 2.4.



Hình 2.4: Bốn bài toán chính trong Thị giác Máy tính: Phân loại ảnh, Phát hiện đối tượng, Phân vùng ngữ nghĩa, và Nhận dạng hành động.

a) Phân loại ảnh (Image Classification)

Phân loại ảnh là bài toán cơ bản nhất trong thị giác máy tính, với mục tiêu gán một nhãn duy nhất cho toàn bộ ảnh đầu vào.

- **Đầu vào:** Một ảnh đơn lẻ (RGB hoặc grayscale)
- **Đầu ra:** Một nhãn lớp (class label) từ tập hợp các lớp định trước
- **Ví dụ:** Phân loại ảnh chứa chó, mèo, xe cộ; nhận diện chữ số viết tay (MNIST)
- **Ứng dụng:** Chẩn đoán y tế từ ảnh X-quang, nhận diện thực vật, kiểm tra chất lượng sản phẩm

b) Phát hiện đối tượng (Object Detection)

Phát hiện đối tượng mở rộng từ phân loại ảnh bằng cách không chỉ nhận diện loại đối tượng mà còn xác định vị trí của chúng trong ảnh.

- **Đầu vào:** Một ảnh có thể chứa nhiều đối tượng
- **Đầu ra:** Danh sách các bounding box (khung giới hạn) kèm nhãn lớp và điểm tin cậy (confidence score)
- **Các mô hình tiêu biểu:** YOLO (You Only Look Once), Faster R-CNN, SSD, RetinaNet
- **Ứng dụng:** Xe tự hành (phát hiện người đi bộ, biển báo), giám sát an ninh, đếm đối tượng

c) Phân vùng ngữ nghĩa (Semantic Segmentation)

Phân vùng ngữ nghĩa đưa việc hiểu ảnh lên mức pixel, gán nhãn lớp cho từng pixel trong ảnh.

- **Đầu vào:** Một ảnh có kích thước $H \times W$
- **Đầu ra:** Ma trận nhãn $H \times W$, mỗi pixel được gán một lớp
- **Các mô hình tiêu biểu:** U-Net, FCN (Fully Convolutional Networks), DeepLab, SegFormer

- **Ứng dụng:** Phân tích ảnh y tế (phân vùng khối u), xe tự hành (phân biệt đường, vỉa hè, xe cộ), chỉnh sửa ảnh

d) Nhận dạng hành động (Action Recognition)

Nhận dạng hành động mở rộng thị giác máy tính sang miền thời gian, phân tích chuỗi các frame video để xác định hành vi của con người hoặc đối tượng.

- **Đầu vào:** Video hoặc chuỗi các frame liên tiếp
- **Đầu ra:** Nhãn hành động (ví dụ: đi bộ, chạy, ngã, vẫy tay)
- **Các phương pháp:**
 - 3D CNN: Mở rộng CNN sang chiều thời gian (ví dụ: I3D, C3D)
 - Two-stream networks: Kết hợp RGB và optical flow
 - Transformer-based: Video Transformer, TimeSformer
- **Ứng dụng:** Phát hiện hành vi bất thường, nhận dạng cử chỉ, phân tích thể thao

e) Phát hiện té ngã (Fall Detection)

Đặc biệt quan trọng trong đề tài này, phát hiện té ngã là bài toán kết hợp nhiều kỹ thuật:

- **Object Detection:** Phát hiện và định vị người trong khung hình
- **Pose Estimation:** Ước lượng tư thế cơ thể (skeleton detection)
- **Action Recognition:** Phân tích chuỗi tư thế theo thời gian
- **Đặc trưng phân biệt:**
 - Góc nghiêng của cơ thể so với mặt đất
 - Tốc độ thay đổi vị trí (velocity)
 - Tỷ lệ chiều cao/chiều rộng của bounding box
 - Vị trí tương đối của các khớp (joints)

Mạng nơ-ron học được các đặc trưng phức tạp từ tư thế cơ thể, chuyển động, và ngữ cảnh không gian để phân biệt giữa các hành vi bình thường (ngồi xuống, cúi người) và trạng thái ngã thực sự.

2.3 Mạng Perceptron Đa tầng (MLP)

2.3.1 Định nghĩa MLP

Multilayer Perceptron (MLP) là một trong những kiến trúc mạng nơ-ron cơ bản nhất, thuộc lớp mạng truyền thẳng (feedforward neural network). MLP được cấu thành từ nhiều lớp neuron xếp chồng, trong đó tín hiệu chỉ truyền theo một chiều từ đầu vào đến đầu ra mà không có vòng lặp phản hồi.

MLP khác biệt với Perceptron đơn lớp ở chỗ có thêm một hoặc nhiều lớp ẩn (hidden layers) giữa lớp đầu vào và lớp đầu ra. Sự hiện diện của các lớp ẩn kết hợp với hàm kích hoạt phi tuyến cho phép MLP học được các hàm ánh xạ phức tạp, vượt qua giới hạn của các mô hình tuyến tính.

2.3.2 Cấu trúc nhiều lớp

Kiến trúc MLP tiêu biểu bao gồm ba loại lớp:

- **Lớp đầu vào (Input Layer):** Nhận dữ liệu thô và truyền đến các lớp tiếp theo. Số neuron bằng số chiều của vector đặc trưng đầu vào. Lớp này không thực hiện bất kỳ phép tính nào.
- **Lớp ẩn (Hidden Layers):** Một hoặc nhiều lớp thực hiện các phép biến đổi phi tuyến trên dữ liệu. Mỗi neuron trong lớp ẩn kết nối đầy đủ (fully connected) với tất cả neuron của lớp trước đó. Số lượng lớp ẩn và số neuron mỗi lớp là siêu tham số (hyperparameter) cần được lựa chọn phù hợp với bài toán.
- **Lớp đầu ra (Output Layer):** Tạo ra dự đoán cuối cùng của mô hình. Số neuron phụ thuộc vào bài toán: một neuron cho hồi quy, C neuron cho phân loại C lớp kết hợp với hàm Softmax.

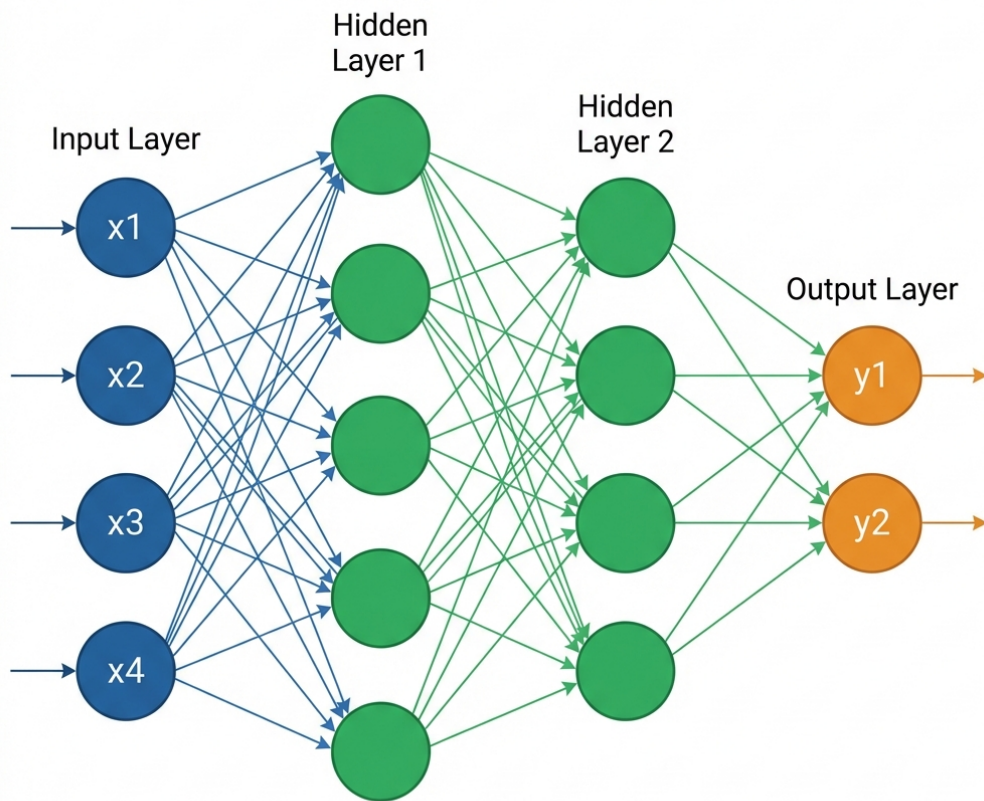
Ví dụ, một MLP với cấu trúc [784, 256, 128, 10] có 784 neuron đầu vào (phù hợp với ảnh MNIST 28×28 pixel), hai lớp ẩn với 256 và 128 neuron, và 10 neuron đầu ra cho 10 lớp chữ số.

2.3.3 Ưu điểm, hạn chế của MLP và giới hạn trong xử lý ảnh

So sánh ưu điểm và hạn chế của MLP

Hạn chế của MLP trong xử lý ảnh và video

Multilayer Perceptron (MLP) Architecture



Hình 2.5: Kiến trúc mạng Perceptron Đa tầng (MLP) với lớp đầu vào, hai lớp ẩn và lớp đầu ra. Mỗi neuron trong một lớp kết nối đầy đủ với tất cả neuron của lớp liền kề.

Bảng 2.1: Ưu điểm và hạn chế của mạng MLP

Ưu điểm	Hạn chế
Có khả năng xấp xỉ mọi hàm liên tục (Universal Approximation Theorem)	Số lượng tham số tăng nhanh theo kích thước đầu vào do kết nối đầy đủ
Cài đặt đơn giản, dễ hiểu và triển khai	Không khai thác được cấu trúc không gian của dữ liệu ảnh
Phù hợp với dữ liệu dạng bảng (tabular data) và các bài toán với đặc trưng được thiết kế sẵn	Nhạy cảm với nhiễu và biến đổi (dịch chuyển, xoay, co giãn) trong ảnh
Huấn luyện ổn định với các phương pháp tối ưu chuẩn	Dễ bị overfitting khi dữ liệu huấn luyện hạn chế

Mặc dù MLP có thể được áp dụng cho dữ liệu ảnh bằng cách “làm phẳng” (flatten) ma trận pixel thành vector một chiều, cách tiếp cận này có nhiều nhược điểm nghiêm

trọng:

1. **Mất thông tin không gian:** Khi chuyển đổi ảnh 2D thành vector 1D, quan hệ không gian giữa các pixel lân cận bị phá vỡ. Ví dụ, với ảnh $224 \times 224 \times 3$, MLP xem 150,528 giá trị pixel như các đặc trưng độc lập, mất đi khái niệm về “vùng”, “biên”, hay “hình dạng”.
2. **Số tham số khổng lồ:** Với ảnh $224 \times 224 \times 3$ và lớp ẩn đầu tiên có 1000 neuron, riêng lớp này đã cần $224 \times 224 \times 3 \times 1000 \approx 150$ triệu tham số, gây khó khăn cho việc huấn luyện và dễ dẫn đến overfitting.
3. **Không bất biến với phép biến đổi:** MLP không có tính bất biến tịnh tiến (translation invariance). Cùng một đối tượng xuất hiện ở vị trí khác trong ảnh sẽ tạo ra vector đầu vào hoàn toàn khác, khiến mô hình khó tổng quát hóa.
4. **Không thể xử lý video hiệu quả:** Video là chuỗi các khung hình, đòi hỏi xử lý thông tin thời gian. MLP không có cơ chế tự nhiên để mô hình hóa sự phụ thuộc theo thời gian.

Những hạn chế này thúc đẩy sự phát triển của mạng nơ-ron tích chập (CNN) - kiến trúc được thiết kế đặc biệt cho dữ liệu có cấu trúc không gian như ảnh và video.

2.4 Mạng Nơ-ron Tích chập (CNN)

2.4.1 Động cơ ra đời của CNN

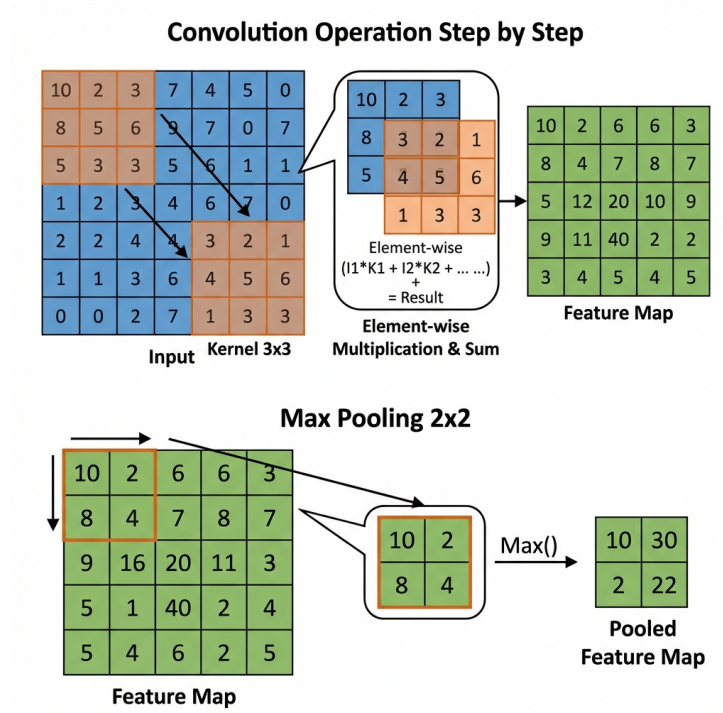
Như đã phân tích, MLP không phù hợp cho xử lý ảnh do không khai thác được cấu trúc không gian cục bộ. Trong khi đó, thị giác sinh học cho thấy não bộ xử lý thông tin thị giác theo cách phân cấp: các vùng thị giác sơ cấp phát hiện các đặc trưng đơn giản như cạnh và góc, sau đó các vùng cao hơn tổng hợp thành các khái niệm phức tạp hơn.

Năm 1998, Yann LeCun và cộng sự đề xuất kiến trúc LeNet-5 cho bài toán nhận dạng chữ số viết tay, đánh dấu sự ra đời của CNN hiện đại. CNN được thiết kế với ba ý tưởng cốt lõi:

- **Kết nối cục bộ (Local Connectivity):** Mỗi neuron chỉ kết nối với một vùng nhỏ của đầu vào thay vì toàn bộ, phù hợp với tính cục bộ của đặc trưng ảnh.
- **Chia sẻ tham số (Weight Sharing):** Cùng một bộ lọc được áp dụng trên toàn bộ ảnh, giảm đáng kể số tham số và tạo tính bất biến tịnh tiến.

- **Pooling:** Giảm kích thước không gian, tăng tính bất biến với biến đổi nhỏ và mở rộng trường nhìn (receptive field).

2.4.2 Các thành phần chính của CNN



Hình 2.6: Minh họa phép tích chập (Convolution) và Max Pooling trong CNN: bộ lọc 3×3 trượt qua ảnh đầu vào tạo ra feature map, sau đó Max Pooling 2×2 giảm kích thước không gian.

a) Lớp Tích chập (Convolution Layer)

Lớp tích chập là thành phần cốt lõi của CNN, thực hiện phép tích chập giữa đầu vào và các bộ lọc (filter/kernel) có thể học được. Mỗi bộ lọc trượt qua toàn bộ ảnh đầu vào, tính tích vô hướng tại mỗi vị trí để tạo ra bản đồ đặc trưng (feature map).

Với ảnh đầu vào I có kích thước $H \times W \times C$ và bộ lọc K có kích thước $k \times k \times C$, phép tích chập được định nghĩa:

$$(I * K)[i, j] = \sum_{m=0}^{k-1} \sum_{n=0}^{k-1} \sum_{c=0}^{C-1} I[i + m, j + n, c] \cdot K[m, n, c] + b \quad (2.5)$$

Các siêu tham số quan trọng của lớp tích chập:

- **Kích thước bộ lọc:** Thường là 3×3 hoặc 5×5
- **Bước nhảy (Stride):** Khoảng cách di chuyển của bộ lọc, stride lớn hơn làm giảm kích thước đầu ra
- **Padding:** Thêm viền 0 xung quanh ảnh để kiểm soát kích thước đầu ra

- **Số bộ lọc:** Quyết định số kênh của feature map đầu ra

b) Lớp Pooling

Lớp pooling thực hiện phép lấy mẫu xuống (downsampling) để giảm kích thước không gian của feature map, từ đó giảm số tham số và chi phí tính toán, đồng thời tăng tính bất biến với các biến đổi nhỏ.

Hai loại pooling phổ biến:

- **Max Pooling:** Lấy giá trị lớn nhất trong mỗi vùng, giữ lại đặc trưng nổi bật nhất
- **Average Pooling:** Lấy giá trị trung bình, cho kết quả mượt mà hơn

Ví dụ, Max Pooling với cửa sổ 2×2 và stride 2 sẽ giảm kích thước feature map đi một nửa theo mỗi chiều.

c) Lớp Kết nối Đầy đủ (Fully Connected Layer)

Sau các lớp tích chập và pooling, feature map được “làm phẳng” thành vector và đưa qua các lớp fully connected để thực hiện phân loại hoặc hồi quy. Các lớp này hoạt động như MLP, tổng hợp các đặc trưng đã trích xuất để đưa ra dự đoán cuối cùng.

2.4.3 Ưu điểm của CNN trong xử lý ảnh và video

- **Học đặc trưng tự động:** CNN tự động trích xuất các đặc trưng phân cấp từ thấp đến cao (cạnh $\rightarrow$ texture $\rightarrow$ bộ phận $\rightarrow$ đối tượng) mà không cần thiết kế thủ công.
- **Hiệu quả tham số:** Nhờ chia sẻ trọng số, CNN có ít tham số hơn nhiều so với MLP tương đương.
- **Bất biến tịnh tiến:** Cùng một bộ lọc áp dụng trên toàn ảnh giúp phát hiện đặc trưng bất kể vị trí.
- **Xử lý video tự nhiên:** CNN có thể mở rộng sang 3D-CNN để xử lý cả chiều thời gian trong video.

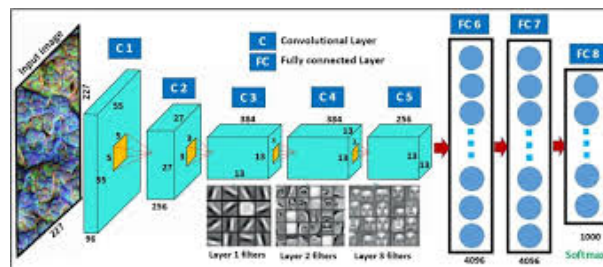
2.4.4 Các kiến trúc CNN tiêu biểu

a) AlexNet (2012)

AlexNet được Alex Krizhevsky và cộng sự phát triển, giành chiến thắng tại cuộc thi ImageNet 2012 với cách biệt lớn, đánh dấu sự bùng nổ của học sâu trong thị giác máy tính.

Đặc điểm chính của AlexNet:

- 5 lớp tích chập và 3 lớp fully connected, tổng cộng khoảng 60 triệu tham số
- Sử dụng hàm kích hoạt ReLU thay cho Sigmoid/Tanh, giúp huấn luyện nhanh hơn
- Áp dụng Dropout để giảm overfitting
- Huấn luyện trên 2 GPU song song lần đầu tiên



Hình 2.7: AlexNet

b) VGGNet (2014)

VGGNet do nhóm Visual Geometry Group tại Oxford phát triển, nổi bật với triết lý thiết kế đơn giản và đồng nhất: sử dụng các bộ lọc 3×3 xuyên suốt.

Kiến trúc VGG-16 bao gồm 13 lớp tích chập và 3 lớp fully connected. Ý tưởng chính là thay vì dùng bộ lọc lớn 7×7 , có thể xếp chồng nhiều bộ lọc 3×3 để đạt cùng receptive field nhưng với ít tham số hơn và thêm các lớp phi tuyến.



Hình 2.8: VGG

c) ResNet (2015)

ResNet (Residual Network) do nhóm Microsoft Research phát triển, giải quyết vấn đề nghiêm trọng của các mạng rất sâu: hiện tượng suy giảm gradient (vanishing gradient) và sự suy giảm độ chính xác.

Vấn đề: Khi mạng sâu hơn, lý thuyết cho rằng độ chính xác phải tốt lên hoặc ít nhất không kém đi (vì mạng sâu có thể “học” thành mạng nông). Tuy nhiên thực tế, mạng quá sâu có độ chính xác giảm cả trên tập huấn luyện lẫn kiểm tra.

Giải pháp - Residual Connection: Thay vì học ánh xạ $H(x)$ trực tiếp, ResNet học “phần dư” $F(x) = H(x) - x$ và đầu ra là:

$$y = F(x, \{W_i\}) + x \quad (2.6)$$

Kết nối tắt (skip connection) $+x$ cho phép gradient lan truyền trực tiếp qua các lớp, giải quyết vanishing gradient. Nếu $F(x)$ không cần thiết, mạng có thể dễ dàng học $F(x) \approx 0$ để trở thành hàm đồng nhất.

ResNet cho phép huấn luyện mạng với hàng trăm thậm chí hàng nghìn lớp (ResNet-152, ResNet-1001), đạt kết quả vượt trội trên nhiều benchmark và trở thành nền tảng cho nhiều kiến trúc hiện đại.

2.5 Giới thiệu YOLO (You Only Look Once)

2.5.1 Bài toán Object Detection

Object Detection (Phát hiện đối tượng) là một trong những bài toán cơ bản và quan trọng nhất trong thị giác máy tính [12]. Khác với Image Classification chỉ xác định loại đối tượng chính trong ảnh, Object Detection yêu cầu đồng thời:

- **Định vị (Localization):** Xác định vị trí của đối tượng trong ảnh thông qua bounding box (hộp giới hạn)
- **Phân loại (Classification):** Xác định loại của mỗi đối tượng được phát hiện

Trong bài toán phát hiện ngã, Object Detection đóng vai trò then chốt: mô hình cần xác định vị trí con người trong khung hình và phân loại trạng thái của họ (đang đứng, đi lại, hay đã ngã).

2.5.2 Lịch sử phát triển Object Detection

Theo khảo sát “Object Detection in 20 Years” của Zou và cộng sự [12], lĩnh vực Object Detection đã trải qua sự phát triển đáng kể trong hơn hai thập kỷ:

a) Giai đoạn truyền thống (1990s-2012)

- **Viola-Jones Detector (2001):** Detector đầu tiên đạt tốc độ real-time cho face detection, sử dụng Haar-like features và AdaBoost

- **HOG Detector (2005)**: Histogram of Oriented Gradients cho pedestrian detection
- **DPM - Deformable Parts Model (2010)**: Mô hình biến dạng các phần, đạt SOTA trên PASCAL VOC nhiều năm

b) Giai đoạn Deep Learning (2012-nay)

- **AlexNet (2012)**: Bước ngoặt của deep learning trong computer vision
- **R-CNN (2014)**: Kết hợp region proposals với CNN features
- **Fast R-CNN, Faster R-CNN (2015)**: Cải tiến tốc độ với RoI pooling và Region Proposal Network
- **YOLO v1 (2016)**: Đột phá với one-stage detection, real-time
- **SSD (2016)**: Single Shot MultiBox Detector với multi-scale features
- **RetinaNet (2017)**: Giới thiệu Focal Loss để giải quyết class imbalance
- **YOLO v3-v11 (2018-2024)**: Liên tục cải tiến về accuracy và speed

c) Xu hướng hiện tại

Theo [12], các xu hướng chính trong Object Detection bao gồm:

- Anchor-free detectors (loại bỏ anchor boxes)
- Transformer-based detectors (DETR, Swin Transformer)
- NAS-based architecture search (tự động tìm kiếm kiến trúc)
- Real-time detection trên edge devices

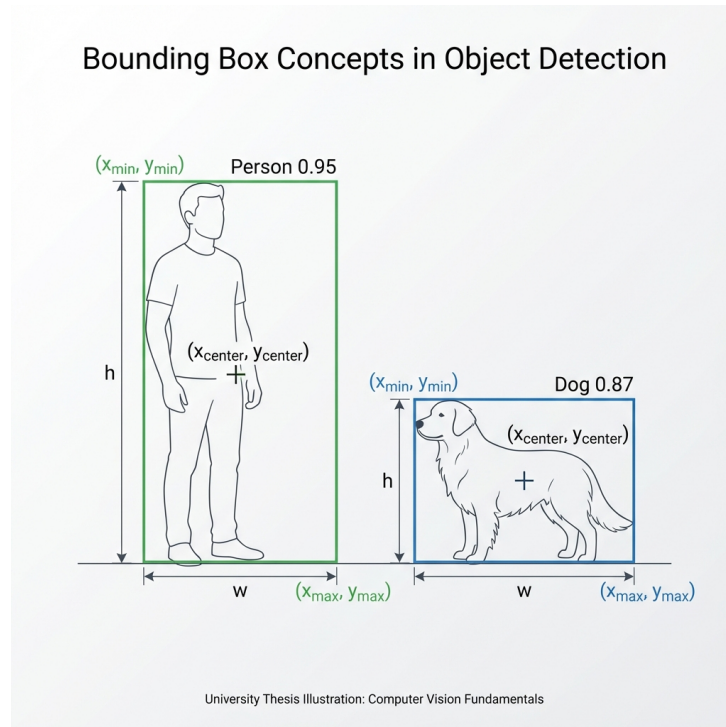
2.5.3 Khái niệm Bounding Box

Bounding Box (hộp giới hạn) là hình chữ nhật bao quanh đối tượng trong ảnh, được sử dụng để xác định vị trí và kích thước của đối tượng.

Có hai cách biểu diễn bounding box phổ biến:

a) Format $(x_{min}, y_{min}, x_{max}, y_{max})$ - PASCAL VOC format

- (x_{min}, y_{min}) : Tọa độ góc trên bên trái của box



Hình 2.9: Ví dụ minh họa Bounding Box trong Object Detection.

- (x_{max}, y_{max}) : Tọa độ góc dưới bên phải của box
- Đơn vị: pixel tuyệt đối

b) Format $(x_{center}, y_{center}, w, h)$ - YOLO format

- (x_{center}, y_{center}) : Tọa độ tâm của box
- w : Chiều rộng của box
- h : Chiều cao của box
- Đơn vị: normalized (chia cho kích thước ảnh, giá trị từ 0 đến 1)

c) Công thức chuyển đổi

Từ PASCAL VOC sang YOLO (với ảnh kích thước $W \times H$):

$$x_{center} = \frac{x_{min} + x_{max}}{2W}, \quad y_{center} = \frac{y_{min} + y_{max}}{2H} \quad (2.7)$$

$$w = \frac{x_{max} - x_{min}}{W}, \quad h = \frac{y_{max} - y_{min}}{H} \quad (2.8)$$

2.5.4 Intersection over Union (IoU)

IoU là metric đo độ chồng lấp giữa hai bounding box, được sử dụng rộng rãi trong đánh giá object detection:

$$IoU = \frac{\text{Area of Intersection}}{\text{Area of Union}} = \frac{|A \cap B|}{|A \cup B|} \quad (2.9)$$

Trong đó:

- A : Bounding box dự đoán (predicted)
- B : Bounding box ground truth (thực tế)
- $IoU = 1$: Hai box trùng khớp hoàn toàn
- $IoU = 0$: Hai box không có phần giao nhau

Ngưỡng IoU thường được sử dụng để xác định dự đoán đúng:

- $IoU \geq 0.5$: Ngưỡng phổ biến (PASCAL VOC, COCO mAP@0.5)
- $IoU \geq 0.75$: Ngưỡng nghiêm ngặt (COCO mAP@0.75)
- IoU từ 0.5 đến 0.95: Đánh giá đa ngưỡng (COCO mAP@[.5:.95])

2.5.5 Các khái niệm True Positive, False Positive, False Negative

Để đánh giá kết quả detection, cần phân loại các dự đoán:

- **True Positive (TP)**: Dự đoán đúng - có bounding box dự đoán khớp với ground truth ($IoU \geq \text{threshold}$) và đúng class
- **False Positive (FP)**: Dự đoán sai - bounding box dự đoán không khớp với bất kỳ ground truth nào, hoặc sai class
- **False Negative (FN)**: Bỏ sót - ground truth không được detect bởi bất kỳ dự đoán nào

Từ đó tính được:

$$\text{Precision} = \frac{TP}{TP + FP} = \frac{\text{Số dự đoán đúng}}{\text{Tổng số dự đoán}} \quad (2.10)$$

$$\text{Recall} = \frac{TP}{TP + FN} = \frac{\text{Số dự đoán đúng}}{\text{Tổng số ground truth}} \quad (2.11)$$

2.5.6 Đánh giá hiệu suất: Mean Average Precision (mAP)

Mean Average Precision (mAP) là metric chuẩn để đánh giá hiệu suất của mô hình object detection, tổng hợp cả precision và recall.

a) Precision-Recall Curve

Với mỗi class, sắp xếp các dự đoán theo confidence score giảm dần. Tại mỗi điểm, tính precision và recall tích lũy, từ đó vẽ được đường cong Precision-Recall.

b) Average Precision (AP)

AP là diện tích dưới đường cong Precision-Recall:

$$AP = \int_0^1 p(r) dr \quad (2.12)$$

Trong thực tế, sử dụng phép nội suy (interpolation):

$$AP = \sum_{i=1}^n (r_i - r_{i-1}) \cdot p_{interp}(r_i) \quad (2.13)$$

với $p_{interp}(r) = \max_{r' \geq r} p(r')$ là precision nội suy tại recall r .

c) Mean Average Precision (mAP)

mAP là trung bình của AP trên tất cả các class:

$$mAP = \frac{1}{N} \sum_{i=1}^N AP_i \quad (2.14)$$

với N là số class trong dataset.

d) Các biến thể mAP

- **mAP@0.5 (mAP50):** AP tính với ngưỡng IoU = 0.5, được sử dụng trong PASCAL VOC
- **mAP@0.75 (mAP75):** AP với ngưỡng IoU = 0.75, nghiêm ngặt hơn
- **mAP@[.5:.95]:** Trung bình AP với IoU từ 0.5 đến 0.95 (bước 0.05), metric chính

của COCO

- **mAP Small/Medium/Large:** mAP theo kích thước đối tượng

e) Ý nghĩa của mAP

- mAP cao $\rightarrow$ Mô hình phát hiện đúng nhiều đối tượng và định vị chính xác
- mAP50 cao nhưng mAP75 thấp $\rightarrow$ Phát hiện được nhưng định vị chưa chính xác
- Precision cao, Recall thấp $\rightarrow$ Mô hình thận trọng, ít false positive nhưng bỏ sót nhiều
- Precision thấp, Recall cao $\rightarrow$ Mô hình phát hiện nhiều nhưng có nhiều false positive

2.5.7 Ý tưởng YOLO: Phát hiện trong một lần suy luận

YOLO (You Only Look Once) được Joseph Redmon, Santosh Divvala, Ross Girshick và Ali Farhadi đề xuất năm 2016 trong bài báo “You Only Look Once: Unified, Real-Time Object Detection” [11]. Đây là bước đột phá mang đến cách tiếp cận hợp nhất cho Object Detection.

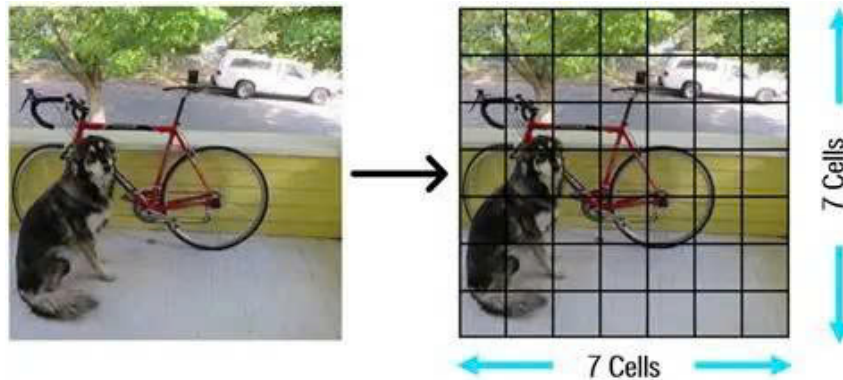
Ý tưởng cốt lõi của YOLO (theo [11]):

“We reframe object detection as a single regression problem, straight from image pixels to bounding box coordinates and class probabilities. Using our system, you only look once (YOLO) at an image to predict what objects are present and where they are.”

Thay vì coi detection như bài toán phân loại (classifier), YOLO đặt lại vấn đề như bài toán hồi quy về tọa độ bounding box và xác suất class, tất cả trong một lần forward pass duy nhất qua mạng neural.

Nguyên lý hoạt động của YOLO:

1. **Chia lưới:** Ảnh đầu vào được chia thành lưới $S \times S$ ô (grid cells). Trong paper gốc, $S = 7$.
2. **Dự đoán bounding boxes:** Mỗi ô dự đoán B bounding boxes (mặc định $B = 2$) cùng với điểm confidence.
3. **Dự đoán class probabilities:** Mỗi ô dự đoán C xác suất class có điều kiện $Pr(Class_i|Object)$.



Hình 2.10: Minh họa chia lưới 7×7 trong YOLO: mỗi ô chịu trách nhiệm phát hiện đối tượng có tâm nằm trong ô đó.

4. **Tensor đầu ra:** Kích thước $S \times S \times (B \times 5 + C)$. Với PASCAL VOC ($C = 20$), tensor có kích thước $7 \times 7 \times 30$.
5. **Post-processing:** Non-Maximum Suppression (NMS) loại bỏ các box trùng lặp.

Đầu ra của mỗi bounding box bao gồm 5 giá trị:

- (x, y) : Tọa độ tâm của box tương đối với ô lưới (normalized 0-1 trong ô)
- (w, h) : Chiều rộng và chiều cao của box tương đối với toàn ảnh (normalized 0-1)
- $confidence = Pr(Object) \times IOU_{pred}^{truth}$: Độ tin cậy rằng box chứa đối tượng VÀ độ chính xác của box so với ground truth

Hiệu suất của YOLO theo bài báo gốc:

- Base YOLO: 45 FPS (frames per second) trên GPU
- Fast YOLO: 155 FPS - nhanh hơn gấp đôi các detector real-time khác
- mAP trên PASCAL VOC 2007: 63.4% (base), 52.7% (fast)

2.5.8 So sánh YOLO với phương pháp truyền thống

Ưu điểm của YOLO:

- Tốc độ: Xử lý real-time với hàng chục đến hàng trăm FPS
- Đơn giản: Một mạng duy nhất, dễ huấn luyện end-to-end

Bảng 2.2: So sánh YOLO với các phương pháp Object Detection khác

Phương pháp	Tốc độ (FPS)	mAP	Thời gian thực
R-CNN	0.02	53.7	Không
Fast R-CNN	0.5	70.0	Không
Faster R-CNN	7-17	73.2	Khó
YOLO v1	45	63.4	Có
YOLOv3	30-50	57.9	Có
YOLOv5	100+	68.9	Có

- Nhìn toàn cục: Xem xét toàn bộ ảnh khi dự đoán, hiểu được ngữ cảnh tốt hơn
- Tổng quát hóa tốt: Ít false positive trên background so với các phương pháp dựa trên sliding window

Hạn chế:

- Khó phát hiện các đối tượng nhỏ hoặc xếp chồng nhau
- Độ chính xác định vị có thể kém hơn phương pháp hai giai đoạn ở một số trường hợp

2.6 Khảo sát YOLOv11 cho bài toán Fall Detection

2.6.1 Giới thiệu YOLOv11

Tại giai đoạn khảo sát, YOLOv11 là một ứng viên trong họ YOLO và được phân tích trong nghiên cứu “YOLOv11: An Overview of the Key Architectural Enhancements” của Rahima Khanam và Muhammad Hussain [27]. Phiên bản được công bố trong hệ sinh thái Ultralytics năm 2024, kế thừa hướng phát triển của các phiên bản trước. Artifact triển khai cuối của đề án được ghi theo đúng tên tệp trong Chương 4, không đồng nhất tên khảo sát với tên model đã đóng gói.

Theo [27], các cải tiến kiến trúc chính của YOLOv11 bao gồm:

- **C3k2 Block (Cross Stage Partial with kernel size 2):** Module convolution mới thay thế C3 trong YOLOv8, sử dụng kernel size nhỏ hơn để tăng tốc độ inference trong khi vẫn giữ hiệu suất feature extraction.
- **SPPF (Spatial Pyramid Pooling - Fast):** Module pooling nhanh hơn SPP truyền thống, cho phép trích xuất đặc trưng đa tỉ lệ hiệu quả.

- **C2PSA (Convolutional block with Parallel Spatial Attention):** Module attention mới giúp mô hình tập trung vào các vùng quan trọng trong ảnh, cải thiện khả năng phát hiện đối tượng trong bối cảnh phức tạp.
- **Neck với PANet cải tiến:** Path Aggregation Network kết hợp đặc trưng từ nhiều mức (multi-scale feature fusion).
- **Head anchor-free:** Loại bỏ anchor boxes, sử dụng dự đoán trực tiếp tọa độ, giảm hyperparameter cần tinh chỉnh.

YOLOv11 hỗ trợ đa nhiệm vụ trong cùng một framework: Object Detection, Instance Segmentation, Pose Estimation, Oriented Object Detection (OBB), và Classification.

2.6.2 Các phiên bản mô hình YOLOv11

Theo [27], YOLOv11 cung cấp nhiều kích thước mô hình phù hợp với các nhu cầu khác nhau:

Model	Params (M)	mAP@50-95	Latency (ms)	Ứng dụng
YOLOv11n	2.6	39.5	1.55	Đề xuất - Edge/Mobile
YOLOv11s	9.4	47.0	2.46	Cân bằng tốc độ/accuracy
YOLOv11m	20.1	51.5	4.70	Desktop GPU
YOLOv11l	25.3	53.4	6.16	High accuracy
YOLOv11x	56.9	54.7	11.31	Maximum accuracy

Nguồn: Ultralytics YOLO11 Documentation, 2024

2.7 Theo dõi và Nhận diện Người (Person Tracking)

2.7.1 Giới thiệu Person Tracking

Person Tracking (Theo dõi người) là bài toán xác định và theo dõi liên tục vị trí của người trong video qua các khung hình liên tiếp. Khác với Object Detection chỉ phát hiện người trong từng frame độc lập, Person Tracking duy trì identity của từng người xuyên suốt video.

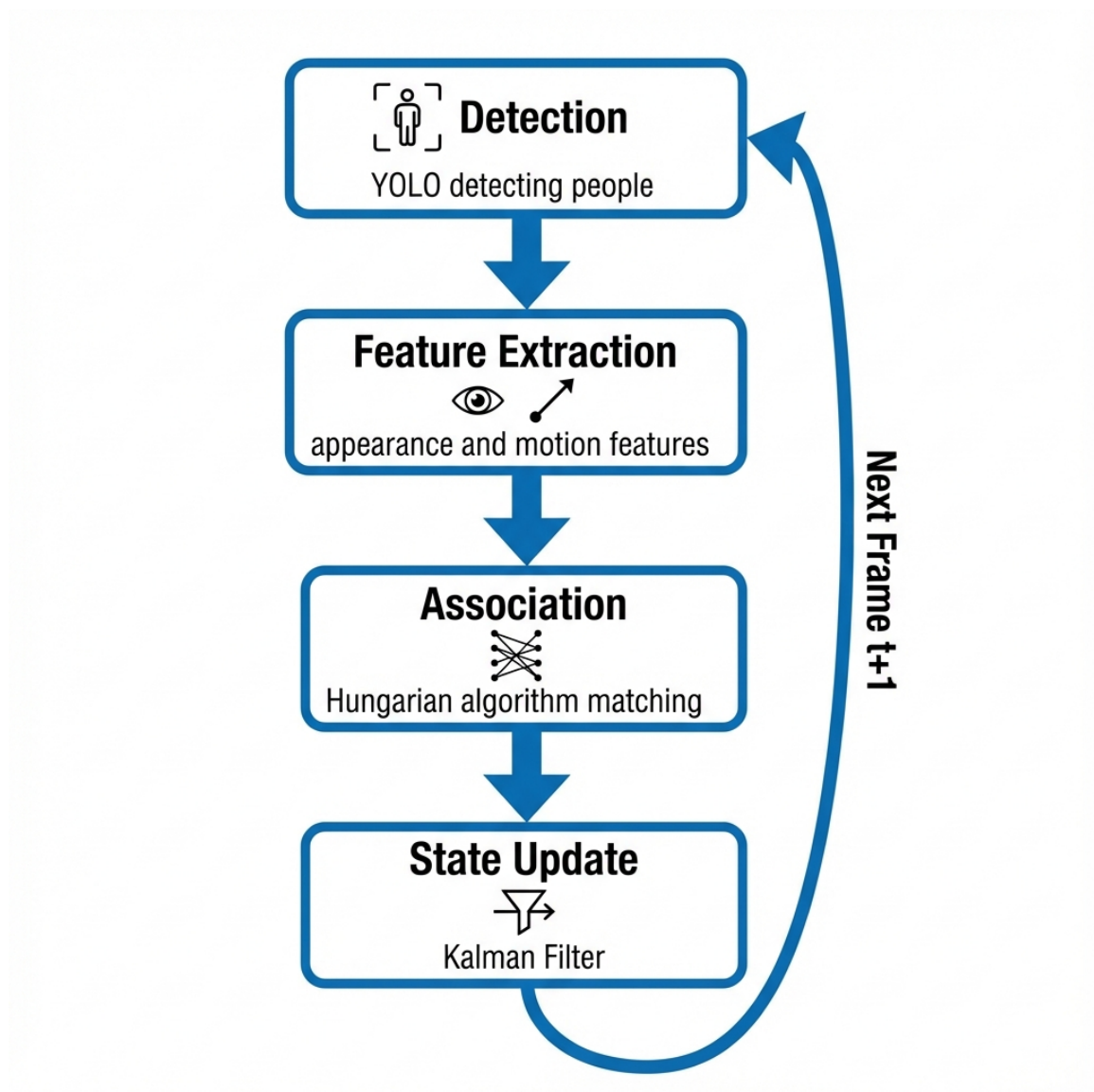
Yêu cầu của hệ thống Person Tracking:

- **Detection:** Phát hiện người trong mỗi frame

- **Association:** Liên kết các detection giữa các frame
- **ID Management:** Gán và duy trì ID duy nhất cho mỗi người
- **Re-identification:** Nhận diện lại người sau khi bị che khuất

2.7.2 Tracking-by-Detection Paradigm

Phương pháp Tracking-by-Detection là hướng tiếp cận phổ biến nhất, hoạt động theo pipeline 4 bước như minh họa trong Hình 2.11.



Hình 2.11: Flowchart pipeline Tracking-by-Detection: Detection → Feature Extraction → Association → State Update, lặp lại cho mỗi frame.

- **Detection:** Sử dụng detector (YOLO, Faster R-CNN) phát hiện người trong mỗi frame
- **Feature Extraction:** Trích xuất đặc trưng appearance và motion cho mỗi detection
- **Association:** So khớp detection với các track đang hoạt động (Hungarian Algorithm)
- **State Update:** Cập nhật trạng thái track bằng Kalman Filter (vị trí, velocity)

2.8 Giới thiệu Dataset Fall Detection – CauCaFall

2.8.1 Mục tiêu và đặc điểm của dataset CauCaFall

CauCaFall là bộ dữ liệu chuyên biệt cho bài toán phát hiện ngã trong môi trường ít kiểm soát, cung cấp video, nhãn fall/no-fall và thông tin về góc, khoảng cách, độ chiếu sáng [25].

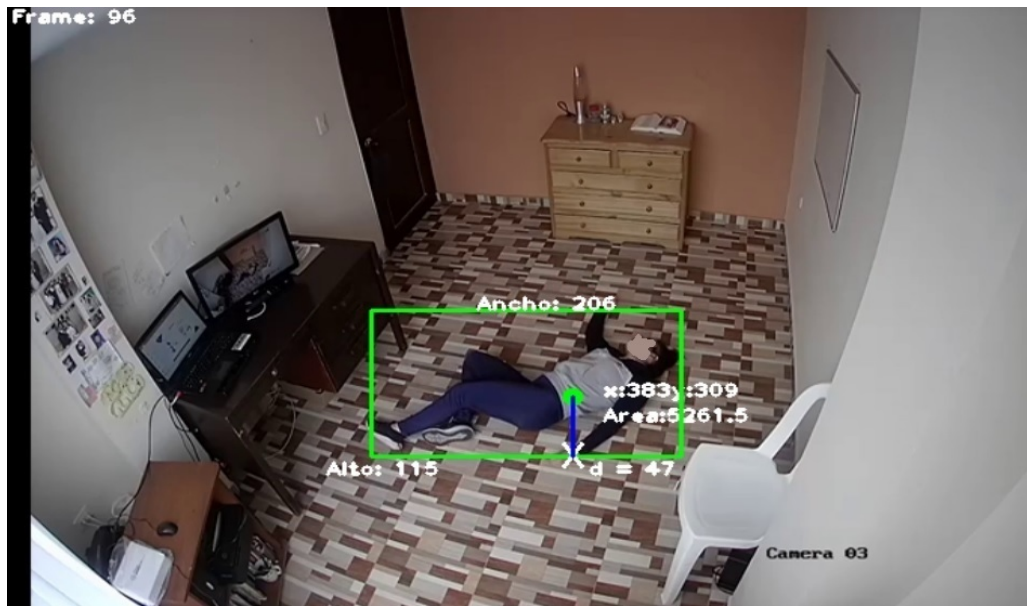
Đặc điểm nổi bật của dataset CauCaFall:

- **Môi trường trong nhà thực tế:** Dữ liệu được thu thập trong các không gian sống thực như phòng khách, phòng ngủ, hành lang, phản ánh đúng môi trường triển khai thực tế của hệ thống giám sát
- **Đa dạng góc quay:** Camera được đặt ở nhiều vị trí và góc độ khác nhau, mô phỏng các thiết lập giám sát phổ biến
- **Nhiều kịch bản ngã:** Bao gồm các tình huống ngã đa dạng từ trượt ngã, ngã do mất thăng bằng, đến ngã từ ghế/giường
- **Đa dạng đối tượng:** Nhiều người tham gia với độ tuổi, giới tính, và trang phục khác nhau

2.8.2 Dữ liệu video trong nhà

Dataset CauCaFall chủ yếu bao gồm các video được quay trong môi trường trong nhà với các đặc điểm kỹ thuật:

- Độ phân giải video đa dạng từ 720p đến 1080p
- Frame rate tiêu chuẩn 25-30 FPS



- Điều kiện ánh sáng đa dạng: ánh sáng tự nhiên ban ngày, đèn nhân tạo ban đêm
- Background bao gồm các đồ vật trong nhà phổ biến: bàn, ghế, giường, tủ

Video được cắt thành các clip ngắn, mỗi clip chứa một sự kiện cụ thể (ngã hoặc không ngã), thuận tiện cho việc huấn luyện và đánh giá mô hình.

2.8.3 Annotation ban đầu

Dataset CauCaFall gốc được gán nhãn với hai lớp cơ bản:

- **fall (1)**: Đánh dấu các khung hình hoặc video clip có người đang trong trạng thái ngã hoặc vừa ngã
- **no_fall (0)**: Đánh dấu các hoạt động bình thường như đi bộ, ngồi, đứng, nằm nghỉ ngơi (không phải do ngã)

Annotation được thực hiện ở mức bounding box cho từng frame, xác định vị trí của người trong ảnh kèm theo nhãn trạng thái.

2.8.4 Ưu điểm và hạn chế của dataset CauCaFall

Ưu điểm:

- Dữ liệu thực tế, phản ánh đúng môi trường ứng dụng
- Chất lượng video tốt, phù hợp cho các mô hình deep learning

- Có diversity về tình huống ngã và đối tượng
- Được gán nhãn cẩn thận ở mức bounding box

Hạn chế:

- Chỉ có hai lớp **fall/no_fall**, thiếu thông tin về các đối tượng khác trong cảnh
- Số lượng mẫu hạn chế so với các dataset lớn như COCO
- Không có annotation cho indoor objects (bàn, ghế, giường) - thông tin quan trọng cho việc hiểu ngữ cảnh
- Class imbalance: số mẫu no_fall thường nhiều hơn fall

2.8.5 Nghiên cứu liên quan: Dataset DiverseFALL10500

Nghiên cứu của Habib Khan và cộng sự (2024) “Visionary vigilance: Optimized YOLOV8 for fallen person detection with large-scale benchmark dataset” [26] đã đề xuất một dataset quy mô lớn cho bài toán Fall Detection:

DiverseFALL10500 Dataset:

- **Quy mô:** Khoảng 10,500 ảnh với annotations tương ứng
- **Đa dạng môi trường:** Indoor và outdoor scenes
- **Điều kiện ánh sáng:** Nhiều mức độ chiếu sáng khác nhau (illumination variations)
- **Đa dạng đối tượng:** Samples từ nhiều độ tuổi khác nhau
- **Góc quay đa dạng**
- **Mã nguồn mở:** công bố tại [kho mã DiverseFALL10500](#)

Theo [26], các dataset Fall Detection hiện có thường gặp hạn chế về:

- Thiếu đa dạng trong samples (môi trường đồng nhất)
- Hạn chế về biểu hiện thị giác của các nhóm tuổi khác nhau
- Thiếu coverage về điều kiện môi trường và ánh sáng
- Hạn chế về đa dạng trang phục

So sánh với CauCaFall:

Tiêu chí	CauCaFall	DiverseFALL10500
Số lượng samples	3,000 frames	10,500 images
Môi trường	Indoor only	Indoor + Outdoor
Điều kiện ánh sáng	Hạn chế	Đa dạng
Nhóm tuổi	Hạn chế	Đa dạng
Góc camera	Cố định	Đa dạng

Đồ án này sử dụng CauCaFall do tính sẵn có và phù hợp với môi trường trong nhà (indoor surveillance), đồng thời kết hợp với COCO để tăng cường đa dạng và annotation cho indoor objects.

2.9 Hệ thống AI Agent và Mô hình Ngôn ngữ Lớn (LLM)

2.9.1 Khái niệm AI Agent

AI Agent (Tác tử trí tuệ nhân tạo) là một hệ thống tự động có khả năng nhận thức môi trường, đưa ra quyết định, và thực hiện hành động để đạt được mục tiêu cụ thể. Khác với các chương trình truyền thống hoạt động theo luật cố định, AI Agent có khả năng:

- **Tự chủ (Autonomy):** Hoạt động độc lập mà không cần sự can thiệp liên tục của con người
- **Phản ứng (Reactivity):** Nhận thức và phản hồi kịp thời với thay đổi của môi trường
- **Chủ động (Proactivity):** Khởi xướng hành động để đạt mục tiêu, không chỉ phản ứng thụ động
- **Xã hội (Social ability):** Tương tác và giao tiếp với con người hoặc các agent khác

Trong hệ thống robot giám sát người cao tuổi, AI Agent đóng vai trò là “bộ não” thông minh, điều phối các thành phần khác nhau (camera, cảm biến, IoT) và giao tiếp tự nhiên với người dùng.

2.9.2 Mô hình Ngôn ngữ Lớn (Large Language Models)

Large Language Models (LLM) là các mô hình học sâu được huấn luyện trên lượng dữ liệu văn bản khổng lồ, có khả năng hiểu và sinh ngôn ngữ tự nhiên ở mức độ cao. LLM đã tạo ra bước đột phá trong AI conversational.

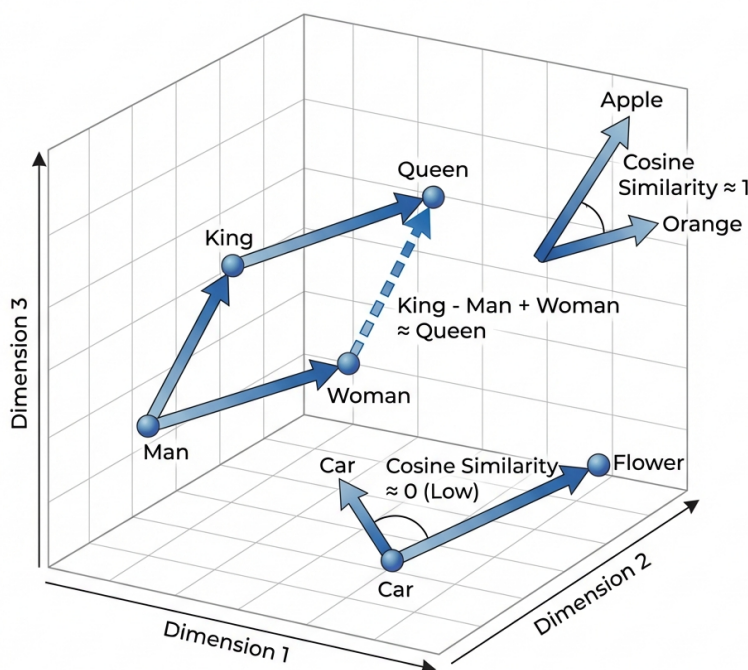
a) Kiến trúc Transformer

LLM hiện đại dựa trên kiến trúc Transformer được giới thiệu trong bài báo “Attention Is All You Need” (Vaswani et al., 2017). Transformer đã tạo ra một cuộc cách mạng trong xử lý ngôn ngữ tự nhiên bằng việc loại bỏ hoàn toàn các cấu trúc hồi quy (RNN, LSTM) và thay thế bằng cơ chế self-attention.

Lớp Embedding và Ánh xạ Không gian Đa chiều

Trước khi dữ liệu được đưa vào Transformer để huấn luyện, mỗi token (từ, ký tự, hoặc subword) cần phải đi qua lớp Embedding. Lớp này có chức năng ánh xạ dữ liệu từ không gian rời rạc (discrete space) sang không gian vector liên tục đa chiều (continuous high-dimensional space).

WORD EMBEDDING CONCEPT IN NLP



Hình 2.12: Minh họa không gian vector embedding và độ tương đồng cosine giữa các từ

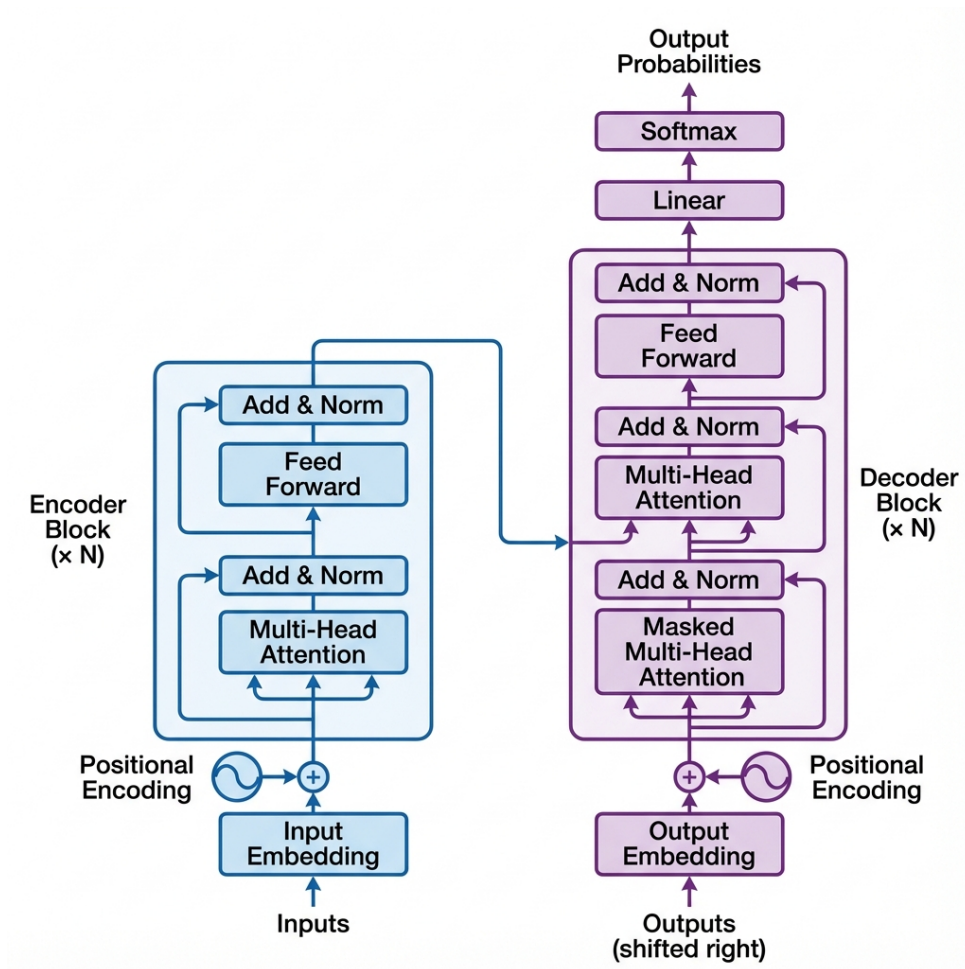
Trong không gian embedding đa chiều (thường từ 512 đến 4096 chiều), các từ có ngữ nghĩa tương đồng sẽ có vector gần nhau. Độ tương đồng giữa hai vector thường được đo bằng cosine similarity:

$$\text{cosine_similarity}(\mathbf{a}, \mathbf{b}) = \frac{\mathbf{a} \cdot \mathbf{b}}{\|\mathbf{a}\| \cdot \|\mathbf{b}\|} = \frac{\sum_{i=1}^n a_i b_i}{\sqrt{\sum_{i=1}^n a_i^2} \cdot \sqrt{\sum_{i=1}^n b_i^2}} \quad (2.15)$$

Việc embedding giúp “làm dày” ngữ cảnh của mỗi data point, cho phép mô hình học được các mối quan hệ ngữ nghĩa phức tạp như:

- $\text{vector}(\text{“King”}) - \text{vector}(\text{“Man”}) + \text{vector}(\text{“Woman”}) \approx \text{vector}(\text{“Queen”})$
- Các từ đồng nghĩa có vector gần nhau trong không gian embedding
- Các từ trong cùng ngữ cảnh có cosine similarity cao

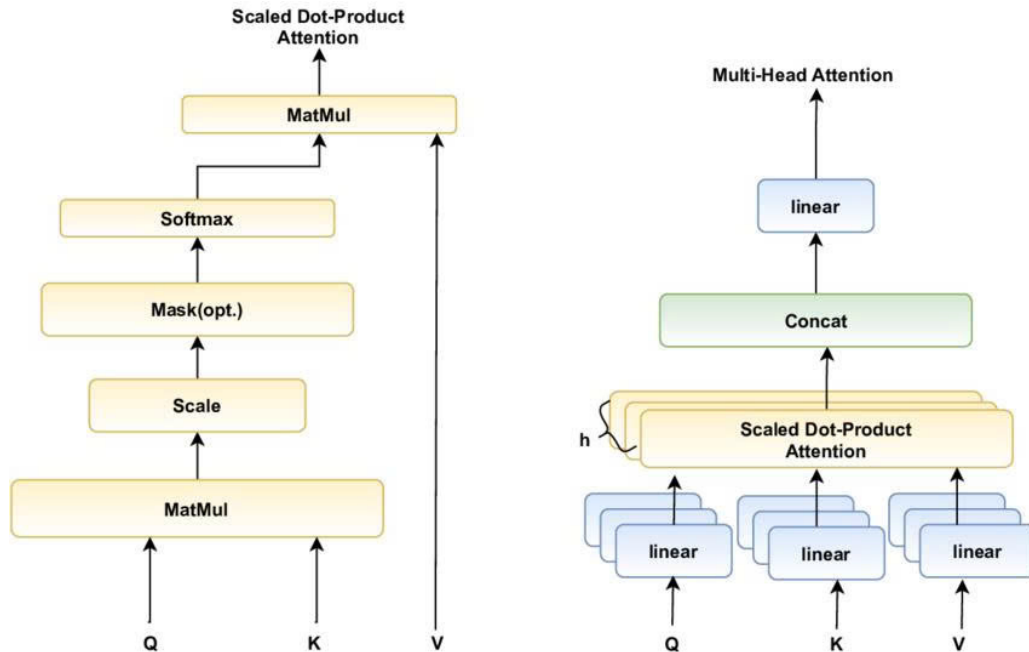
Cơ chế Self-Attention



Hình 2.13: Kiến trúc Transformer với các khối Encoder và Decoder

Self-Attention cho phép mỗi vị trí trong chuỗi đầu vào “chú ý” đến tất cả các vị trí khác, học được mối quan hệ giữa các từ bất kể khoảng cách. Cơ chế này hoạt động thông qua ba ma trận: Query (Q), Key (K), và Value (V):

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \quad (2.16)$$



Hình 2.14: Cơ chế Self-Attention: Query, Key, Value và Scaled Dot-Product Attention

Trong đó:

- Q (Query): Biểu diễn “câu hỏi” - token hiện tại muốn tìm thông tin gì
- K (Key): Biểu diễn “chìa khóa” - các token khác chứa thông tin gì
- V (Value): Biểu diễn “giá trị” - nội dung thực sự của các token
- d_k : Chiều của vector key, dùng để scale gradient ổn định

Multi-Head Attention mở rộng cơ chế này bằng cách chạy song song nhiều attention head, mỗi head học các loại quan hệ khác nhau:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O \quad (2.17)$$

Trong đó mỗi head được tính toán độc lập: $\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$

Ưu điểm của Multi-Head Attention:

- **Đa dạng hóa biểu diễn:** Mỗi head có thể học các loại quan hệ khác nhau (ví dụ: head 1 học quan hệ cú pháp, head 2 học quan hệ ngữ nghĩa, head 3 học quan hệ vị trí).

- **Tối ưu hóa song song trên GPU:** Việc chia thành nhiều head độc lập cho phép tận dụng tối đa khả năng tính toán song song của GPU. Thay vì thực hiện một phép attention lớn với chiều d_{model} , ta thực hiện h phép attention nhỏ với chiều $d_k = d_{model}/h$ song song. Điều này phù hợp hoàn hảo với kiến trúc SIMD (Single Instruction Multiple Data) của GPU, giúp:
 - Tăng tốc độ training đáng kể nhờ batch matrix multiplication
 - Tối ưu hóa throughput trong inference
 - Sử dụng hiệu quả băng thông bộ nhớ GPU (memory bandwidth)
 - Cho phép scale lên nhiều GPU dễ dàng hơn với tensor parallelism
- **Giảm chi phí tính toán:** Tổng chi phí tính toán tương đương với single-head attention có cùng chiều d_{model} , nhưng hiệu quả hơn nhờ song song hóa.

So sánh Transformer và CNN: Global Context vs Local Context

Một trong những điểm mấu chốt của Transformer so với CNN là cách nắm bắt ngữ cảnh:

Bảng 2.3: So sánh Transformer và CNN

Đặc điểm	Transformer	CNN
Ngữ cảnh	Global Context - Mỗi token có thể attention đến tất cả các token khác trong chuỗi	Local Context - Chỉ nhìn thấy các phần tử trong receptive field (kernel window)
Inductive Bias	Ít inductive bias → Cần lượng dữ liệu rất lớn để học	Có inductive bias mạnh về locality và translation equivariance
Dữ liệu cần	Yêu cầu dataset khổng lồ (hàng trăm GB văn bản)	Có thể hoạt động tốt với dataset nhỏ hơn
Phức tạp	$O(n^2)$ với độ dài chuỗi n	$O(n \cdot k^2)$ với kernel size k
Ưu điểm	Học được long-range dependencies tốt	Hiệu quả với dữ liệu có cấu trúc không gian (ảnh)

Inductive Bias: Transformer có rất ít giả định tiên nghiệm về cấu trúc dữ liệu (minimal inductive bias). Điều này có nghĩa là mô hình phải học toàn bộ ngữ cảnh và các pattern từ dữ liệu, thay vì có sẵn các “bias” như locality trong CNN. Hệ quả là Transformer cần lượng dữ liệu huấn luyện cực lớn để đạt được hiệu suất tốt.

Kiến trúc Encoder-Decoder và Decoder-Only

Transformer gốc bao gồm cả khối Encoder và Decoder:

- **Encoder:** Xử lý chuỗi đầu vào, tạo biểu diễn ngữ cảnh. Sử dụng bidirectional attention (nhìn cả trái và phải).
- **Decoder:** Sinh chuỗi đầu ra từng token một, sử dụng cross-attention để tham chiếu đến encoder output.

Tuy nhiên, các mô hình sinh văn bản như GPT chỉ sử dụng **Decoder-Only** architecture với cơ chế **Masked Self-Attention** (hay Causal Attention):

$$\text{Masked-Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} + M \right) V \quad (2.18)$$

Trong đó M là ma trận mask với $M_{ij} = -\infty$ nếu $j > i$ (các vị trí tương lai bị che). Điều này đảm bảo:

- Mỗi token chỉ attend được đến các token trước nó (autoregressive)
- Tránh “học vẹt” (data leakage) - mô hình không thể nhìn thấy câu trả lời khi đang dự đoán
- Phù hợp cho các tác vụ sinh văn bản (text generation)

Các Khối Phụ trợ trong Transformer

- **Layer Normalization:** Khác với Batch Normalization trong CNN, Layer Norm chuẩn hóa theo chiều feature thay vì theo batch. Điều này phù hợp hơn cho sequence data có độ dài thay đổi:

$$\text{LayerNorm}(x) = \gamma \cdot \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}} + \beta \quad (2.19)$$

- **Residual Connection:** Skip connection giúp gradient flow tốt hơn và cho phép đào tạo các mạng rất sâu.
- **Feed-Forward Network (FFN):** Mỗi layer có một FFN với hidden dimension lớn (thường 4x embedding dimension):

$$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2 \quad (2.20)$$

- **Positional Encoding:** Vì Transformer không có khái niệm về thứ tự, positional encoding được thêm vào để mã hóa vị trí của các token trong chuỗi.

b) Các mô hình LLM tiêu biểu và các bước ngoặt

Sự phát triển của LLM đã trải qua nhiều bước ngoặt quan trọng:

Các bước ngoặt trong lịch sử LLM:

1. **GPT-1 (2018):** OpenAI giới thiệu khái niệm pre-training + fine-tuning, chứng minh unsupervised pre-training có thể cải thiện NLP tasks.
2. **BERT (2018):** Google đề xuất bidirectional pre-training với masked language modeling, tạo bước đột phá trong NLU (Natural Language Understanding).
3. **GPT-3 (2020):** Với 175 tỷ tham số, GPT-3 chứng minh khả năng few-shot learning mà không cần fine-tuning, mở ra kỷ nguyên prompt engineering.
4. **ChatGPT (2022):** Sử dụng RLHF (Reinforcement Learning from Human Feedback) để align với mong muốn người dùng, tạo AI conversational thực sự hữu ích.
5. **GPT-4 (2023):** Mô hình đa phương thức (multimodal) có thể xử lý cả văn bản và hình ảnh, reasoning ability được cải thiện đáng kể.
6. **Open-source revolution (2023-2024):** LLaMA (Meta), Mistral, Qwen mở cửa cho cộng đồng nghiên cứu và triển khai local.
7. **DeepSeek (2024):** Bước ngoặt quan trọng từ Trung Quốc, DeepSeek-V3 đạt hiệu suất cạnh tranh với GPT-4 với chi phí huấn luyện thấp hơn đáng kể (chỉ \$5.6M), chứng minh hiệu quả của kiến trúc Mixture of Experts (MoE) và kỹ thuật Multi-head Latent Attention (MLA).

Bảng so sánh các mô hình LLM tiêu biểu:

Mô hình	Nhà phát triển	Tham số	Open-source	Đa phương thức	Đặc điểm nổi bật
GPT-4o	OpenAI	N/A	Không	Có	SOTA reasoning, API phổ biến
Gemini 2.0	Google	N/A	Không	Có	Tích hợp Google Search, dài context
Claude 3.5	Anthropic	N/A	Không	Có	An toàn, coding mạnh
LLaMA 3.1	Meta	8B-405B	Có	Hạn chế	Cộng đồng lớn, dễ fine-tune
Mistral Large	Mistral AI	7B-123B	Có	Có	Nhẹ, hiệu quả cho edge
Qwen 2.5	Alibaba	0.5B-72B	Có	Có	Đa ngôn ngữ, nhiều size
DeepSeek-V3	DeepSeek	671B MoE	Có	Có	Chi phí thấp, MoE hiệu quả

Xu hướng hiện tại trong LLM:

- **Mixture of Experts (MoE):** Chỉ kích hoạt một phần nhỏ tham số mỗi lần, giảm chi phí inference (DeepSeek, Mixtral).
- **Long context:** Mở rộng context window lên 128K-2M tokens (Gemini, Claude).
- **Multimodal:** Xử lý văn bản, hình ảnh, âm thanh, video trong cùng một mô hình.
- **Edge deployment:** Các mô hình nhỏ (1B-7B) chạy được trên smartphone và thiết bị nhúng.
- **Reasoning:** Cải thiện khả năng suy luận logic và toán học (o1, DeepSeek-R1).

c) Kỹ thuật Prompt Engineering

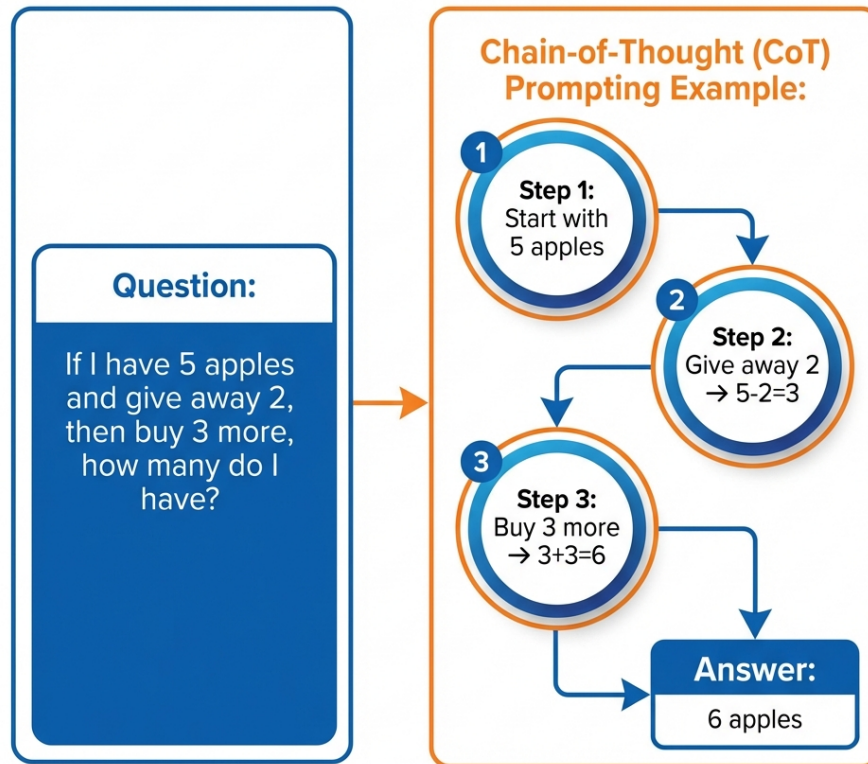
Prompt engineering là nghệ thuật thiết kế câu lệnh (prompt) để điều khiển hành vi của LLM. Các kỹ thuật prompting chính được so sánh trong bảng sau:

Kỹ thuật	Mô tả	Ví dụ prompt	Phù hợp với
Zero-shot	Yêu cầu trực tiếp không có ví dụ	“Dịch sang tiếng Anh: Xin chào”	Tác vụ đơn giản
Few-shot	Cung cấp 2-5 ví dụ mẫu trước khi hỏi	“Ví dụ: Mèo → Cat. Chó → Dog. Nhà → ?”	Tác vụ cần định dạng cụ thể
Chain-of-Thought (CoT)	Yêu cầu mô hình suy luận từng bước	“Hãy giải thích từng bước: $5 + 3 \times 2 = ?$ ”	Toán học, logic phức tạp
System Prompt	Định nghĩa vai trò và quy tắc cho agent	“Bạn là trợ lý y tế, luôn khuyên bệnh nhân gặp bác sĩ”	Chatbot, AI Agent
Self-consistency	Sinh nhiều câu trả lời, chọn đáp án phổ biến nhất	Chạy CoT nhiều lần, vote kết quả	Tăng độ chính xác

Ví dụ minh họa Chain-of-Thought Prompting:

Chain-of-Thought là kỹ thuật quan trọng giúp LLM giải quyết các bài toán phức tạp bằng cách yêu cầu suy luận từng bước.

Academic Diagram: Chain-of-Thought (CoT) Prompting Example



Hình 2.15: Ví dụ Chain-of-Thought: thay vì trả lời trực tiếp, mô hình giải thích từng bước suy luận để đi đến kết quả cuối cùng.

2.9.3 Retrieval-Augmented Generation (RAG)

RAG là kỹ thuật kết hợp mô hình sinh với hệ thống truy xuất thông tin, giúp mô hình tạo câu trả lời dựa trên tài liệu được truy hồi mà không phải đưa toàn bộ tri thức vào tham số [44]:

1. **Indexing:** Chuyển đổi tài liệu thành vector embeddings và lưu vào vector database
2. **Retrieval:** Khi nhận câu hỏi, tìm kiếm các đoạn văn bản liên quan nhất
3. **Generation:** LLM sinh câu trả lời dựa trên context được truy xuất

Trong ứng dụng chăm sóc người cao tuổi, RAG cho phép robot:

- Trả lời câu hỏi về lịch uống thuốc, lịch khám bệnh từ dữ liệu cá nhân

- Cung cấp thông tin y tế chính xác từ nguồn tin cậy
- Nhớ các cuộc hội thoại trước đó để tương tác cá nhân hóa

2.10 Hệ thống Nhận dạng Giọng nói và Xử lý Ngôn ngữ Tự nhiên

2.10.1 Một số giải pháp Nhận dạng Giọng nói:

Dưới đây là bảng so sánh các giải pháp chuyển đổi giọng nói thành văn bản (Speech-to-Text) phổ biến hiện nay:

Tiêu chí	Google Speech-to-Text	OpenAI Whisper	Vosk
Hình thức	Cloud API (SaaS)	Open-source (Local/Cloud)	Open-source (Local)
Độ chính xác	Rất cao (đặc biệt tiếng Việt)	Cực cao (State-of-the-art)	Khá (tùy thuộc model)
Internet	Bắt buộc	Không bắt buộc	Không cần thiết
Tốc độ	Phụ thuộc băng thông	Phụ thuộc GPU/CPU	Rất nhanh (Real-time)
Chi phí	Trả phí theo lưu lượng	Miễn phí phần mềm	Miễn phí
Phù hợp	Ứng dụng mobile cần độ chính xác tối đa.	Robot có phần cứng mạnh, cần hiểu ngữ cảnh sâu.	Thiết bị nhúng (Raspberry Pi), phản hồi cực nhanh.

Bảng 2.4: So sánh các công nghệ Speech-to-Text.

2.10.2 So sánh các phương pháp Xử lý Ngôn ngữ Tự nhiên (NLP)

Việc lựa chọn phương pháp NLP quyết định khả năng ”thông minh” và linh hoạt của Robot khi nhận lệnh.

Phương pháp	Đặc điểm	Độ phức tạp	Mở rộng	Ưu/Nhược điểm
Rule-based	Từ khóa và mẫu câu cố định.	Thấp	Kém	Ưu: Phản hồi tức thì. Nhược: Cứng nhắc.
Machine Learning	Phân loại ý định (Intent) qua TF-IDF, SVM.	Trung bình	Trung bình	Ưu: Hiểu biến thể nhẹ. Nhược: Cần dữ liệu.
Deep Learning	Transformer (BERT, GPT, Llama) hiểu sâu.	Cao	Rất cao	Ưu: Tự nhiên như người. Nhược: Cần GPU lớn.

Bảng 2.5: So sánh các phương pháp NLP.

2.10.3 Phương thức triển khai: Cloud Computing và Edge Computing

Để đảm bảo hệ thống hoạt động mượt mà trên phần cứng thực tế (Smartphone 1), tác giả lựa chọn giải pháp kết hợp giữa xử lý trực tiếp trên thiết bị và tận dụng tài nguyên máy chủ đám mây:

a. Cloud Computing Đây là phương án chủ đạo để xử lý các tác vụ nặng về tính toán và được ưu tiên sử dụng.

- **Cơ chế:** Robot gửi dữ liệu âm thanh hoặc văn bản lên các dịch vụ AI như Google Cloud hoặc OpenAI thông qua mạng Internet.
- **Ưu điểm:** Cho phép Robot trò chuyện thông minh, hiểu được các câu nói phức tạp nhờ sức mạnh xử lý cực mạnh của máy chủ mà không làm nóng điện thoại.
- **Lý do áp dụng:** Giúp khắc phục giới hạn về phần cứng, đảm bảo nội dung phản hồi từ Chatbot luôn nhanh chóng và chính xác.

b. Edge Computing Chỉ áp dụng cho các tác vụ xử lý NLP đơn giản

- **Cơ chế:** Sử dụng các thư viện nhẹ tích hợp sẵn để nhận diện các câu lệnh ngắn hay xử lý prompt
- **Hạn chế:** Chỉ xử lý được các mẫu câu đơn giản hay làm mượt các prompt hay phản hồi nhận được, không thể thực hiện các cuộc hội thoại linh hoạt nếu không có kết nối Internet.

2.11 Mô hình chuỗi LSTM cho nhận dạng hành động

Một detector trên từng ảnh chỉ quan sát tư thế tại một thời điểm. Trong bài toán té ngã, quá trình chuyển từ đứng/ngồi sang hạ thấp nhanh rồi nằm mới là tín hiệu quan trọng. Recurrent Neural Network (RNN) đưa trạng thái ẩn từ bước $t - 1$ sang bước t , nhưng có thể gặp mất hoặc bùng nổ gradient khi chuỗi dài. Long Short-Term Memory (LSTM) bổ sung cell state và ba cổng vào, quên, ra để điều tiết thông tin [18].

Với input x_t , hidden state h_{t-1} và cell state c_{t-1} :

$$f_t = \sigma(W_f[x_t, h_{t-1}] + b_f), \quad (2.21)$$

$$i_t = \sigma(W_i[x_t, h_{t-1}] + b_i), \quad (2.22)$$

$$\tilde{c}_t = \tanh(W_c[x_t, h_{t-1}] + b_c), \quad (2.23)$$

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t, \quad (2.24)$$

$$o_t = \sigma(W_o[x_t, h_{t-1}] + b_o), \quad (2.25)$$

$$h_t = o_t \odot \tanh(c_t). \quad (2.26)$$

Trong hệ thống, một mẫu là tensor $X \in \mathbb{R}^{15 \times 69}$. Số 15 là chiều thời gian; 69 là vector pose/hình học mỗi frame. Đầu ra softmax hai lớp được diễn giải là normal/fall. Threshold xác suất chỉ là tầng đầu; hysteresis theo số frame liên tiếp và hậu kiểm hình học giúp ngăn trạng thái dao động. Cần lưu ý nếu FPS thay đổi, 15 frame tương ứng thời gian thực khác nhau; một triển khai tốt nên log timestamp hoặc resample chuỗi theo thời gian.

2.12 Tracking-by-detection và duy trì danh tính

Tracking-by-detection tách bài toán thành phát hiện đối tượng ở mỗi frame và liên kết detection qua thời gian. SORT sử dụng Kalman để dự báo chuyển động, IoU làm độ tương đồng và Hungarian cho phép gán tối ưu [19]. Deep SORT bổ sung embedding ngoại hình để giảm đổi ID khi che khuất [20]; ByteTrack chỉ ra rằng các detection confidence thấp vẫn có thể hữu ích khi liên kết với track [21].

Kalman gồm hai pha. Pha dự báo:

$$\hat{x}_{t|t-1} = F\hat{x}_{t-1|t-1} + Bu_t, \quad (2.27)$$

$$P_{t|t-1} = FP_{t-1|t-1}F^T + Q. \quad (2.28)$$

Pha cập nhật với quan sát z_t :

$$K_t = P_{t|t-1} H^T (H P_{t|t-1} H^T + R)^{-1}, \quad (2.29)$$

$$\hat{x}_{t|t} = \hat{x}_{t|t-1} + K_t (z_t - H \hat{x}_{t|t-1}), \quad (2.30)$$

$$P_{t|t} = (I - K_t H) P_{t|t-1}. \quad (2.31)$$

Ma trận chi phí giữa n track và m detection được đưa vào thuật toán Hungarian, vốn giải bài toán gán tuyến tính [23]. IoU đơn độc có thể thất bại khi người từ đứng chuyển sang nằm làm vùng bao thay đổi mạnh. Vì vậy đề tài kết hợp IoU, khoảng cách tâm, neo vai/hông, tỷ lệ diện tích và histogram vùng thân. Định danh ID0 còn được bảo vệ bởi tuổi track dài hơn và phục hồi có điều kiện.

2.13 Suy luận AI trên thiết bị và TensorFlow Lite

Edge AI xử lý dữ liệu gần nguồn thay vì gửi toàn bộ lên cloud. Với camera trong nhà, cách này giảm lưu lượng, độ trễ và phạm vi truyền ảnh; đồng thời vẫn cần kiểm soát quyền và lưu trữ cục bộ. TensorFlow Lite cung cấp runtime cho thiết bị di động/biên, hỗ trợ interpreter CPU và delegate phần cứng [28].

Chuỗi chuyển đổi thường gồm model huấn luyện $\rightarrow$ biểu diễn trung gian $\rightarrow$ TFLite. Ba vấn đề phải kiểm chứng là: shape/layout tensor; toán tử được runtime/delegate hỗ trợ; và sai số số học sau tối ưu. FP16 giảm kích thước trọng số và băng thông bộ nhớ, nhưng không bảo đảm tăng tốc nếu delegate không sử dụng FP16. INT8 có thể giảm sâu hơn nhưng cần representative dataset và kiểm tra suy giảm keypoint/metric.

NMS có thể đặt trong graph hoặc ngoài graph. NMS trong graph giảm code hậu xử lý nhưng dễ tạo toán tử không tương thích delegate. NMS ngoài graph yêu cầu decoder Kotlin chính xác, đổi lại graph gọn và dễ chạy GPU hơn. Vì vậy kết quả chuyển đổi cần so ở cả tensor và detection cuối cùng.

2.14 Firebase Authentication và Realtime Database

Realtime Database tổ chức dữ liệu thành cây JSON và đồng bộ thay đổi tới client đang lắng nghe [29]. Client Android có cache và hàng đợi write, do đó callback “gần thời gian thực” không đồng nghĩa bản ghi luôn mới. Đối với lệnh robot, timestamp/expiry là phần của ngữ nghĩa dữ liệu, không chỉ là metadata.

Firebase Authentication cung cấp UID cho người dùng; Security Rules chạy phía

máy chủ quyết định read/write và có thể validate kiểu/cấu trúc [30]. Rules mặc định cần từ chối, sau đó cấp quyền tối thiểu theo UID, family và robot. Các node lệnh/signaling không nên dùng chung toàn cục trong hệ thống nhiều robot. Chúng cũng cần validate miền giá trị, người gửi và vòng đời phiên.

Ưu điểm của RTDB là tích hợp Android nhanh, listener đơn giản và phù hợp state/event nhỏ. Hạn chế là schema phi quan hệ, truy vấn có giới hạn và dễ tạo coupling bằng string path. Tách repository/data access, định nghĩa model chung và test rules bằng emulator giúp giảm rủi ro.

2.15 WebRTC, signaling và ICE

WebRTC là tập API/giao thức cho media và dữ liệu thời gian thực giữa các peer [31]. Một phiên cần signaling để trao đổi Session Description Protocol (SDP) và ICE candidate, nhưng chuẩn không bắt buộc một công nghệ signaling cụ thể. Đề tài dùng RTDB cho phần này.

ICE thu thập candidate và kiểm tra cặp đường truyền để vượt Network Address Translator [32]. STUN giúp peer biết địa chỉ public; TURN relay media khi không tạo được đường trực tiếp. Do đó có TURN làm tăng tỷ lệ kết nối nhưng thêm băng thông/độ trễ/chi phí máy chủ. Trạng thái call ở database và trạng thái PeerConnection là hai state machine khác nhau, cần cleanup idempotent để tránh phiên cũ.

WebRTC mã hóa media theo kiến trúc bảo mật giao thức, nhưng ứng dụng vẫn phải bảo vệ signaling, quyền camera/micro và danh tính peer [33]. SDP/ICE có thể lộ thông tin mạng; log không nên lưu vô hạn. Trong hệ thống chăm sóc, phải có chỉ báo rõ khi camera/micro đang truyền.

2.16 Bluetooth Low Energy và GATT

BLE phù hợp liên kết cự ly gần giữa điện thoại và vi điều khiển. GATT xây trên bảng attribute, tổ chức dữ liệu thành service, characteristic và descriptor [34]. Server công bố service/characteristic; client quét, kết nối, khám phá và thực hiện read/write/notify theo property.

Write without response giảm round-trip và phù hợp dòng lệnh cập nhật thường xuyên, nhưng không có ACK ở tầng GATT cho từng write. Ứng dụng phải dùng gửi lặp, sequence/timestamp và watchdog. UUID 128-bit cho custom service tránh xung đột profile chuẩn. Payload văn bản dễ debug nhưng tốn byte và parser cần chặt; protocol nhị

phân với version, sequence và checksum là hướng nâng cấp.

BLE disconnect là trạng thái bình thường có thể xảy ra do khoảng cách, radio hoặc vòng đời Android. Motor vì vậy không được giữ setpoint cuối. Callback disconnect và timeout độc lập ở firmware là yêu cầu an toàn, không nên chỉ dựa vào app.

2.17 Động cơ BLDC và điều khiển FOC

Động cơ BLDC/PMSM tạo mô-men bằng tương tác từ trường stator và rotor nam châm. Field-Oriented Control biến đổi dòng ba pha sang hệ trục quay $d-q$, tách thành phần từ thông và mô-men, sau đó dùng PWM để điều khiển điện áp [37]. Với encoder từ AS5600, hệ thống có phản hồi góc/tốc độ cho vòng kín.

Vòng điều khiển vận tốc so sánh tốc độ đặt với tốc độ đo. PID có dạng khái quát:

$$u(t) = K_p e(t) + K_i \int_0^t e(\tau) d\tau + K_d \frac{de(t)}{dt}. \quad (2.32)$$

Giới hạn điện áp bảo vệ driver/motor; giới hạn vận tốc bảo vệ cơ khí; lọc thông thấp giảm nhiễu đo. Deadzone giúp thẳng ma sát tĩnh nhưng có thể tạo bước nhảy tốc độ, vì vậy bản hoàn thiện nên bổ sung ramp/giới hạn gia tốc. SimpleFOC cung cấp các abstraction motor, driver, sensor và vòng loopFOC/move cho vi điều khiển [36].

2.18 An toàn và quản trị rủi ro AI

Một hệ thống cảnh báo người cao tuổi phải xem cả false positive và false negative. False positive lặp lại làm người dùng mất tin cậy; false negative có thể trì hoãn hỗ trợ. NIST AI RMF đề xuất các chức năng govern, map, measure và manage xuyên suốt vòng đời [41]. Với đồ án, điều này được cụ thể hóa bằng phạm vi sử dụng rõ, metric theo sự cố, log bằng chứng, giám sát lỗi, fallback và công bố giới hạn.

LLM không nên là thành phần duy nhất quyết định an toàn. Từ khóa cục bộ xử lý câu rõ, LLM chỉ phân loại trường hợp mơ hồ và lỗi mặc định nguy hiểm. Tương tự, output LSTM đi qua state machine/hậu kiểm. Các lớp độc lập làm hệ thống dễ kiểm tra hơn, nhưng không thay thế đánh giá thực địa.

2.19 Kết luận chương

Chương đã trình bày nền tảng robot dựa trên smartphone, mạng nơ-ron, CNN, detector họ YOLO, pose, dữ liệu té ngã, tracking, LSTM, LLM/RAG, nhận dạng giọng nói, AI biên, Firebase, WebRTC, BLE và FOC. Các nội dung được liên kết với lựa chọn hiện thực thay vì giả định một mô hình khảo sát chính là model cuối. Điểm then chốt là phát hiện té ngã của đề tài không dựa trên một nhãn ảnh duy nhất mà kết hợp pose, duy trì ID0, chuỗi LSTM và hậu kiểm có bằng chứng. Chương tiếp theo chuyển các khái niệm này thành yêu cầu, kiến trúc và hợp đồng cụ thể.

Chương 3 – Phân tích và thiết kế hệ thống

3.1 Mục tiêu và nguyên tắc thiết kế

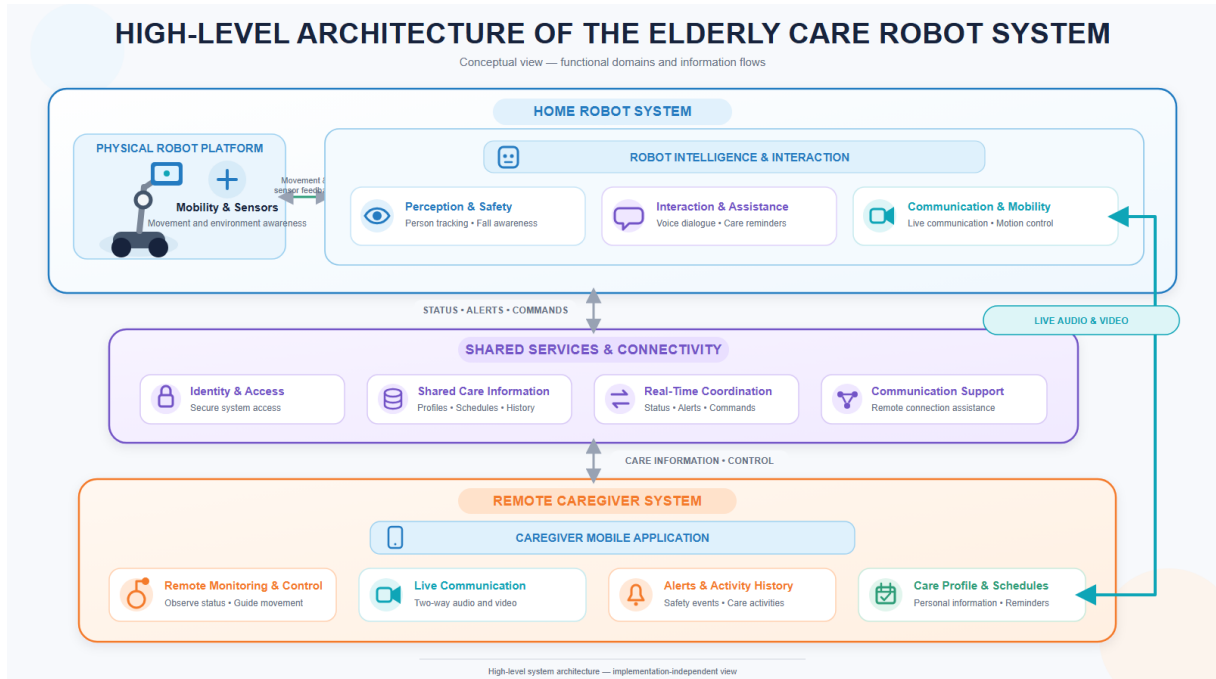
Thiết kế hệ thống được xây dựng từ ba mục tiêu: hỗ trợ người cao tuổi tại nhà, cho phép người chăm sóc can thiệp từ xa và bảo đảm robot chuyển về trạng thái an toàn khi một mắt xích truyền thông bị lỗi. Thay vì đặt toàn bộ xử lý lên đám mây, đề tài phân phối trách nhiệm theo nơi dữ liệu được tạo ra. Ảnh camera được suy luận tại RobotApp; lệnh tốc độ cuối cùng được RobotApp gửi cục bộ qua BLE; ESP32 thực hiện vòng điều khiển động cơ; Firebase đồng bộ trạng thái giữa hai ứng dụng; WebRTC chuyên chở media thời gian thực.

Các nguyên tắc chi phối thiết kế gồm:

- **Tách vai trò:** CaregiverApp là giao diện quản lý và điều khiển từ xa; RobotApp là bộ điều phối tại robot; ESP32 chỉ chấp hành vận tốc và bảo vệ mức thấp.
- **Một chủ sở hữu tài nguyên:** tại một thời điểm chỉ CameraX hoặc WebRTC sở hữu camera; chỉ một RobotControlManager sở hữu kết nối BLE.
- **Mặc định an toàn:** dữ liệu không hợp lệ, lệnh quá cũ, mất BLE hoặc quá hạn đều dẫn đến dừng; sự cố té ngã không được xác nhận an toàn được xử lý theo nhánh nguy hiểm.
- **Truy vết được:** trạng thái, lịch nhắc, cảnh báo và cuộc gọi được biểu diễn thành bản ghi có timestamp và trạng thái.
- **Suy luận gần nguồn:** ảnh không cần gửi lên máy chủ để phát hiện pose/té ngã, nhờ đó giảm độ trễ và giảm bề mặt lộ dữ liệu hình ảnh.
- **Hợp đồng rõ ràng:** mỗi kênh có schema, miền giá trị, đơn vị, tần suất và hành vi lỗi được xác định.

3.2 Mô hình bối cảnh và tác nhân

Hình 3.1 mô tả ba vùng chức năng: hệ thống robot tại nhà, các dịch vụ dùng chung và ứng dụng người chăm sóc. Sơ đồ ở mức khái niệm không ràng buộc công nghệ, nhấn mạnh luồng thông tin chăm sóc, cảnh báo, điều khiển và cuộc gọi.



Hình 3.1: Kiến trúc mức cao của hệ thống robot chăm sóc người cao tuổi

Bảng 3.1: Tác nhân và trách nhiệm

Tác nhân	Mục tiêu	Tương tác chính
Người cao tuổi	Nhận trợ giúp, nhắc lịch, hội thoại và liên lạc	Nói/nhập với RobotApp; xuất hiện trong camera; trả lời câu hỏi xác nhận
Người chăm sóc	Theo dõi, cấu hình, nhận cảnh báo và can thiệp	Dùng CaregiverApp; tạo lịch; bật/tắt AI; gọi video; lái robot; xem nhật ký
RobotApp	Điều phối chức năng tại robot	Suy luận AI, hội thoại, lịch nhắc, signaling/media, nhận lệnh RTDB, gửi BLE
CaregiverApp	Giao diện quản lý từ xa	Xác thực, dữ liệu chăm sóc, lịch, cảnh báo, cuộc gọi và joystick
Firebase	Dịch vụ danh tính và đồng bộ	Authentication; RTDB cho hồ sơ, lịch, còi, lệnh, cảnh báo và signaling
Groq API	Suy luận ngôn ngữ từ xa	Chat streaming, diễn đạt lời nhắc, phân loại phản hồi và ý định gọi
ESP32/firmware	Điều khiển phần thân	Nhận payload BLE; điều khiển hai BLDC; dùng bằng watchdog
STUN/TURN	Hỗ trợ kết nối WebRTC	Khám phá đường P2P và chuyển tiếp media khi cần

3.3 Phân tích yêu cầu

3.3.1 Yêu cầu chức năng

Các yêu cầu chức năng được gán mã để liên kết với thiết kế và kiểm thử. “Phải” chỉ hành vi bắt buộc của nguyên mẫu; “nên” là hành vi cần đạt khi điều kiện thiết bị/mạng cho phép.

Bảng 3.2: Danh sách yêu cầu chức năng

Mã	Yêu cầu	Tiêu chí chấp nhận
FR01	Xác thực tài khoản	Người dùng có thể đăng ký, đăng nhập và sử dụng UID để truy cập hồ sơ/ lịch tương ứng.
FR02	Quản lý hồ sơ	Có thể đọc/cập nhật tên, cách xưng hô, người liên hệ, tình trạng và ghi chú chăm sóc.
FR03	Quản lý lịch nhắc	CaregiverApp tạo, sửa, bật/tắt và xóa lịch dưới Schedules/{uid}.
FR04	Thực hiện lời nhắc	RobotApp nhận lịch đang hoạt động, phát nội dung bằng TTS và ghi nhật ký phản hồi.
FR05	Hội thoại giọng nói	RobotApp nhận giọng nói, gửi yêu cầu LLM và phát câu trả lời hoàn chỉnh bằng TTS.
FR06	Bật/tắt theo dõi	CaregiverApp cập nhật cờ; RobotApp phản ứng thời gian gần thực và gửi dừng khi tắt.
FR07	Phát hiện người/pose	RobotApp trả về vùng bao, độ tin cậy và keypoint từ khung hình camera.
FR08	Duy trì người mục tiêu	Hệ thống gán người cần theo dõi thành ID0 và hạn chế đổi mục tiêu khi xuất hiện nhiều.
FR09	Tự bám theo	Từ vị trí ID0, RobotApp sinh lệnh tiến/quay phù hợp vùng chết và giới hạn tốc độ.
FR10	Phát hiện nghi té ngã	LSTM xử lý cửa sổ 15 khung hình $\times$ 69 đặc trưng và sinh trạng thái theo dõi.
FR11	Hậu kiểm sự cố	Bộ luật dùng vị trí thân trên, keypoint, vùng bao, tỷ lệ và mất dấu để chấp nhận/từ chối.
FR12	Xác nhận bằng giọng nói	Robot dừng, hỏi người cao tuổi và chỉ tạo một phiên xác nhận cho mỗi sự cố.

Mã	Yêu cầu	Tiêu chí chấp nhận
FR13	Ghi cảnh báo	RobotApp tạo/cập nhật bản ghi <code>fall_alerts</code> với trạng thái và bằng chứng.
FR14	Gọi khẩn cấp	Khi nguy hiểm/không phản hồi, RobotApp tạo cuộc gọi có <code>emergencyReason=fall</code> .
FR15	Gọi video hai chiều	Hai ứng dụng trao đổi offer/answer/ICE và truyền âm thanh, hình ảnh bằng WebRTC.
FR16	Điều khiển joystick	CaregiverApp ghi x, y, ts khoảng 20 Hz và ghi lệnh dừng khi thả joystick.
FR17	Cầu nối lệnh	RobotApp kiểm tra lệnh RTDB, ánh xạ sang hai bánh và tạo payload <code>L...R...</code>
FR18	Điều khiển động cơ	ESP32 nhận payload BLE, áp dụng dead-zone/giới hạn và điều khiển vận tốc kín.
FR19	Nhật ký chăm sóc	CaregiverApp hiển thị lịch sử nhắc việc, cảnh báo và cuộc gọi theo thời gian.
FR20	Dừng khi lỗi	Lệnh thiếu/cũ/không hữu hạn, mất BLE hoặc timeout firmware phải dẫn đến vận tốc 0.

3.3.2 Yêu cầu phi chức năng

Mô hình chất lượng tham chiếu các thuộc tính phù hợp ISO/IEC 25010 [43]. Vì đây là nguyên mẫu nghiên cứu, các ngưỡng chưa được đo trên mọi thiết bị được xem là mục tiêu kiểm thử, không phải tuyên bố đã chứng nhận.

Bảng 3.3: Yêu cầu phi chức năng

Mã	Thuộc tính	Yêu cầu/biện pháp
NFR01	An toàn chuyển động	Tốc độ bị chặn tại Android và firmware; có STOP khi tắt chức năng, thả joystick, mất lệnh hoặc ngắt BLE.
NFR02	Tính kịp thời	Lệnh điều khiển có timestamp; RobotApp không thi hành lệnh quá 2 s; firmware timeout 800 ms.

Mã	Thuộc tính	Yêu cầu/biện pháp
NFR03	Độ tin cậy	Listener được gỡ đúng vòng đời; camera/BLE có một owner; sự cố có khóa chống phát lặp.
NFR04	Khả dụng	Giao diện ưu tiên nút lớn, trạng thái rõ, TTS và thao tác tối thiểu cho người cao tuổi.
NFR05	Bảo mật	Xác thực người dùng, quy tắc RTDB theo UID/robot, HTTPS, không nhúng khóa thật vào kho mã.
NFR06	Riêng tư	Ảnh AI được xử lý cục bộ; chỉ truyền media khi có cuộc gọi; giới hạn log chứa dữ liệu nhạy cảm.
NFR07	Khả năng bảo trì	Phân module UI, tracking, điều khiển, hội thoại và firmware; hằng số giao thức tập trung.
NFR08	Tương thích	Android minSdk 23, target/compile SDK 36; quyền BLE tách theo trước/sau Android 12.
NFR09	Hiệu quả tài nguyên	Drop-frame khi AI đang bận; TFLite nhiều luồng/biến thể GPU; không mở hai camera.
NFR10	Truy vết	Lịch, cảnh báo, cuộc gọi và lệnh đều có định danh hoặc timestamp phục vụ đối chiếu.
NFR11	Khả năng mở rộng	Schema đích cần tách users/uid/robots/robotId; snapshot hiện tại mới tối ưu một robot.
NFR12	Minh bạch AI	Lưu lý do hậu kiểm và đặc trưng bằng chứng; công bố giới hạn, không gọi kết quả là chẩn đoán.

3.4 Mô hình use case

Các use case được nhóm theo bốn miền. Nhóm quản lý gồm đăng nhập, hồ sơ, lịch và nhật ký. Nhóm tương tác gồm hội thoại và lời nhắc. Nhóm an toàn gồm theo dõi, phát hiện té ngã, hỏi xác nhận và cảnh báo. Nhóm điều khiển/liên lạc gồm gọi video, joystick, auto-follow và dừng khẩn cấp.

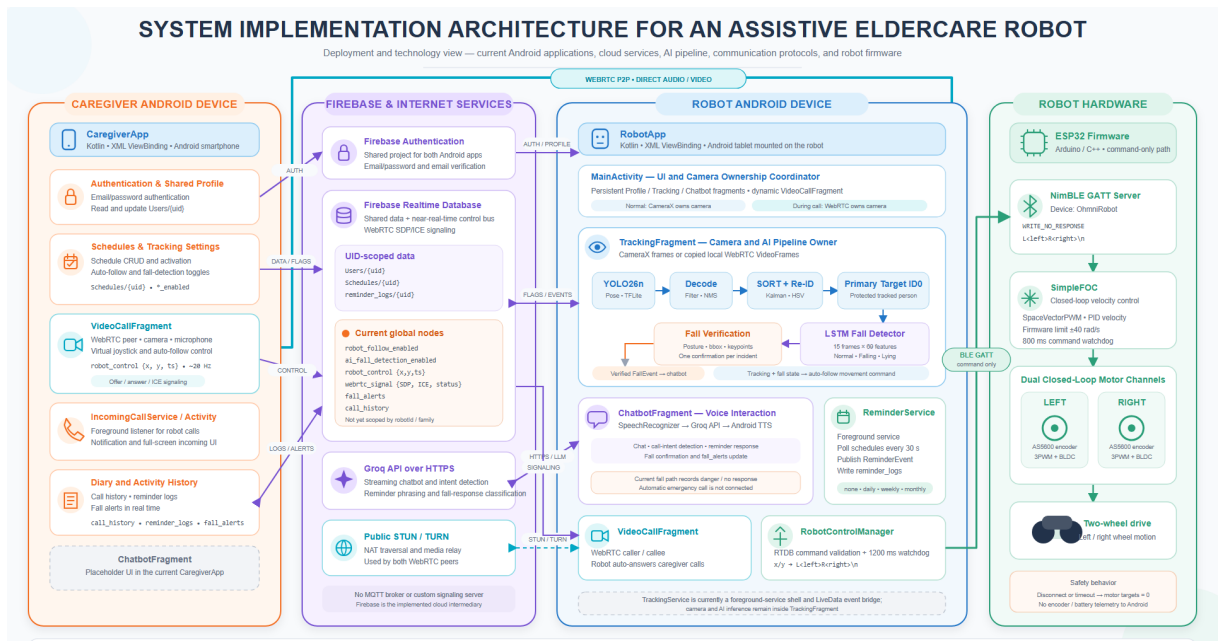
Đặc tả đầy đủ tiền điều kiện, luồng chính, ngoại lệ và hậu điều kiện được trình bày ở Phụ lục. Việc tách đặc tả giúp chương thiết kế tập trung vào quan hệ giữa use case và kiến trúc.

Bảng 3.4: Tóm tắt use case chính

Mã	Use case	Tác nhân chính	Kết quả
UC01	Đăng ký/đăng nhập	Người chăm sóc	Phiên Firebase Auth và UID hợp lệ
UC02	Quản lý hồ sơ chăm sóc	Người chăm sóc	Hồ sơ dùng chung được cập nhật
UC03	Quản lý lịch nhắc	Người chăm sóc	Lịch hoạt động được đồng bộ
UC04	Nhận lời nhắc	Người cao tuổi	TTS phát nhắc và phản hồi được ghi
UC05	Hội thoại trợ lý	Người cao tuổi	Câu trả lời phù hợp hồ sơ/ngữ cảnh
UC06	Bật theo dõi/té ngã	Người chăm sóc	Pipeline tương ứng hoạt động
UC07	Phát hiện và xác nhận té ngã	RobotApp/người cao tuổi	An toàn, nguy hiểm hoặc không phản hồi
UC08	Gọi video	Hai người dùng	Phiên WebRTC được thiết lập/kết thúc
UC09	Lái robot từ xa	Người chăm sóc	Lệnh bánh xe hợp lệ tới ESP32
UC10	Xem nhật ký	Người chăm sóc	Danh sách sự kiện theo thời gian

3.5 Kiến trúc triển khai

Hình 3.2 là sơ đồ triển khai được đối chiếu với mã nguồn. Firebase RTDB vừa là kho dữ liệu dùng chung vừa là bus điều phối gần gần thời gian thực và kênh signaling. Đây không phải MQTT broker. STUN/TURN chỉ hỗ trợ WebRTC đi qua NAT hoặc relay, không lưu hồ sơ/lịch. Groq được gọi qua HTTPS từ RobotApp; CaregiverApp không chạy chatbot LLM trong snapshot hiện tại.



Hình 3.2: Kiến trúc hiện thực của hệ thống

Bảng 3.5: Phân rã thành phần và trách nhiệm

Thành phần	Công nghệ	Trách nhiệm
CaregiverApp	Kotlin, XML ViewBinding, Firebase, WebRTC	UI từ xa, lịch, nhật ký, cuộc gọi, joystick và cờ tính năng
RobotApp	Kotlin, CameraX, TFLite, OpenCV, Firebase, WebRTC, BLE, OkHttp	Camera/AI, hội thoại, lịch nhắc, cảnh báo, cuộc gọi và cầu nối robot
Firebase Auth	Dịch vụ Firebase	Danh tính email/mật khẩu, UID và phiên đăng nhập
Firebase RTDB	Cơ sở dữ liệu JSON thời gian thực	Dữ liệu, cờ, lệnh, signaling và lịch sử
Groq API	HTTPS, Chat Completions	Sinh/phân loại ngôn ngữ; streaming delta cho hội thoại
WebRTC	PeerConnection, SDP, ICE, STUN/TURN	Media âm thanh/hình ảnh thời gian thực [31]
BLE GATT	Custom service/characteristic	Kênh lệnh cục bộ từ RobotApp tới ESP32 [34]
ESP32 firmware	NimBLE, SimpleFOC, I2C, PWM	Parser lệnh, deadzone, watchdog và điều khiển vận tốc kín

3.6 Thiết kế dữ liệu Firebase

Firebase RTDB lưu cây JSON và đồng bộ thay đổi tới listener của client [29]. Bảng 3.6 tổng hợp schema được suy ra từ hai ứng dụng. Các node UID-scoped phù hợp nhiều tài khoản hơn, trong khi node toàn cục hiện chỉ an toàn cho kịch bản demo một robot.

Bảng 3.6: Cây dữ liệu Realtime Database

Đường dẫn	Trường chính	Mục đích/phạm vi
Users/{uid}	fullName, email, robotName, pronoun, caregiver, medical-Conditions, notes, hobbies	Hồ sơ cá nhân; theo UID
Schedules/{uid}/{id}	title, note, time, days, active	Lịch nhắc; theo UID

Đường dẫn	Trường chính	Mục đích/phạm vi
reminder_logs/{uid}/{id}	scheduleId, triggeredAt, status, elderlyResponse, retryCount	Nhật ký lời nhắc; theo UID
robot_follow_enabled	Boolean	Cờ auto-follow; hiện toàn cục
ai_fall_detection_enabled	Boolean	Cờ phát hiện té ngã; hiện toàn cục
robot_control	x, y, ts	Lệnh joystick/tự bấm; hiện toàn cục
webrtc_signal	status, callerRole, emergencyReason, offer, answer, robotCandidates, caregiverCandidates	Signaling cuộc gọi; hiện toàn cục
fall_alerts/{pushId}	timestamp, status, response, verificationReason, evidence	Cảnh báo té ngã; hiện toàn cục
call_history/{id}	timestamp, direction, status, duration/reason	Nhật ký cuộc gọi; hiện toàn cục

Schema mục tiêu nên đưa các node toàn cục vào `families/{familyId}/robots/{robotId}`. Quy tắc bảo mật phải kiểm tra thành viên gia đình, quyền caregiver và quyền của thiết bị robot. Firebase Security Rules có thể kiểm soát read/write và validate cấu trúc tại máy chủ [30]; vì vậy miền $[-1, 1]$, kiểu Number và giới hạn timestamp nên được kiểm tra cả ở rules lẫn RobotApp.

3.7 Thiết kế kênh điều khiển chuyển động

3.7.1 Hợp đồng lệnh trên RTDB

CaregiverApp phát lệnh:

```

1 robot_control = {
2   "x": Number,    // translation, [-1, 1]
3   "y": Number,    // steering,    [-1, 1]
4   "ts": Long      // Unix epoch milliseconds
5 }
```

Listing 1: Cấu trúc lệnh điều khiển trên RTDB

RobotApp chỉ nhận lệnh khi x, y hữu hạn, timestamp tồn tại, tuổi lệnh không quá 2.000 ms và timestamp không vượt tương lai quá 5.000 ms. Giá trị sau đó bị chặn về $[-1, 1]$. Cơ chế này ngăn snapshot cũ trong cơ sở dữ liệu làm robot di chuyển sau khi ứng dụng kết nối lại.

3.7.2 Ánh xạ joystick sang hai bánh

Trong CaregiverApp, độ lệch dọc và ngang của joystick được chuẩn hóa theo bán kính:

$$x = \frac{\Delta y}{r}, \quad y = -\frac{\Delta x}{r}. \quad (3.1)$$

Do hệ tọa độ màn hình có chiều dương đi xuống, kéo lên tạo $x < 0$. RobotApp tính:

$$t = \text{clip}(x, -1, 1), \quad (3.2)$$

$$s = 0.15 \text{ clip}(y, -1, 1), \quad (3.3)$$

$$v_L = (s + t)V_{max}, \quad (3.4)$$

$$v_R = (s - t)V_{max}, \quad (3.5)$$

với $V_{max} = 20$. Nếu $|x| > 0,3$ và $|y| < 0,3$, thành phần quay được đặt về 0 để xe đi thẳng ổn định. Hai động cơ lắp đối xứng nên dấu setpoint không thể được diễn giải riêng rẽ mà phải theo hiệu chỉnh robot thật.

Bảng 3.7: Các điểm chuẩn ánh xạ lệnh

x	y	(v_L, v_R)	Ý nghĩa hiệu chỉnh
-1	0	$(-20, 20)$	Tiến
1	0	$(20, -20)$	Lùi
0	1	$(3, 3)$	Kênh quay dương
0	-1	$(-3, -3)$	Kênh quay âm
0	0	$(0, 0)$	Dừng

3.7.3 Hợp đồng BLE và chuỗi dừng an toàn

RobotApp là GATT client và ESP32 là GATT server. Hai định danh của kết nối gồm:

- service UUID: 4FAFC201-1FB5-459E-8FCC-C5C9C331914B;
- characteristic UUID: BEB5483E-36E1-4688-B7F5-EA07361B26A8.

Payload ASCII có dạng L<float>R<float>\n và được ghi bằng WRITE_NO_RESPONSE. GATT tổ chức dữ liệu thành service, characteristic và descriptor [34].

Ba tầng thời gian bảo vệ chuyển động: tuổi lệnh RTDB 2 s chống lệnh lưu cũ; watchdog RobotApp 1,2 s chủ động gửi STOP khi không nhận lệnh mới; watchdog firmware 800 ms đặt hai setpoint bằng 0 nếu BLE không cập nhật. Khi ngắt kết nối, callback firmware cũng dừng ngay. Việc đặt watchdog ngắn nhất ở tầng chấp hành bảo đảm lỗi ở Android hoặc mạng không làm động cơ giữ lệnh vô thời hạn.

3.8 Thiết kế pipeline thị giác và té ngã

3.8.1 Luồng dữ liệu

Khung hình từ CameraX hoặc bản sao VideoFrame cục bộ của WebRTC được resize và chuyển thành tensor đầu vào. Mô hình pose TFLite tạo vùng bao, confidence và keypoint. Kết quả được lọc và NMS, sau đó đưa vào SORT. Bộ gán Hungarian tìm ghép tối ưu giữa track dự đoán và detection; Kalman dự báo vùng bao; Re-ID bổ sung histogram vùng thân. SORT có nền tảng là tracking-by-detection thời gian thực [19], Kalman cung cấp mô hình dự báo tuyến tính [22], còn Hungarian giải bài toán gán tối ưu [23].

3.8.2 Khóa người mục tiêu ID0

Khi chưa có mục tiêu, người có vùng bao lớn nhất được ưu tiên và chuyển thành ID0. Track ID0 có tuổi tối đa 45 khung hình, trong khi track nhiều chỉ giữ 10 khung hình. Chi phí ghép kết hợp IoU, khoảng cách tâm chuẩn hóa, khoảng cách neo pose, biến thiên diện tích và tương quan histogram. Trọng số hình học trong mã là:

$$C = 0,30(1 - \text{IoU}) + 0,32d_c + 0,23d_p + 0,15p_a, \quad (3.6)$$

trong đó d_c là khoảng cách tâm chuẩn hóa, d_p là khoảng cách neo thân và p_a là phạt thay đổi diện tích. Track được bảo vệ có ngưỡng ghép rộng hơn nhưng detection cạnh tranh bị phạt để tránh chiếm ID0.

Khi ID0 mất tạm thời, hệ thống chỉ khôi phục nếu đúng một người xuất hiện, gần vị trí cuối trong phạm vi 28% đường chéo khung hình, nằm trong vùng trung tâm và ổn định ba khung hình. Cửa sổ phục hồi là 8 s. Nếu có nhiều người, hệ thống không tự gán lại để tránh theo nhầm người khác.

3.8.3 Đặc trưng LSTM và máy trạng thái

LSTM xử lý 15 khung hình, mỗi khung có 69 đặc trưng. Vector gồm tọa độ/độ tin cậy keypoint và đặc trưng hình học như tỷ lệ vùng bao. LSTM được thiết kế để giữ quan hệ dài/ngắn hạn và giảm mất gradient so với RNN cơ bản [18]. Đầu ra hai lớp là xác suất normal và fall; ngưỡng fall là 0,5.

Để tránh dao động theo từng frame, hệ thống dùng hysteresis: cần 10 frame fall liên tiếp để vào Falling; một frame normal reset bộ đếm vào fall; trạng thái Lying dùng cửa sổ xác nhận dài hơn; cần 20 frame biểu hiện đứng để xác nhận phục hồi. Chỉ ID0 được đưa vào LSTM, tránh sự kiện của người đi ngang kích hoạt cảnh báo dành cho chủ thể cần chăm sóc.

3.8.4 Hậu kiểm và quản lý một sự cố

Khi tín hiệu fall thô kết thúc, FallVerificationEngine đánh giá bằng chứng thay vì chờ toàn bộ cửa sổ chuyển sang Lying. Các tín hiệu gồm: thân trên nằm thấp trong ảnh; không quá 8 keypoint nhìn thấy; tỷ lệ vùng bao theo chiều ngang từ 1,05; độ tụt thân trên ít nhất 0,12 so với baseline; vùng bao bị cắt; hoặc mất mục tiêu ba frame. Cần điểm nghi ngờ tối thiểu 3 trong trường hợp vùng bao còn đầy đủ. Nếu đầu/vai ổn

định trong 30% phía trên khung hình, sự kiện có thể bị loại là nhiều.

Khi quyết định hỏi xác nhận, khóa sự cố được bật. Robot dừng và chỉ phát một `FallEvent`. Chatbot tạo một record `fall_alerts`, hỏi bằng TTS rồi nghe tối đa 10 s. Câu trả lời rõ được phân loại bằng từ khóa cục bộ; câu mơ hồ mới dùng LLM. Lỗi API, nội dung không chắc chắn hoặc không phản hồi đều đi theo nhánh nguy hiểm. Khóa chỉ được giải phóng sau khi xử lý và phục hồi ổn định, tránh bão cảnh báo.

3.9 Thiết kế cuộc gọi WebRTC và quyền sở hữu camera

WebRTC cho phép hai peer trao đổi media thời gian thực; ICE, STUN và TURN hỗ trợ vượt NAT [31][32]. Firebase không mang video mà chỉ mang thông điệp điều phối. Luồng gọi gồm: đặt `status=calling`, ghi `callerRole`; tạo offer SDP; peer kia tạo answer; hai bên đẩy ICE candidate vào hai nhánh riêng; khi mô tả từ xa và candidate sẵn sàng thì `PeerConnection` hình thành; trạng thái chuyển `connected`; kết thúc đặt `ended` và gỡ listener.

Camera là tài nguyên xung đột quan trọng. Ở trạng thái bình thường, `TrackingFragment` dùng `CameraX` cho preview và `ImageAnalysis`. Khi gọi, `CameraX` được unbind; WebRTC tạo camera capturer duy nhất. Local `VideoTrack` gắn thêm `VideoSink`, sao chép/downscale frame và gửi vào executor AI với cơ chế bỏ frame khi bận. Khi cuộc gọi kết thúc, capturer phải được dispose trước rồi `CameraX` mới bind lại. Thiết kế này cho phép video call, auto-follow và phát hiện té ngã đồng thời mà không mở camera lần thứ hai.

3.10 Thiết kế hội thoại và lịch nhắc

RobotApp dùng máy trạng thái hội thoại gồm lắng nghe, đang xử lý, đang nói và chờ tương tác tiếp. `SpeechRecognizer` chuyển giọng nói thành văn bản; `GroqChatService` gọi Chat Completions qua HTTPS; phần delta streaming được ghép thành câu; TTS phát theo đoạn câu hoàn chỉnh. Phát theo câu giảm cảm giác chờ nhưng tránh đọc từng token rời rạc. Sau khi TTS hoàn tất, state machine trở về lắng nghe.

Hồ sơ cung cấp tên robot, cách xưng hô, người chăm sóc, tình trạng và sở thích để xây dựng system prompt. Lịch nhắc nằm dưới UID; `ReminderService` theo dõi lịch hoạt động, tạo nội dung nhắc và phát sự kiện cho chatbot. Phản hồi được cập nhật vào `reminder_logs`. Khóa API phải được đưa qua cấu hình cục bộ hoặc backend; không

ghi khóa thật vào mã nguồn/tài liệu. API Chat Completions được mô tả trong tài liệu Groq [40].

3.11 Thiết kế firmware và điều khiển động cơ

ESP32 khởi tạo hai bus I2C riêng cho hai encoder AS5600, hai driver 3PWM và hai đối tượng BLDCMotor. Mỗi động cơ có 7 cặp cực, điện áp nguồn 17,5 V, giới hạn điện áp 10 V và giới hạn vận tốc 40 rad/s. Bộ điều khiển velocity dùng $P = 0, 1, I = 1, 0, D = 0$ và lọc thông thấp $T_f = 0, 01$ s. Field-Oriented Control điều khiển vector dòng/từ thông để tạo mô-men hiệu quả cho máy điện nam châm vĩnh cửu [37]; SimpleFOC cung cấp hiện thực vòng lặp cho nền tảng Arduino/ESP32 [36].

Trong mỗi vòng loop, `loopFOC()` được gọi trước, sau đó kiểm tra watchdog, áp dụng deadzone và gọi `move`. Giá trị nhỏ hơn 0,5 được đưa về 0; giá trị khác 0 nhưng nhỏ hơn 3 được nâng lên ± 3 để vượt ma sát/rung vùng thấp; cuối cùng chặn trong $[-40, 40]$. Parser chỉ nhận chuỗi có ký tự L trước R. Thiết kế tương lai cần kiểm tra chặt chuỗi số, NaN/vô hạn và ký tự thừa ở firmware, mặc dù Android đã lọc trước.

3.12 An toàn, bảo mật và quyền riêng tư

Hệ thống chăm sóc xử lý dữ liệu hồ sơ, âm thanh, hình ảnh và thông tin sự cố, do đó an toàn chức năng phải đi cùng an toàn thông tin. NIST AI RMF khuyến nghị quản trị, lập bản đồ bối cảnh, đo lường và quản lý rủi ro AI theo vòng đời [41]. Đối với ứng dụng di động, OWASP MASVS cung cấp nhóm kiểm soát về lưu trữ, mật mã, xác thực, mạng, nền tảng và chất lượng mã [42].

Bảng 3.8: Phân tích mối đe dọa và biện pháp

Mối đe dọa	Hậu quả	Biện pháp thiết kế
Ghi trái phép <code>robot_control</code>	Robot di chuyển ngoài ý muốn	Auth, rules theo robot, validate miền/timestamp, dừng vật lý
Replay lệnh cũ	Robot thi hành snapshot sau reconnect	Timestamp, tuổi tối đa 2 s, watchdog nhiều tầng
Rò rỉ API key	Lạm dụng dịch vụ/chi phí	local.properties/BuildConfig hoặc proxy backend, xoay khóa

Mối đe dọa	Hậu quả	Biện pháp thiết kế
Độc hồ sơ/ lịch trái phép	Lộ dữ liệu sức khỏe và thói quen	UID/family scoped rules, tối thiểu quyền, kiểm tra log
Chiếm signaling	Nghe nhìn hoặc phá cuộc gọi	Rules theo phiên/cặp thiết bị; dọn SDP/ICE; TLS và WebRTC security [33]
Cảnh báo giả	Hoảng loạn, mất tin cậy	Theo dõi ID0, chuỗi thời gian, hậu kiểm và hỏi xác nhận
Bỏ sót sự cố	Chậm hỗ trợ	Mặc định nguy hiểm khi không phản hồi; đo recall và giám sát lỗi
Tranh cam-era/ BLE	Mất AI hoặc điều khiển	Một owner; chuyển vòng đời theo trạng thái

3.13 Ma trận truy vết yêu cầu

Bảng 3.9: Truy vết từ yêu cầu đến module và kiểm thử

Yêu cầu	Module hiện thực	Dữ liệu/giao thức	Kiểm thử
FR01– FR02	Login, SignUp, Profile	Auth, Users/uid	TC-AUTH, TC-PROFILE
FR03– FR04	Schedule, ReminderService, chatbot	Schedules, reminder_logs	TC-REM
FR05	chatbot, SpeechRecognizer, TTS, GroqChat-Service	HTTPS streaming	TC-VOICE
FR06– FR09	tracking fragments, SortTracker, RobotControlManager	flags, robot_control, BLE	TC-TRACK, TC-MOVE
FR10– FR14	LstmFallDetector, FallVerificationEngine, chatbot	fall_alerts, webrtc_signal	TC-FALL
FR15	video call, Incoming-CallService	SDP, ICE, WebRTC	TC-CALL

Yêu cầu	Module hiện thực	Dữ liệu/giao thức	Kiểm thử
FR16– FR18	joystick, RobotControlManager, ESP32 firmware	x/y/ts, L/R payload	TC-CONTROL
FR19	Diary fragments/adapters	logs/alerts/history	TC-DIARY
FR20	Android và firmware watchdog	timestamp, timeout	TC-SAFETY

3.14 Đánh đổi và quyết định kiến trúc

Firestore RTDB giúp giảm khối lượng backend và phù hợp đồng bộ sự kiện, nhưng schema không quan hệ và node toàn cục là rủi ro mở rộng. WebRTC giảm tải truyền media qua backend nhưng cần signaling và TURN trong mạng khó. AI biên giảm độ trễ và riêng tư, đổi lại bị giới hạn CPU/GPU, pin và nhiệt. BLE có độ trễ thấp ở cự ly gần nhưng kết nối có thể gián đoạn, vì vậy cần watchdog. LLM cloud giúp hội thoại linh hoạt nhưng phụ thuộc mạng và có rủi ro nội dung; các quyết định an toàn quan trọng không được giao hoàn toàn cho LLM.

Việc kết hợp LSTM và hậu kiểm luật cũng là một đánh đổi. Mô hình học quan hệ thời gian tốt hơn luật một frame; luật lại cho phép giải thích bằng chứng và chặn một số tình huống hiển nhiên. Hai tầng tạo độ trễ xác nhận nhất định nhưng phù hợp hơn với nguyên tắc giảm cảnh báo giả có kiểm soát. Độ phù hợp cuối cùng phải được lượng hóa bằng thử nghiệm trên dữ liệu và thiết bị mục tiêu.

3.15 Kết luận chương

Chương đã chuyển nhu cầu chăm sóc thành yêu cầu có mã, phân rõ trách nhiệm cho hai ứng dụng, dịch vụ đám mây và firmware, đồng thời đặc tả schema, luồng điều khiển, pipeline AI, cuộc gọi và bảo vệ an toàn. Các quyết định quan trọng đều gắn với điểm kiểm thử và giới hạn đã biết. Chương tiếp theo trình bày cách các thiết kế này được hiện thực trong mã nguồn và mô hình của đồ án.

Chương 4 – Hiện thực hệ thống

4.1 Môi trường và tổ chức mã nguồn

Đồ án được hiện thực thành bốn khối mã/tài nguyên độc lập nhưng dùng chung hợp đồng dữ liệu. Hai ứng dụng Android được viết bằng Kotlin, giao diện XML và ViewBinding. RobotApp có package `com.example.datn_v1`; CaregiverApp có package `com.example.datn_caregiver`. Cả hai đặt `minSdk=23`, `targetSdk=36`, Java/Kotlin JVM target 11. Firmware là một sketch Arduino cho ESP32. Khối AI chứa mô hình huấn luyện, mô hình trung gian ONNX, mô hình TFLite và các script chuyển đổi/thử suy luận.

Bảng 4.1: Các khối mã nguồn của đồ án

Khối	Công nghệ chính	Vai trò
RobotApp	Android/Kotlin, CameraX, TFLite, OpenCV, Firebase, WebRTC, BLE, OkHttp	Bộ não trên robot, AI, hội thoại, cuộc gọi và cầu nối phần thân
CaregiverApp	Android/Kotlin, Firebase Auth/RTDB, WebRTC	Ứng dụng cho người thân quản lý, giám sát và điều khiển
AI model	Python, Keras/TensorFlow, Ultralytics, ONNX, onnx2tf, TFLite	Huấn luyện/chuẩn bị và chuyển đổi mô hình cho Android
Firmware	ESP32, NimBLE, SimpleFOC, AS5600, 3PWM	GATT server, parser lệnh và điều khiển hai BLDC

Các thư viện RobotApp gồm CameraX, TensorFlow Lite CPU/GPU/Support/Select TF Ops, OpenCV Android, Firebase Auth/Database, WebRTC native, OkHttp và Kotlin Coroutines. CaregiverApp dùng Firebase Auth/Database, Android Credentials, RecyclerView và WebRTC. Mô hình TFLite được đánh dấu `noCompress` để Android đóng gói nguyên dạng, thuận lợi cho memory mapping.

4.2 Hiện thực RobotApp

4.2.1 Khởi động, điều hướng và vòng đời dịch vụ

LoginActivity là launcher của RobotApp. Sau xác thực, MainActivity quản lý các fragment hồ sơ, theo dõi, chatbot và cuộc gọi. Activity cũng lắng nghe `webrtc_signal` để chuyển giao diện khi có lời mời gọi. Các foreground service gồm ReminderService và TrackingService. AndroidManifest khai báo camera, microphone, Internet, notification, foreground service và quyền BLE tương thích nhiều

phiên bản Android.

Quyền Bluetooth được tách theo nền tảng. Android 11 trở xuống dùng BLUETOOTH, BLUETOOTH_ADMIN và vị trí để quét. Android 12 trở lên dùng BLUETOOTH_SCAN và BLUETOOTH_CONNECT; cách phân chia này phù hợp tài liệu Android về quyền Bluetooth [39]. Camera được khai báo `required=false` để ứng dụng vẫn có thể cài trên thiết bị không báo camera theo feature flag, nhưng chức năng thị giác tất nhiên cần camera thực tế.

4.2.2 TrackingFragment và quyền sở hữu camera

`tracking_fragment.kt` là module lớn điều phối CameraX, mô hình pose, tracker, LSTM, hậu kiểm và điều khiển bấm. CameraX cung cấp `Preview` và `ImageAnalysis`; analyzer chuyển `ImageProxy` sang `bitmap/tensor` rồi đóng frame đúng thời điểm. Tài liệu CameraX mô tả use case được bind theo lifecycle, phù hợp cách fragment quản lý camera [38].

Khi không có cuộc gọi, fragment bind CameraX. Khi WebRTC bắt đầu, CameraX được unbind để tránh lỗi “camera already in use”. `video_call_fragment` lấy camera và đẩy bản sao local `VideoFrame` về tracking fragment. Frame được chuyển I420 sang RGB, giảm kích thước và đưa vào executor riêng. Nếu lần suy luận trước chưa xong, frame mới bị bỏ thay vì xếp hàng vô hạn. Khi kết thúc gọi, WebRTC capturer được dừng/dispose rồi CameraX bind lại.

Thiết kế chỉ tạo một `RobotControlManager` trong tracking fragment. Video call không tạo GATT client thứ hai. Điều này loại bỏ tranh chấp kết nối với cùng ESP32 và giữ một điểm duy nhất chịu trách nhiệm dừng.

4.2.3 Nạp và suy luận mô hình pose

RobotApp đóng gói hai biến thể pose trong assets: `yolo26n-pose_320.tflite` kích thước khoảng 12,03 MB và `yolo26n-pose_320_gpu.tflite` khoảng 11,84 MB. Tên tệp “yolo26n” là định danh artifact đang triển khai của đồ án. Interpreter ưu tiên cấu hình tương thích thiết bị; biến thể GPU nhằm giảm thời gian suy luận trên phần cứng hỗ trợ, còn CPU là đường dự phòng.

Pipeline tiền xử lý chuẩn hóa ảnh về kích thước 320×320 . Đầu ra được giải mã thành detection người, vùng bao, confidence và danh sách keypoint. Detection dưới ngưỡng bị bỏ, sau đó NMS loại vùng bao trùng. TFLite được chọn vì hướng tới triển

khai mô hình trên mobile/edge [28].

4.2.4 SORT, Kalman, Hungarian và Re-ID

Các lớp SortTracker, KalmanBoxTracker, HungarianAlgorithm và ReIdModule hiện thực tracking-by-detection. Mỗi detection gồm bbox, score, classId và keypoint. Kalman dự báo vùng bao kế tiếp; ma trận chi phí ghép được giải bằng Hungarian. Cách tiếp cận giữ độ phức tạp thấp phù hợp suy luận trên điện thoại, kế thừa tinh thần SORT [19] nhưng bổ sung tín hiệu pose và ngoại hình.

Hàm chi phí sử dụng 30% sai khác IoU, 32% khoảng cách tâm, 23% khoảng cách pose và 15% thay đổi diện tích. Match thông thường có chi phí tối đa 0,76; track được bảo vệ cho phép đến 0,88. Ngưỡng IoU cơ sở là 0,15 vì pose/bbox có thể rung mạnh. Track mới cần hai hit để xuất hiện. Track ID0 được giữ tối đa 45 frame, track nhiều 10 frame.

Re-ID trích histogram HSV vùng thân, nơi ít bị thay đổi hơn nên và mất khi người quay. Histogram không được cập nhật liên tục khi thiếu keypoint vì một vùng crop sai có thể làm “học nhầm” người khác. Khi detection mới phù hợp với hồ sơ track cũ, restoreWithId khôi phục định danh. Đây là Re-ID nhẹ, không tương đương embedding sâu của Deep SORT [20]; ưu điểm là chi phí tính toán thấp, hạn chế là nhảy với quần áo giống nhau và ánh sáng.

4.2.5 Khóa mục tiêu ID0 và sinh lệnh auto-follow

Đối tượng được chăm sóc được chuẩn hóa thành PRIMARY_TRACK_ID=0. Ở lần chọn đầu, người có vùng bao lớn thường được xem là gần robot nhất. Sau đó ID0 được truyền qua pipeline LSTM và điều khiển. Khi mất mục tiêu, logic phục hồi yêu cầu một người duy nhất, ở gần vị trí cũ, thuộc vùng trung tâm và ổn định; trong cảnh nhiều người, robot chờ thay vì tự đổi chủ thể.

Auto-follow so sánh tâm vùng bao với vùng mục tiêu. Nếu lệch ngang, hệ thống phát lệnh quay; nếu kích thước người cho thấy khoảng cách xa, phát lệnh tiến. Các mức quay hiện dùng kênh $y = \pm 0,50$ theo hệ tọa độ camera. Cách đặt dấu khác trực giác joystick nhưng đã được tách rõ theo từng hệ tọa độ. Lệnh cuối vẫn đi qua RobotControlManager, nhờ đó cùng được kiểm tra và format.

4.2.6 LSTM phát hiện té ngã

LstmFallDetector nạp `best_model_lstm_v2.tflite` khoảng 63,9 KB, dùng hai luồng CPU. Mỗi track có `FallDetectionQueue`; queue đủ 15 frame mới tạo tensor $1 \times 15 \times 69$ kiểu float. Output 1×2 được diễn giải là xác suất normal/fall. Model gốc Keras khoảng 193,8 KB, cho thấy TFLite giảm đáng kể kích thước đóng gói.

Máy trạng thái dùng threshold 0,5 và nhiều bộ đếm: 10 frame fall liên tiếp để vào Falling; 15 frame phù hợp để xác nhận trạng thái nằm; 20 frame đứng để phục hồi; track mất quá 60 frame thì dọn state. Khi tracker đổi ID hợp lệ, `transferStateTo` chuyển queue và trạng thái để chuỗi thời gian không bị mất. LSTM được dùng vì khả năng mô hình hóa phụ thuộc theo thời gian [18], nhưng đầu ra không được dùng đơn độc để phát cảnh báo.

4.2.7 Hậu kiểm té ngã

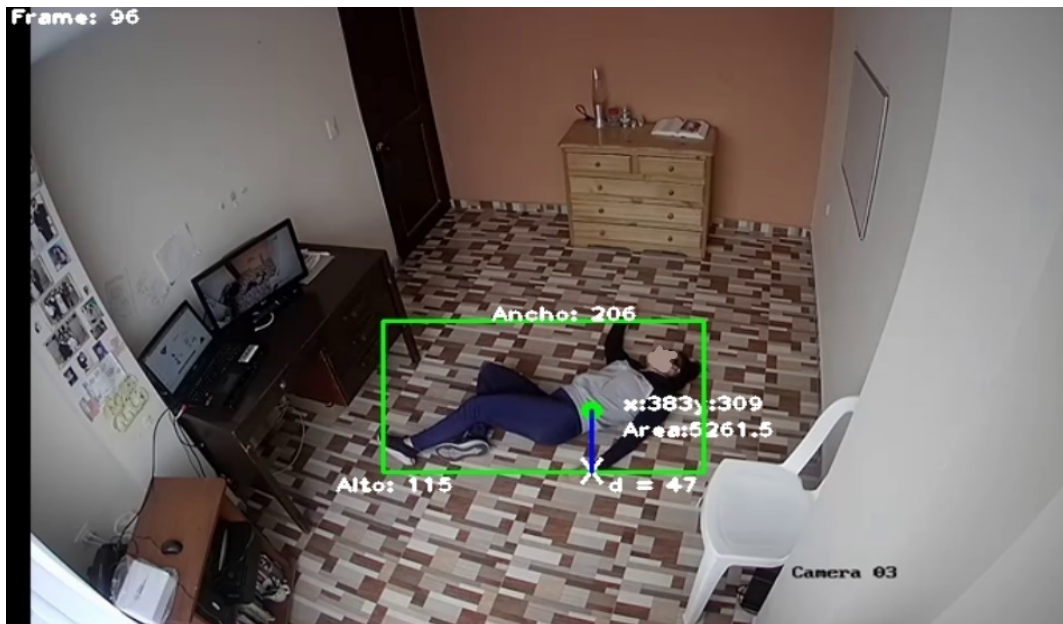
FallVerificationEngine nhận `FrameEvidence`: mục tiêu có thấy hay không, tỷ lệ y của thân trên, số keypoint, vùng bao có nằm trọn ảnh và aspect ratio. Baseline thân trên được cập nhật bằng trung bình trượt mũ với $\alpha = 0,08$ trong trạng thái bình thường. Khi có sự kiện, engine chấm bằng chứng qua các ngưỡng đã mô tả ở Chương 3.

Engine trả một trong ba loại: chưa quyết định, `Rejected(reason)` hoặc `AskForConfirmation(reason, score, evidence)`. Tách quyết định khỏi UI giúp unit test từng nhánh mà không cần camera. `isIncidentActive` và `canAcquireNewTarget` ngăn một sự cố tạo nhiều yêu cầu đồng thời. `resolveAsSafe`, `resolveAsDanger` và `reset` làm rõ vòng đời sự cố.

4.2.8 Luồng xác nhận và cảnh báo

TrackingService truyền `FallEvent` đến chatbot. Chatbot là owner duy nhất tạo `fall_alerts`, bảo đảm một record cho một sự cố. Robot trước hết dùng, phát câu hỏi TTS và mở cửa sổ nghe tối đa 10 s. Các cụm từ an toàn/nguy hiểm rõ được đối chiếu cục bộ để giảm độ trễ và tránh phụ thuộc LLM. Phản hồi mơ hồ gọi model phân loại không streaming và yêu cầu kết quả cấu trúc. Nếu phân loại lỗi hoặc người dùng im lặng, trạng thái được cập nhật nguy hiểm và gọi caregiver sau khi TTS hoàn tất.

Bản ghi cảnh báo chứa timestamp, trạng thái, phản hồi và bằng chứng như `verificationReason`, `visibleKeypoints`, `bboxFullyInside`,



Hình 4.1: Ví dụ khung hình té ngã trong dữ liệu tham khảo

upperBodyYRatio. Thông tin này giúp truy vết vì sao engine yêu cầu xác nhận, nhưng không phải là giải thích đầy đủ của mô hình LSTM.

4.2.9 Chatbot giọng nói

chatbot_fragment phối hợp ba lớp quản lý. SpeechRecognizerManager nhận dạng tiếng nói; GroqChatService xử lý hội thoại; còn AndroidTtsService tổng hợp tiếng nói. Giao diện chính tôi giản thành trạng thái, icon lớn, tên robot và câu đang nghe/nói. Hồ sơ người dùng được đọc từ Users/{uid} để xây prompt về cách xưng hô, tên caregiver, tình trạng và sở thích.

GroqChatService gọi endpoint /openai/v1/chat/completions qua OkHttp. Hội thoại streaming dùng model cấu hình qwen/qwen3.6-27b; các tác vụ phân loại ngăn dùng openai/gpt-oss-20b. Lịch sử được giới hạn 20 message để kiểm soát context. Dòng SSE có tiền tố data::; service đọc choices[0].delta.content, phát từng phần qua Kotlin Flow và lưu câu hoàn chỉnh khi stream kết thúc.

Tác vụ an toàn không dựa hoàn toàn vào sinh văn bản tự do. Hàm phân loại phản hồi té ngã quét từ khóa nguy hiểm/an toàn trước; chỉ trường hợp mơ hồ mới gọi API và parse JSON. detectCallIntent cũng có từ khóa, kiểm tra tên/quan hệ caregiver và model phân loại dự phòng. Khóa API được truyền vào service; bản phát hành phải lấy từ cấu hình không commit hoặc một proxy backend, không hard-code.

4.2.10 Lịch nhắc

ReminderService lắng nghe Schedules/{uid} và hồ sơ Users/{uid}. Khi tới thời điểm, service tạo log dưới reminder_logs/{uid}, truyền sự kiện đến chatbot và phát lời nhắc. Người cao tuổi có thể xác nhận, từ chối hoặc không phản hồi; log cập nhật status, câu trả lời, retry count và confirmed time. Cấu trúc này tách định nghĩa lịch khỏi lần thực hiện, cho phép caregiver xem lịch sử mà không làm thay đổi lịch gốc.

4.2.11 WebRTC trong RobotApp

video_call_fragment khởi tạo PeerConnectionFactory, nguồn audio/video, renderer và Firebase reference webrtc_signal. Khi RobotApp gọi, nó ghi offer và đẩy ICE vào robotCandidates; khi nhận gọi, nó đọc offer, tạo answer và nghe caregiverCandidates. Listener chỉ được dùng cho đúng pha và được gỡ trong cleanup.

WebRTC cung cấp mã hóa và truyền media; Firebase chỉ trao đổi SDP, ICE và trạng thái [31]. Trường emergencyReason=fall phân biệt cuộc gọi khẩn cấp với cuộc gọi thường. Khi kết thúc, status được đặt ended, track/capturer/renderer/PeerConnection được giải phóng và quyền camera trả lại tracking.

4.2.12 RobotControlManager và BLE

RobotControlManager lắng nghe robot_control. Hàm ánh xạ được tách để kiểm thử bốn điểm chuẩn. Mọi input được kiểm tra hữu hạn, giới hạn, tuổi và skew tương lai. Payload được format với Locale.US để dấu thập phân luôn là dấu chấm:

```
1 left = clamp((steering * 0.15 + translation) * 20, -20, 20)
2 right = clamp((steering * 0.15 - translation) * 20, -20, 20)
3 payload = "L%.2fR%.2f\n".format(Locale.US, left, right)
```

Listing 2: Định dạng lệnh BLE ở mức logic

Manager quét theo tên OhmniRobot và custom UUID, timeout quét 10 s, reconnect sau 3 s. Sau discovery, characteristic được cấu hình WRITE_TYPE_NO_RESPONSE. API ghi khác nhau theo phiên bản Android được xử lý tương thích. Watchdog 1.200 ms gọi STOP nếu luồng RTDB không cập nhật. stop() gỡ listener, callback, scan và GATT để tránh leak.

4.3 Hiện thực CaregiverApp

4.3.1 Khởi động, xác thực và hồ sơ

CaregiverApp có LoginActivity, SignUpActivity, MainActivity và ProfileFragment. Firebase Authentication cấp UID; profile được lưu dưới Users/{uid} dùng chung với RobotApp. MainActivity là launcher trong manifest hiện tại và khởi động IncomingCallService. Khi đóng gói sản phẩm, cần bảo đảm auth gate tại MainActivity để người chưa có phiên hợp lệ không đi thẳng vào màn hình chính.

4.3.2 Bật/tắt chức năng theo dõi

tracking_fragment ghi hai cờ robot_follow_enabled và ai_fall_detection_enabled. Fragment đồng thời theo dõi .info/connected để phản ánh kết nối RTDB. Khi tắt auto-follow, CaregiverApp ghi $x = 0, y = 0, ts = now$; đây là hành vi quan trọng vì chỉ đối cờ có thể để lại lệnh chuyển động trước đó.

4.3.3 Lịch và nhật ký

ScheduleFragment đọc danh sách Schedules/uid; AddScheduleFragment tạo/cập nhật lịch; ScheduleAdapter xử lý RecyclerView và công tắc active. Tên trường được giữ đúng là active, tránh sai lệch với isActive. Ba nhánh nhật ký có fragment/adaptor riêng: reminder, fall alert và call history. Danh sách cảnh báo/cuộc gọi được truy vấn theo timestamp trong bảy ngày gần nhất.

4.3.4 Cuộc gọi đến nền

IncomingCallService là foreground service lắng nghe webrtc_signal. Khi status là calling và callerRole là robot, service tạo notification và có thể mở IncomingCallActivity toàn màn hình. Nếu emergencyReason=fall, tiêu đề/nội dung thông báo nêu đây là cuộc gọi do chưa xác nhận an toàn. Manifest khai báo notification, foreground service và full-screen intent; hành vi thực tế còn phụ thuộc chính sách hệ điều hành và quyền người dùng.

4.3.5 Video call và joystick

video_call_fragment hiện thực cả hai vai trò caller/callee, renderer cục bộ/từ xa và signaling tương ứng. Trong màn hình gọi, joystick tính $x = \Delta y/r, y = -\Delta x/r$,

chặn $[-1, 1]$ và throttle khoảng 20 Hz. Khi người dùng bắt đầu lái, app tắt auto-follow để hai nguồn không tranh chấp. ACTION_UP/ACTION_CANCEL, kết thúc gọi và rời fragment đều gửi STOP.

CaregiverApp không có BLE và không biết dấu vật lý của motor. Ranh giới này giúp điện thoại caregiver hoạt động ở bất kỳ khoảng cách Internet nào; RobotApp gần phần thân chịu trách nhiệm kiểm tra lệnh và hiệu chỉnh motor.

4.4 Hiện thực dữ liệu dùng chung

Firebase listener cung cấp cập nhật gần thời gian thực, còn write được phản ánh cục bộ trước khi đồng bộ máy chủ [29]. Vì vậy timestamp trong lệnh là bắt buộc: tính “realtime” của callback không đồng nghĩa dữ liệu luôn mới. Các class model như UserProfile, ScheduleItem, FallAlertEntry, ReminderLogEntry và CallHistoryEntry phản ánh dữ liệu JSON.

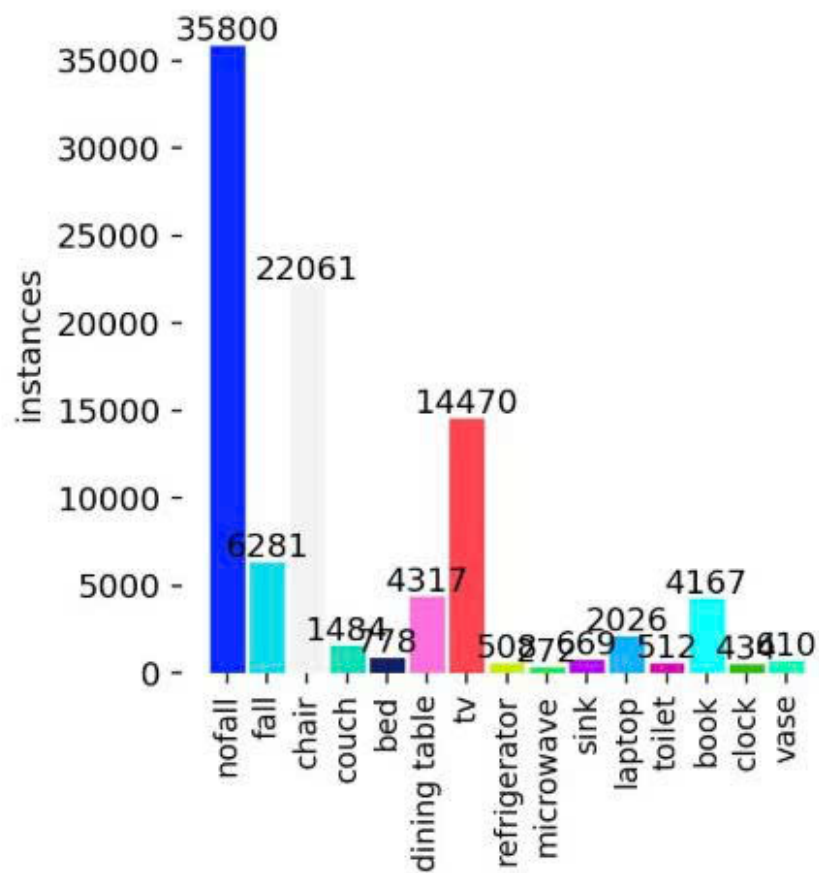
Snapshot hiện dùng kết hợp node theo UID và node toàn cục. Điều này giảm độ phức tạp demo một robot nhưng chưa đủ cách ly nhiều gia đình. Trong triển khai thực, rules phải từ chối mặc định, cho phép theo UID/family/robot và validate từng trường [30]. Chuyển schema cần cập nhật đồng thời hai app; nếu chỉ sửa một phía sẽ tạo lỗi im lặng do listener đọc sai path.

4.5 Huấn luyện và chuyển đổi mô hình AI

4.5.1 Dữ liệu và bài toán

Khởi thị giác dùng mô hình pose tổng quát để lấy 17 keypoint người và mô hình LSTM phân loại chuỗi. Dữ liệu té ngã tham khảo CAUCAFall, vốn cung cấp video fall/no-fall trong điều kiện ít kiểm soát [25]. Dữ liệu dạng frame cần được tách theo video/chủ thể trước khi train/validation/test; nếu chia ngẫu nhiên từng frame, các frame gần nhau có thể lọt sang tập test và làm chỉ số lạc quan.

Hình 4.2 cho thấy ví dụ phân bố nhãn của một tập dữ liệu liên quan: các lớp có chênh lệch lớn. Đối với bài toán nhị phân fall/normal, mất cân bằng phải được xử lý bằng lấy mẫu, trọng số lớp và báo cáo precision/recall thay vì chỉ accuracy.



Hình 4.2: Ví dụ phân bố số lượng thể hiện sự mất cân bằng lớp trong dữ liệu

4.5.2 Xây dựng vector đặc trưng chuỗi

Từ pose, tọa độ keypoint được chuẩn hóa theo vùng bao hoặc kích thước ảnh để giảm phụ thuộc khoảng cách camera. Confidence giúp biểu diễn keypoint bị khuất. Các đặc trưng hình học bổ sung tỷ lệ vùng bao và trạng thái thân người. Cửa sổ trượt 15 frame cân bằng giữa đủ thông tin chuyển động và độ trễ. Mỗi cửa sổ có nhãn normal/fall; các chuỗi chuyển tiếp cần được gán nhất quán để tránh model học sai thời điểm.

Quy trình huấn luyện LSTM cần lưu seed, split, scaler/chuẩn hóa, class mapping và phiên bản thư viện. Chỉ artifact `best_model_lstm.h5` và model đã chuyển đổi hiện có trong thư mục bàn giao; vì vậy báo cáo không suy diễn epoch hay metric không còn log. Việc tái lập đầy đủ yêu cầu bổ sung notebook/script huấn luyện và lịch sử loss/metric.

4.5.3 Chuyển Keras LSTM sang TFLite

Script `convert_to_tflite.py` nạp model H5, đọc `input_shape` để xác nhận sequence length và feature count, sau đó dùng `TFLiteConverter.from_keras_model`. Supported ops gồm TFLite built-ins và Select TF Ops để bao phủ toán tử LSTM. Model đầu ra được nạp lại bằng Interpreter, allocate tensor và kiểm tra shape. Artifact cuối `best_model_lstm_v2.tflite` được sao chép vào assets RobotApp.

4.5.4 Xuất mô hình pose qua ONNX

Các script dùng Ultralytics nạp `yolo26n-pose.pt`, xuất ONNX với batch 1 và kích thước ảnh cố định. Biến thể `convert_to_tflite_320.py` cố định input 320×320 ở bước export; kích thước này được “bake” vào graph nên Android phải tiền xử lý đúng. ONNX đóng vai trò biểu diễn trung gian để kiểm tra input/output và chuyển bằng `onnx2tf`.

Một khó khăn là graph end-to-end chứa NMS có thể tạo toán tử không tương thích GPU delegate. `convert_to_tflite_gpu.py` đặt `module.end2end=false` trước export để bỏ NMS head, chuyển NMS sang Kotlin. Điều này làm output thô lớn hơn nhưng graph chỉ còn các toán tử thuận lợi hơn cho delegate. Android chịu trách nhiệm decode, threshold và NMS.

4.5.5 Chuyển ONNX sang TFLite và FP16

`onnx2tf` chuyển graph ONNX tĩnh sang SavedModel/TFLite. Script FP16 dùng `TFLiteConverter.from_saved_model`, bật `Optimize.DEFAULT` và đặt

`supported_types=[tf.float16]`. Trọng số FP16 giảm kích thước và băng thông bộ nhớ; input/output có thể vẫn float32 tùy converter/delegate. Sau chuyển đổi, script kiểm tra tensor shape/type, chạy một input thử và kiểm tra output hữu hạn trước khi copy vào assets.

Bảng 4.2: Các artifact mô hình chính

Artifact	Định dạng	Kích thước gần đúng	Mục đích
best_model_lstm.h5	Keras H5	193,8 KB	Model huấn luyện/nguồn chuyển đổi
best_model_lstm_v2.tflite	TFLite	63,9 KB	LSTM triển khai Android
yolo26n-pose.pt	PyTorch	7,88 MB	Trọng số pose nguồn
yolo26n-pose_320.onnx	ONNX	Theo artifact	Graph trung gian input cố định
yolo26n-pose_320.tflite	TFLite	12,03 MB	Model CPU/fallback trong RobotApp
yolo26n-pose_320_gpu.tflite	TFLite	11,84 MB	Graph hướng tới GPU delegate
yolo26n-pose_320_fp16.tflite	TFLite FP16	6,28 MB	Biến thể giảm trọng số/băng thông

4.5.6 Kiểm tra tương đương sau chuyển đổi

Kiểm tra “model mở được” chưa đủ để kết luận tương đương. Cần chạy cùng tập ảnh qua PyTorch, ONNX và TFLite; đưa output về cùng layout; so sai số số học; rồi so detection sau decode/NMS theo confidence, IoU, keypoint và số lượng người. Với LSTM, cần so vector xác suất trên cùng cửa sổ 15×69 . Chênh lệch FP16 phải được đánh giá ở metric cuối, không chỉ chênh tensor trung gian.

4.6 Hiện thực firmware phần thân robot

4.6.1 Khởi tạo phần cứng

Firmware dùng ESP32 với hai AS5600 trên hai bus I2C. Motor trái dùng SDA=19/SCL=18 và motor phải dùng SDA=23/SCL=5. Driver trái dùng IN1=32, IN2=33, IN3=25, EN=22; driver phải dùng IN1=26, IN2=27, IN3=14, EN=12. Hai bus riêng tránh xung đột địa chỉ mặc định giống nhau của AS5600.

Hai BLDCMotor có 7 pole pairs, liên kết encoder và driver 3PWM. Điều chế SpaceVectorPWM, controller velocity, PID và LPF được thiết lập trước `init/initFOC`. Firmware gọi `loopFOC` liên tục; không đặt delay trong loop điều khiển.

4.6.2 BLE GATT server

NimBLE khởi tạo tên OhmniRobot, công suất phát cao và advertising có service UUID trong scan response. Characteristic chỉ có WRITE_NR. Callback kết nối bật cờ và tiếp tục advertising; callback ngắt đặt target hai bánh về 0. Callback write lấy `std::string`, trim và gọi parser.

Parser tìm vị trí L và R, tách hai substring và dùng `toFloat`. Khi hợp lệ, target và `lastCmdTime` được cập nhật. Mức Android đã bảo đảm format, nhưng firmware tương lai nên dùng parser xác nhận toàn chuỗi và `isfinite` để phòng client lỗi/độc hại.

4.6.3 Deadzone và watchdog

`applyDeadzone` đưa $|v| < 0,5$ về 0; nâng biên độ khác 0 nhưng dưới 3 lên 3; rồi constrain $[-40, 40]$. Watchdog 800 ms kiểm tra khi BLE đang kết nối. Nếu timeout và target khác 0, firmware ghi log và dừng. Vì `motor.move` được gọi sau bước watchdog, setpoint 0 có hiệu lực ngay vòng hiện tại.

```
1 loopFOC(left); loopFOC(right)
2 if connected and now - lastCommand > 800 ms:
3     targetLeft = 0; targetRight = 0
4 leftCommand = applyDeadzone(targetLeft)
5 rightCommand = applyDeadzone(targetRight)
6 move(leftCommand); move(rightCommand)
```

Listing 3: Giả mã vòng điều khiển firmware

4.7 Cấu hình và triển khai

Hai ứng dụng phải trở cùng Firebase project và có tệp cấu hình Android tương ứng. RTDB Rules phải được triển khai trước khi dùng dữ liệu thật. STUN/TURN cần được cấu hình cho cả hai peer; TURN nên dùng credential ngắn hạn. API key Groq không xuất hiện trong Git hoặc báo cáo. Trên thiết bị robot, phải cấp camera, microphone, notification, BLE scan/connect; trên caregiver phải cấp camera, microphone, notification và full-screen intent theo chính sách hệ điều hành.

Quy trình tích hợp khuyến nghị:

1. Nạp firmware, kiểm tra encoder và motor với bánh nhắc khỏi mặt đất.
2. Cài RobotApp, kiểm tra quyền và kết nối BLE; gửi STOP trước mọi thử nghiệm.

3. Kiểm tra riêng dấu bốn lệnh chuẩn ở tốc độ thấp.
4. Cài CaregiverApp, xác nhận cùng tài khoản/project và đồng bộ profile/ lịch.
5. Kiểm tra cuộc gọi trên cùng mạng, sau đó hai mạng khác nhau/TURN.
6. Bật tracking rồi fall detection; thử kịch bản an toàn trước kịch bản té ngã dừng lại.
7. Kiểm tra mất Internet, mất BLE, khóa màn hình và vòng đời ứng dụng.

4.8 Những điểm hiện thực cần lưu ý

Mã nguồn phản ánh một nguyên mẫu đang phát triển. Một số node Firebase toàn cục phù hợp demo nhưng chưa phù hợp nhiều robot. CaregiverApp có màn chatbot placeholder trong khi chatbot thật ở RobotApp. Secret cần được externalize. Parser firmware cần được gia cố. Phiên bản model và graph chuyển đổi cần manifest/checksum để tránh nhầm artifact. Các điểm này không phủ nhận chức năng đã hiện thực, nhưng là điều kiện cần giải quyết trước triển khai thực tế.

4.9 Kết luận chương

Chương đã mô tả cách kiến trúc được chuyển thành hai ứng dụng Android, chuỗi mô hình AI, cơ chế chuyển đổi và firmware ESP32. Các chi tiết được gắn với class, path dữ liệu, hằng số và artifact cụ thể, qua đó bảo đảm báo cáo có thể đối chiếu mã nguồn. Chương tiếp theo trình bày kế hoạch kiểm thử, kết quả được xác nhận và thảo luận giới hạn của nguyên mẫu.

Chương 5 – Kết quả và thảo luận

5.1 Mục tiêu đánh giá

Đánh giá nguyên mẫu nhằm trả lời bốn câu hỏi: các khối có được hiện thực đúng kiến trúc hay không; hợp đồng liên ứng dụng có nhất quán hay không; các cơ chế an toàn có điểm kiểm chứng hay không; và bằng chứng hiện có cho phép kết luận tới mức nào. Do hệ thống gồm AI, cloud, mobile và phần cứng, một con số accuracy đơn lẻ không đại diện cho toàn hệ thống.

Chương này phân biệt ba loại bằng chứng. *Bằng chứng tĩnh* được rút từ mã nguồn, cấu hình và artifact. *Bằng chứng tự động* đến từ unit test/build có thể chạy lặp. *Bằng chứng tích hợp* đến từ thử nghiệm hai điện thoại, Internet, BLE và robot thật. Chỉ các kết quả có artifact hoặc được nhóm dự án xác nhận mới được ghi “đạt”; hạng mục thiếu log định lượng được ghi rõ để tránh tạo số liệu.

5.2 Cấu hình và tiêu chí đánh giá

Bảng 5.1: Cấu hình phần mềm được đánh giá

Hạng mục	Cấu hình
RobotApp	Android/Kotlin; minSdk 23; target/compile SDK 36; CameraX; TFLite; OpenCV; Firebase; WebRTC; BLE
CaregiverApp	Android/Kotlin; minSdk 23; target/compile SDK 36; Firebase; WebRTC
AI trên Android	Pose input 320×320 ; LSTM input 15×69 ; CPU/GPU TFLite artifacts
Firmware	ESP32; NimBLE; SimpleFOC; hai AS5600; hai BLDC; velocity limit 40 rad/s
Cloud	Firebase Authentication/RTDB; Groq HTTPS; STUN/TURN cho WebRTC
Giao thức điều khiển	RTDB x, y, ts ; BLE $L<float>R<float>\backslash n$

Các chỉ số AI thích hợp gồm precision, recall, F1 và confusion matrix ở cấp sự cố. Với TP là phát hiện đúng sự cố, FP là cảnh báo giả, FN là bỏ sót:

$$Precision = \frac{TP}{TP + FP}, \quad (5.1)$$

$$Recall = \frac{TP}{TP + FN}, \quad (5.2)$$

$$F_1 = 2 \frac{Precision \cdot Recall}{Precision + Recall}. \quad (5.3)$$

Đơn vị đánh giá phải là một sự kiện té ngã, không phải từng frame; nhiều frame cảnh báo liên tiếp của cùng sự cố chỉ tính một sự kiện. Với tracking, chỉ số cần theo dõi là số lần ID0 đổi sai, tỷ lệ phục hồi đúng và thời gian mất mục tiêu. Với điều khiển, chỉ số là độ trễ end-to-end, tần số lệnh, tỷ lệ lệnh bị loại và thời gian dừng sau mất kênh. Với WebRTC, chỉ số gồm thời gian thiết lập, tỷ lệ cuộc gọi thành công, RTT, jitter và packet loss từ getStats.

5.3 Kết quả hiện thực và artifact

5.3.1 Mức độ hoàn thành thành phần

Bảng 5.2: Kết quả hiện thực theo thành phần

Thành phần	Trạng thái	Bằng chứng
Thành phần	Trạng thái	Bằng chứng
RobotApp Android	Đã hiện thực	24 tệp Kotlin lỗi; manifest; resource; model trong assets; APK debug
CaregiverApp Android	Đã hiện thực	24 tệp Kotlin; giao diện quản lý, diary, lịch, call; APK debug
Pose trên thiết bị	Đã tích hợp	Hai model TFLite CPU/GPU và pipeline decode/NMS
Tracking và Re-ID	Đã tích hợp	SORT, Kalman, Hungarian, HSV histogram, ID0
LSTM và hậu kiểm	Đã tích hợp	Model TFLite, queue 15×69, state machine, verification engine
Hội thoại giọng nói	Đã tích hợp RobotApp	SpeechRecognizer, Groq streaming/classify, Android TTS

Lịch nhắc và nhật ký	Đã tích hợp	Schedules, reminder_logs, adapters và service
WebRTC	Đã tích hợp	Caller/callee, SDP/ICE RTDB, incoming foreground service
Điều khiển robot	Đã tích hợp	Joystick, RTDB, RobotControlManager, BLE GATT
Firmware BLDC	Đã hiện thực	Sketch ESP32, NimBLE, SimpleFOC, dual I2C, watchdog
Backend riêng/MQTT	Không thuộc hiện thực	Hệ thống dùng Firebase RTDB; không có broker/server signaling riêng
Chatbot Caregiver-App	Placeholder	Logic hội thoại thật chỉ nằm ở RobotApp

5.3.2 Kết quả đóng gói Android

Nhóm dự án xác nhận cả hai ứng dụng build thành công. Trong cây build có APK debug RobotApp khoảng 504,24 MiB và CaregiverApp khoảng 51,03 MiB. Chênh lệch chủ yếu do RobotApp đóng gói nhiều runtime native và thư viện AI (TFLite, GPU/Select TF Ops, OpenCV, WebRTC) cùng model; CaregiverApp không chứa model pose/LSTM và stack AI.

Kích thước 504 MiB là chấp nhận được cho prototype debug nhưng không phù hợp phát hành. Bản release cần bật shrinking, loại ABI không dùng, kiểm tra trùng native library, cân nhắc Play App Bundle/split APK và bỏ dependency AI dư. Model thực chỉ chiếm khoảng vài chục MB; phần lớn dung lượng tăng đến từ runtime/native binaries và ký hiệu debug.

Bảng 5.3: Artifact đóng gói hiện có

Artifact	Kích thước	Ý nghĩa
RobotApp debug APK	504,24 MiB	Bản debug tích hợp AI/WebRTC/BLE
CaregiverApp debug APK	51,03 MiB	Bản debug quản lý/WebRTC

5.3.3 Kết quả artifact mô hình

Chuỗi chuyển đổi đã tạo đủ artifact H5, PT, ONNX, SavedModel và TFLite. LSTM giảm từ khoảng 193,8 KB xuống 63,9 KB. Biến thể pose FP16 khoảng 6,28

MB, gần một nửa biến thể float32 tương ứng; biến thể app CPU/GPU lần lượt khoảng 12,03/11,84 MB. Đây là kết quả về kích thước tệp, không tự động chứng minh tốc độ hoặc độ chính xác.

Bảng 5.4: Đánh giá định tính các biến thể mô hình

Biến thể	Lợi ích kỳ vọng	Điều phải đo
Float32 CPU	Tương thích rộng, dễ debug	Latency, nhiệt, FPS, bộ nhớ
Graph GPU	Tăng throughput nếu delegate hỗ trợ toàn graph	Tỷ lệ op chạy GPU, overhead copy, fallback
FP16	Giảm dung lượng/băng thông bộ nhớ	Sai số keypoint, mAP/PCK, latency thật
NMS ngoài graph	Tăng tính tương thích delegate	Độ đúng decode/NMS Kotlin so với nguồn

5.4 Kiểm thử tự động ở mức logic

5.4.1 Ánh xạ điều khiển

RobotControlManagerTest định nghĩa bốn test bảo vệ hiệu chỉnh dấu. Sai sót dấu là rủi ro cao vì tên “trái/phải/tiến/lùi” đi qua ba hệ tọa độ: màn hình, camera và chiều lắp motor. Các assertion dùng sai số 0,001.

Bảng 5.5: Bộ unit test ánh xạ điều khiển

Ca kiểm thử	Input (x, y)	Kỳ vọng (L, R)
forward calibrated	$(-1, 0)$	$(-20, 20)$
backward calibrated	$(1, 0)$	$(20, -20)$
positive steering	$(0, 1)$	$(3, 3)$
negative steering	$(0, -1)$	$(-3, -3)$

Bốn test này kiểm tra điểm chuẩn, nhưng chưa bao phủ clipping, NaN/infinity, vùng đi thẳng, deadzone, timestamp và format Locale.US. Các ca đó được đề xuất bổ sung trước khi release.

5.4.2 Hậu kiểm té ngã

FallVerificationEngineTest định nghĩa bốn nhánh quan trọng. Engine trước hết được “arm” bằng năm frame đứng và một raw fall, sau đó đánh giá tình huống.

Các test xác nhận khả năng tách engine khỏi Android UI. Coverage còn thiếu: khóa

Bảng 5.6: Bộ unit test hậu kiểm té ngã

Ca	Kích thích	Kỳ vọng
clipped bbox after fall	Thân thấp, bbox không nằm trọn ảnh	Hỏi ngay
stable upper body top 30%	Ba frame thân trên ở vùng an toàn	Từ chối sự cố
lower occluded horizontal	Thân thấp, 7 keypoint, aspect 1,6 trong ba frame	Hỏi xác nhận
lost target after fall	Mất mục tiêu ba frame	Hỏi xác nhận

một sự cố, phục hồi 10 frame, baseline EMA, ngưỡng biên, reset, safe/danger resolution và chuỗi nhiễu xen kẽ.

5.5 Đánh giá hợp đồng tích hợp bằng kiểm tra tĩnh

5.5.1 Nhất quán Firebase

Đối chiếu hai ứng dụng cho thấy các path quan trọng trùng nhau: robot_control, robot_follow_enabled, ai_fall_detection_enabled, webrtc_signal, fall_alerts, call_history, Users, Schedules và reminder_logs. Offer/answer dùng cùng trường sdp,type. ICE dùng candidate,sdpMid,sdpMLineIndex; hai nhánh candidate được tách theo vai trò.

Một điểm cần chú ý là nhiều path vẫn toàn cục. Với một robot demo, hai app cùng thấy đúng dữ liệu. Với hai gia đình, lệnh/call/cảnh báo có thể trộn nếu rules cho phép. Vì vậy kết quả “nhất quán” chỉ áp dụng schema một robot hiện tại, chưa chứng minh multi-tenant.

5.5.2 Nhất quán BLE

Tên thiết bị, service UUID và characteristic UUID trong RobotControlManager trùng firmware. Android format L trước R, hai số có dấu chấm và newline; firmware trim rồi tách tại L/R. Android giới hạn $[-20, 20]$, hẹp hơn firmware $[-40, 40]$, tạo lớp an toàn bổ sung. Characteristic đều dùng write without response.

5.5.3 Chuỗi timeout

Ba timeout có thứ tự hợp lý: firmware 800 ms, Android resend/watchdog 1.200 ms và độ mới RTDB 2.000 ms. Firmware dừng sớm nhất khi không có gói BLE; Android dừng nếu nguồn RTDB ngừng; timestamp chặn lệnh cache khi reconnect. Tần suất

joystick khoảng 20 Hz tương ứng chu kỳ 50 ms, nhỏ hơn nhiều so với 800 ms, do đó có biên cho jitter bình thường.

5.5.4 Quyền sở hữu tài nguyên

Mã nguồn chỉ định tracking fragment là owner của RobotControlManager. Luồng cuộc gọi chia sẻ local WebRTC frame về AI và unbind CameraX. Đây là kết quả kiến trúc quan trọng vì xung đột camera/GATT thường không xuất hiện trong unit test. Kiểm thử vòng đời trên thiết bị vẫn cần bao phủ gọi đến khi app nền, từ chối, mất mạng, xoay màn hình và kết thúc bất thường.

5.6 Kịch bản kiểm thử chức năng đầu cuối

Bảng 5.7: Danh mục kiểm thử đầu cuối

Mã	Kịch bản	Thao tác/kích thích	Kết quả mong đợi
E01	Đăng nhập dùng chung	Đăng nhập cùng tài khoản hai app	Cùng UID/hồ sơ, không lộ user khác
E02	Tạo lịch nhắc	Tạo lịch từ caregiver, chờ thời điểm	Robot phát TTS, log xuất hiện
E03	Hội thoại bình thường	Nói câu tiếng Việt khi robot lắng nghe	STT-LLM-TTS hoàn thành
E04	Bật auto-follow	Chỉ một người đứng trước robot	ID0 ổn định, robot bám trong giới hạn
E05	Người thứ hai đi qua	Người nhiều cắt ngang mục tiêu	ID0 không đổi sai
E06	Mất mục tiêu ngắn	ID0 ra khỏi ảnh rồi trở lại	Khôi phục trong điều kiện cho phép
E07	Nằm chủ động	Người từ từ nằm nhưng còn rõ/an toàn	Không cảnh báo sai hoặc xác nhận an toàn
E08	Té ngã dừng lại	Thực hiện với đệm và người giám sát	Robot dừng, hỏi, tạo một alert

Mã	Kịch bản	Thao tác/kích thích	Kết quả mong đợi
E09	Phản hồi an toàn	Trả lời “tôi ổn” rõ ràng	Cập nhật safe, không gọi khẩn cấp
E10	Không phản hồi	Im lặng quá cửa sổ nghe	Cập nhật danger/no-response, gọi caregiver
E11	Video call	Gọi giữa hai mạng khác nhau	Offer/answer/ICE, audio/video kết nối
E12	AI trong cuộc gọi	Bật tracking khi call	Một camera, frame chia sẻ, không crash
E13	Joystick	Kéo bốn hướng rồi thả	Đúng dấu; thả gửi STOP
E14	Mất Internet	Ngắt mạng khi joystick đang kéo	Không nhận lệnh mới; watchdog dừng
E15	Mất BLE	Tắt Bluetooth/ngắt nguồn tablet	Callback/watchdog firmware dừng
E16	Lệnh cũ	Ghi timestamp cũ hơn 2 s	RobotApp bỏ lệnh và dừng
E17	Input lỗi	NaN/vượt miền/thiếu timestamp	Không chuyển động
E18	Cuộc gọi khẩn nên	Caregiver app ở nền, robot gọi fall	Notification/full-screen nêu đúng lý do

Bảng là bộ kiểm thử hồi quy đề xuất. Các kịch bản phần cứng phải bắt đầu với bánh xe nhấc khỏi mặt đất, tốc độ thấp, vùng trống và nút ngắt nguồn sẵn sàng. Không cho người cao tuổi thật thực hiện té ngã; chỉ dùng diễn viên khỏe mạnh, đệm bảo vệ và giám sát.

5.7 Thiết kế phép đo hiệu năng

5.7.1 Độ trễ pipeline AI

Mỗi frame cần gắn timestamp t_0 khi nhận, t_1 sau preprocess, t_2 sau interpreter, t_3 sau decode/NMS, t_4 sau tracking/LSTM và t_5 khi UI/lệnh được cập nhật. Báo cáo median, P90, P95 thay vì chỉ trung bình. FPS hiệu dụng phải tính frame thật được xử lý; drop-frame count phản ánh quá tải.

$$T_{AI} = (t_1 - t_0) + (t_2 - t_1) + (t_3 - t_2) + (t_4 - t_3). \quad (5.4)$$

Nhiệt và pin ảnh hưởng mạnh sau vài phút. Do đó benchmark cần warm-up 30–60 s, chạy ít nhất 10 phút cho mỗi CPU/GPU/FP16, ghi model thiết bị, Android version, delegate, thread count và độ phân giải camera.

5.7.2 Độ trễ điều khiển

Caregiver gắn sequence number và t_c vào lệnh; RobotApp log t_r khi listener nhận và t_b khi ghi BLE; ESP32 log micros/millis khi callback nhận. Nếu đồng hồ thiết bị chưa đồng bộ, đo từng chặng hoặc quay video tốc độ cao; không trừ trực tiếp timestamp khác clock. Độ trễ cần báo cáo cùng packet loss và khoảng cách BLE.

5.7.3 WebRTC

Thu thập RTCStatsReport định kỳ: candidate pair, currentRoundTripTime, packetsLost, jitter, framesEncoded/Decoded, framesDropped và availableOutgoingBitrate. Thử cùng Wi-Fi, Wi-Fi-4G và hai mạng NAT khó; ghi rõ có dùng TURN. Thời gian setup tính từ calling đến connected.

5.7.4 Độ chính xác té ngã

Tập kiểm thử phải độc lập theo chủ thể/video, bao gồm đi, ngồi, nằm, cúi nhặt đồ, khuất người, ra khỏi ảnh, té nhiều hướng và nhiều điều kiện sáng. So sánh ít nhất ba cấu hình: pose/luật một frame; pose+LSTM; pose+LSTM+hậu kiểm. Ablation này đo được đóng góp của mỗi tầng thay vì chỉ nêu pipeline cuối.

5.8 Thảo luận kết quả

5.8.1 Điểm mạnh

Kết quả nổi bật nhất là mức tích hợp xuyên suốt. Lệnh caregiver không dừng ở giao diện mà đi đến hai setpoint motor. Cảnh báo AI không dừng ở nhấn mà nổi với hỏi xác nhận, bản ghi và cuộc gọi. Camera được dùng chung đúng vòng đời. Các chi tiết timestamp, watchdog, khóa sự cố và ID0 cho thấy thiết kế đã xử lý nhiều lỗi đặc thù hệ thống thời gian thực.

Việc chạy pose và LSTM tại RobotApp giảm nhu cầu gửi video liên tục lên cloud. Hệ thống vẫn dùng Internet cho điều phối/hội thoại/call, nhưng quyết định thị giác cơ bản ở gần nguồn. Tách CaregiverApp khỏi BLE cũng giảm phụ thuộc khoảng cách vật lý và giữ hiệu chỉnh motor ở một nơi.

5.8.2 Hạn chế

Thứ nhất, chưa có bảng metric huấn luyện/validation và benchmark thiết bị được lưu cùng artifact; vì vậy không thể đưa ra mAP, PCK, precision/recall, FPS hay latency đáng tin cậy. Thứ hai, dữ liệu té ngã dừng lại khác người cao tuổi thật; domain shift có thể lớn. Thứ ba, camera RGB gây vấn đề riêng tư và có điểm mù. Thứ tư, Groq/Firebase/TURN là phụ thuộc ngoài; mất Internet làm suy giảm chức năng. Thứ năm, node toàn cục và auth/rules cần hoàn thiện cho multi-tenant. Thứ sáu, APK RobotApp debug còn quá lớn.

Re-ID histogram nhẹ không đủ mạnh khi nhiều người mặc màu giống nhau. Logic người có bbox lớn nhất chỉ là heuristic ban đầu. LSTM 15 frame tạo trễ phụ thuộc FPS và có thể nhận cửa sổ không đều khi drop-frame. Bộ hậu kiểm dùng ngưỡng thủ công, cần calibration theo góc camera/nhà. Parser firmware chấp nhận `String.toFloat`, nên cần validate nghiêm hơn.

5.8.3 Môi đe dọa tới tính hợp lệ

Nội tại: kết quả có thể phụ thuộc actor, trình tự kịch bản, vị trí camera và việc điều chỉnh ngưỡng trên cùng dữ liệu. *Ngoại tại:* một hoặc vài nhà/thiết bị không đại diện mọi căn hộ, mạng và người cao tuổi. *Đo lường:* accuracy theo frame có thể thổi phồng hiệu năng; clock khác nhau gây sai độ trễ. *Kết luận:* số mẫu nhỏ làm confidence interval rộng. Giảm rủi ro bằng protocol cố định, split theo chủ thể, log tự động, blind annotation và công bố cả trường hợp thất bại.

5.9 Mức đáp ứng mục tiêu

Bảng 5.8: Đối chiếu mục tiêu và kết quả

Mục tiêu	Mức đạt	Nhận xét
Hai ứng dụng theo vai trò	Đạt nguyên mẫu	Có APK và module chức năng chính
Firmware thân robot	Đạt nguyên mẫu	BLE, dual BLDC, encoder, FOC, watchdog
Theo dõi người và auto-follow	Đạt hiện thực	Có ID0, SORT/Re-ID và cầu nối motor; cần metric dài hạn
Phát hiện/xác nhận té ngã	Đạt hiện thực	Pose+LSTM+hậu kiểm+voice+alert; chưa có metric test độc lập
Hội thoại và nhắc lịch	Đạt nguyên mẫu	STT–Groq–TTS và Schedules/logs
Video call/điều khiển xa	Đạt nguyên mẫu	WebRTC signaling RTDB và joystick
Đánh giá định lượng toàn diện	Chưa hoàn tất	Cần log dataset, benchmark thiết bị và thử nghiệm nhiều bối cảnh
Sẵn sàng sản phẩm/y tế	Ngoài phạm vi	Cần security, safety, UX, pháp lý và thử nghiệm thực địa

5.10 Khuyến nghị trước nghiệm thu và trình diễn

Trước buổi trình diễn, nên đóng băng commit của bốn khối, lưu checksum model, build release/debug từ môi trường ghi phiên bản và chạy danh mục E01–E18. Mỗi thất bại cần lưu thời gian, thiết bị, logcat/serial và video. Chuẩn bị chế độ demo offline tối thiểu cho AI/BLE, kiểm tra credential TURN/Groq/Firebase, sạc đầy thiết bị và thử nút dừng vật lý.

Các bảng kết quả định lượng nên được điền từ log thật theo mẫu ở Phụ lục. Việc để trống có chú thích tốt hơn một con số không truy nguồn. Nếu thời gian hạn chế, ưu tiên đo recall/false alarm theo sự cố, latency P95 AI, thời gian dừng khi mất lệnh, tỷ lệ cuộc gọi thành công và số lần ID switch.

5.11 Kết luận chương

Nguyên mẫu đã hiện thực đầy đủ các khối kiến trúc và tạo được hai ứng dụng đóng gói, model triển khai cùng firmware. Kiểm tra tĩnh xác nhận sự nhất quán của hợp đồng Firebase/BLE và các lớp bảo vệ. Bộ unit test hiện có tập trung vào dấu điều khiển và hậu kiểm té ngã. Tuy nhiên, báo cáo không khẳng định độ chính xác hay độ trễ khi



chưa có log benchmark; đây là giới hạn quan trọng và cũng là kế hoạch đánh giá tiếp theo. Chương cuối tổng kết đóng góp và hướng phát triển từ các kết quả này.

Chương 6 – Kết luận

6.1 Tổng kết

Đồ án đã nghiên cứu và xây dựng một nguyên mẫu robot trợ lý ảo hỗ trợ chăm sóc, giám sát người cao tuổi tại nhà. Khác với một ứng dụng AI đơn lẻ, hệ thống bao phủ toàn chuỗi từ cảm nhận bằng camera, theo dõi chủ thể, nhận biết tình huống nghi ngờ, tương tác xác nhận, lưu cảnh báo, gọi người chăm sóc đến điều khiển phần thân robot. Bốn khối sản phẩm gồm RobotApp, CaregiverApp, mô hình/quy trình chuyển đổi AI và firmware ESP32 được kết nối bằng các hợp đồng Firebase, WebRTC và BLE cụ thể.

RobotApp tận dụng smartphone/tablet như bộ não robot theo định hướng OpenBot: camera, microphone, loa, mạng và năng lực suy luận được dùng chung cho nhiều nhiệm vụ. Ứng dụng tích hợp CameraX, TFLite, OpenCV, Firebase, WebRTC, BLE, nhận dạng giọng nói, TTS và Groq. CaregiverApp cung cấp hồ sơ, lịch nhắc, chờ theo dõi/té ngã, nhật ký, cuộc gọi và joystick. Firmware ESP32 dùng NimBLE và SimpleFOC để điều khiển kín hai BLDC theo phản hồi AS5600.

Pipeline thị giác kết hợp mô hình pose với tracking-by-detection, Kalman, Hungarian và Re-ID nhẹ. Người cần chăm sóc được bảo vệ dưới ID0; chỉ ID0 đi vào LSTM. Model LSTM xử lý cửa sổ 15 khung hình với 69 đặc trưng, sau đó máy trạng thái và bộ hậu kiểm dùng các bằng chứng vùng bao, keypoint, tư thế, vị trí thân trên và mất dấu. Khi cần xác nhận, robot dừng, hỏi bằng giọng nói và mặc định theo nhánh nguy hiểm nếu người dùng không phản hồi hoặc dịch vụ phân loại lỗi.

Hệ thống giải quyết hai xung đột kỹ thuật đáng chú ý. Thứ nhất, CameraX sở hữu camera khi bình thường, WebRTC sở hữu camera trong cuộc gọi và chia sẻ local frame cho AI. Thứ hai, tracking fragment là chủ sở hữu duy nhất của RobotControlManager/BLE; mọi nguồn joystick và auto-follow hội tụ tại cùng lớp kiểm tra. Các quyết định này tăng tính ổn định hơn so với việc mở camera hoặc GATT client riêng cho từng fragment.

Chuỗi an toàn chuyển động gồm timestamp và miền lệnh trên RTDB, bộ lọc lệnh cũ/không hữu hạn tại RobotApp, watchdog 1,2 s ở Android, giới hạn tốc độ, callback ngắt BLE và watchdog 800 ms ở firmware. Đây là đóng góp thiết kế có thể kiểm tra độc lập với độ chính xác AI. Hai ứng dụng đã được nhóm dự án build thành công và có APK debug; các artifact model H5/PT/ONNX/TFLite cùng script chuyển đổi cho thấy luồng triển khai AI trên Android đã được thực hiện.

6.2 Kết quả đạt được

- Xây dựng kiến trúc nhiều tầng phù hợp robot chăm sóc tại nhà với phân công rõ giữa caregiver, cloud, robot Android và vi điều khiển.
- Xây dựng RobotApp có theo dõi người, phát hiện/hậu kiểm té ngã, hội thoại, nhắc lịch, video call và BLE.
- Xây dựng CaregiverApp có quản lý tài khoản/hồ sơ/lịch, bật tắt AI, xem lịch sử, nhận cuộc gọi và điều khiển joystick.
- Xây dựng firmware ESP32 điều khiển hai động cơ BLDC bằng SimpleFOC, giao tiếp BLE GATT và dừng an toàn.
- Tạo pipeline chuyển đổi LSTM Keras và pose PyTorch qua ONNX/SavedModel sang TFLite, gồm biến thể 320×320 , GPU và FP16.
- Thiết kế khóa mục tiêu ID0, khôi phục có điều kiện và bộ hậu kiểm sự cố có lý do/bằng chứng.
- Đặc tả đầy đủ path dữ liệu, trạng thái WebRTC, format BLE, mapping motor và timeout.
- Xây dựng khung kiểm thử truy vết từ yêu cầu tới module, unit test và kịch bản đầu cuối.

6.3 Hạn chế

Nguyên mẫu chưa phải sản phẩm y tế và chưa trải qua đánh giá lâm sàng. Báo cáo chưa có log huấn luyện/validation đầy đủ để công bố metric AI độc lập, cũng chưa có benchmark chuẩn hóa trên nhiều thiết bị Android và nhiều môi trường nhà. Dữ liệu té ngã dựng lại có thể khác đáng kể với sự cố thật của người cao tuổi. Camera RGB có điểm mù, nhạy ánh sáng/che khuất và tạo rủi ro riêng tư.

Kiến trúc hiện phụ thuộc Firebase, Groq và STUN/TURN. Khi mất Internet, AI/BLE cục bộ có thể tiếp tục hoặc dừng an toàn, nhưng hội thoại cloud, đồng bộ caregiver và cuộc gọi suy giảm. Một số node RTDB như lệnh, signaling, cảnh báo và call history còn ở phạm vi toàn cục. Cấu trúc này phù hợp demo một robot nhưng phải thay đổi trước khi hỗ trợ nhiều gia đình. Firebase Rules, quản lý secret, TURN credential và auth gate cần được harden.

Re-ID histogram chỉ là đặc trưng ngoại hình nhẹ. Khi nhiều người mặc giống nhau, ID0 có thể mất hoặc ghép sai. Các ngưỡng hậu kiểm được thiết lập theo kinh nghiệm và cần calibration theo vị trí camera. Firmware parser còn dựa vào `String.toFloat`; cơ cấu robot cần nút dừng vật lý, bảo vệ dòng/điện áp và kiểm thử cơ khí. APK RobotApp debug lớn và cần tối ưu đóng gói.

6.4 Hướng phát triển

6.4.1 Đánh giá AI có thể tái lập

Ưu tiên đầu tiên là hoàn thiện dữ liệu và pipeline thí nghiệm. Cần lưu script huấn luyện, file cấu hình, seed, split theo chủ thể/video, lịch sử loss/metric, confusion matrix và checksum model. Xây tập test trong nhà độc lập, gồm hành vi gần giống té ngã, che khuất và nhiều người. Báo cáo precision, recall, F1 ở cấp sự cố cùng confidence interval; thực hiện ablation pose, LSTM và hậu kiểm.

6.4.2 Tăng độ bền theo dõi và phát hiện

Có thể thay histogram bằng embedding Re-ID nhẹ đã lượng tử hóa, kết hợp quần áo, khuôn mặt khi có sự đồng ý và quỹ đạo. Cần mô hình hóa thời gian không đều khi drop-frame, bổ sung quality score của pose và hiệu chỉnh ngưỡng theo camera. Sensor fusion với IMU trên robot, radar hoặc thiết bị đeo tùy chọn có thể giảm điểm mù, nhưng phải cân nhắc chi phí và trải nghiệm.

6.4.3 An toàn và điều khiển robot

Firmware nên dùng parser hữu hạn xác nhận toàn chuỗi, sequence number và CRC/checksum, từ chối NaN/vô hạn, giới hạn gia tốc và ramp setpoint. Bổ sung bumper, cảm biến khoảng cách, đo dòng, giám sát pin, emergency stop vật lý và state machine safety độc lập. Lệnh từ caregiver và auto-follow nên đi qua arbiter có ưu tiên: E-stop > manual > auto-follow > idle.

6.4.4 Backend và bảo mật nhiều người dùng

Schema cần chuyển sang families/familyId/robots/robotId, tách session call theo callId và dùng rules kiểm tra membership/role. Lệnh điều khiển nên có sequence, issuedAt, expiresAt, sender UID và robotId. API key LLM phải đặt sau backend/proxy; TURN dùng credential ngắn hạn; log nhạy cảm có thời hạn lưu và cơ chế

xóa. Thực hiện threat modeling, kiểm thử rules bằng Firebase Emulator và rà soát theo OWASP MASVS.

6.4.5 Khả năng hoạt động khi mất mạng

Các chức năng an toàn cốt lõi nên hoạt động cục bộ: phát hiện té ngã, câu hỏi xác nhận mẫu, TTS offline và lưu hàng đợi cảnh báo. Khi mạng trở lại, event được đồng bộ với idempotency key. Có thể bổ sung SMS/call GSM trên thiết bị hỗ trợ như kênh dự phòng, nhưng phải kiểm soát quyền và chi phí.

6.4.6 Cải thiện trải nghiệm người cao tuổi

Giao diện cần được đánh giá với người dùng mục tiêu: cỡ chữ, tương phản, tốc độ nói, phương ngữ, câu hỏi ngắn, thời gian chờ và cách sửa nhận dạng sai. Robot phải giải thích rõ khi camera/micro hoạt động và cho phép tắt. Thiết kế không nên khuyến khích thay thế hoàn toàn tương tác con người; mục tiêu là hỗ trợ người chăm sóc và tăng cơ hội liên lạc.

6.4.7 Tối ưu triển khai

Do CPU/GPU/NNAPI trên thiết bị mục tiêu để chọn delegate thay vì mặc định một biến thể. Tối ưu model bằng FP16/INT8 sau calibration, giảm copy bitmap, dùng buffer tái sử dụng và kiểm soát nhiệt. Bản release dùng R8, ABI split/App Bundle và loại dependency không dùng. CI nên build cả hai app, chạy unit test, kiểm tra schema contract và tạo manifest artifact.

6.5 Kết luận cuối cùng

Đề án cho thấy một smartphone Android có thể trở thành trung tâm của robot chăm sóc chi phí hợp lý, kết hợp AI biên với dịch vụ thời gian thực và phần thân điều khiển kín. Giá trị của hệ thống không chỉ nằm ở từng mô hình, mà ở cách nhiều thành phần phối hợp, chuyển quyền tài nguyên và trở về trạng thái an toàn khi thông tin thiếu hoặc kết nối gián đoạn. Những hạn chế đã nêu xác định rõ ranh giới của nguyên mẫu và tạo lộ trình cụ thể để phát triển thành một hệ thống đáng tin cậy, bảo mật và có thể đánh giá trong môi trường thực tế.

TÀI LIỆU THAM KHẢO

1. World Health Organization, “Ageing and health,” 2025. [Online]. Available: [WHO – Ageing and health](#). [Accessed: 31-Aug-2026].
2. General Statistics Office of Viet Nam and UNFPA Viet Nam, “Population ageing and projections of older population: The Population and Housing Census 2019,” 2021. [Online]. Available: https://vietnam.unfpa.org/sites/default/files/resource-pdf/unfpa_6_projection_en_7.11.pdf.
3. UNFPA Viet Nam, “Applying foresight to curate a care economy for older persons in Viet Nam – Executive summary,” 2025. [Online]. Available: https://vietnam.unfpa.org/sites/default/files/pub-pdf/2025-12/Applying%20foresight%20to%20curate%20a%20care%20economy%20for%20OP%20in%20VN_EN_FINAL.pdf.
4. World Health Organization, “Falls,” Apr. 2021. [Online]. Available: <https://www.who.int/news-room/fact-sheets/detail/falls>.
5. S. Yap, V. Thirumoorthy, and Y. H. Kwan, “Factors associated with medication adherence in older patients: A systematic review,” *Aging Medicine*, vol. 2, no. 4, pp. 254–266, 2019, doi: 10.1002/agm2.12079.
6. World Health Organization, “Mental health of older adults,” 2025. [Online]. Available: <https://www.who.int/news-room/fact-sheets/detail/mental-health-of-older-adults>.
7. M. S. Søraa et al., “The use of robotic technology in the healthcare of people above the age of 65—A systematic review,” *Healthcare*, vol. 11, no. 6, Art. no. 904, 2023, doi: 10.3390/healthcare11060904.
8. X. Wang, J. Shen, and Q. Chen, “How PARO can help older people in elderly care facilities: A systematic review of randomized controlled trials,” *International Journal of Nursing Knowledge*, vol. 33, no. 1, 2022, doi: 10.1111/2047-3095.12327.
9. H. Hung et al., “The benefits of and barriers to using a social robot PARO in care settings: A scoping review,” *BMC Geriatrics*, vol. 19, Art. no. 232, 2019, doi: 10.1186/s12877-019-1244-6.

10. M. Müller, V. Koltun, and O. Biza, “OpenBot: Turning smartphones into robots,” in *Proc. IEEE International Conference on Robotics and Automation (ICRA)*, 2021, pp. 9305–9311, doi: 10.1109/ICRA48506.2021.9561788.
11. J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, “You Only Look Once: Unified, real-time object detection,” in *Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 779–788, doi: 10.1109/CVPR.2016.91.
12. Z. Zou, K. Chen, Z. Shi, Y. Guo, and J. Ye, “Object detection in 20 years: A survey,” *Proceedings of the IEEE*, vol. 111, no. 3, pp. 257–276, 2023, doi: 10.1109/JPROC.2023.3238520.
13. T.-Y. Lin et al., “Microsoft COCO: Common objects in context,” in *Proc. European Conference on Computer Vision (ECCV)*, 2014, pp. 740–755, doi: https://doi.org/10.1007/978-3-319-10602-1_48.
14. A. Krizhevsky, I. Sutskever, and G. E. Hinton, “ImageNet classification with deep convolutional neural networks,” in *Advances in Neural Information Processing Systems*, vol. 25, 2012.
15. K. Simonyan and A. Zisserman, “Very deep convolutional networks for large-scale image recognition,” in *Proc. International Conference on Learning Representations (ICLR)*, 2015.
16. K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” in *Proc. IEEE CVPR*, 2016, pp. 770–778, doi: 10.1109/CVPR.2016.90.
17. A. Vaswani et al., “Attention is all you need,” in *Advances in Neural Information Processing Systems*, vol. 30, 2017.
18. S. Hochreiter and J. Schmidhuber, “Long short-term memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997, doi: 10.1162/neco.1997.9.8.1735.
19. A. Bewley, Z. Ge, L. Ott, F. Ramos, and B. Upcroft, “Simple online and realtime tracking,” in *Proc. IEEE International Conference on Image Processing (ICIP)*, 2016, pp. 3464–3468, doi: 10.1109/ICIP.2016.7533003.
20. N. Wojke, A. Bewley, and D. Paulus, “Simple online and realtime tracking with a deep association metric,” in *Proc. IEEE ICIP*, 2017, pp. 3645–3649, doi: 10.1109/ICIP.2017.8296962.

21. Y. Zhang et al., “ByteTrack: Multi-object tracking by associating every detection box,” in *Proc. ECCV*, 2022, pp. 1–21, doi: https://doi.org/10.1007/978-3-031-20047-2_1.
22. R. E. Kalman, “A new approach to linear filtering and prediction problems,” *Journal of Basic Engineering*, vol. 82, no. 1, pp. 35–45, 1960, doi: 10.1115/1.3662552.
23. H. W. Kuhn, “The Hungarian method for the assignment problem,” *Naval Research Logistics Quarterly*, vol. 2, no. 1–2, pp. 83–97, 1955, doi: 10.1002/nav.3800020109.
24. A. Alam et al., “Vision-based human fall detection systems using deep learning: A review,” *Computers in Biology and Medicine*, vol. 146, Art. no. 105626, 2022, doi: 10.1016/j.combiomed.2022.105626.
25. L. E. Sucerquia, J. D. López, and J. F. Vargas-Bonilla, “Dataset for human fall recognition in an uncontrolled environment,” *Data in Brief*, vol. 45, Art. no. 108610, 2022, doi: 10.1016/j.dib.2022.108610.
26. H. Khan, I. Ullah, M. Shabaz, M. F. Omer, M. T. Usman, M. S. Guellil, and J. Koo, “Visionary vigilance: Optimized YOLOv8 for fallen person detection with large-scale benchmark dataset,” *Image and Vision Computing*, vol. 149, Art. no. 105195, 2024, doi: 10.1016/j.imavis.2024.105195.
27. R. Khanam and M. Hussain, “YOLOv11: An overview of the key architectural enhancements,” *arXiv:2410.17725*, 2024, doi: 10.48550/arXiv.2410.17725.
28. Google, “TensorFlow Lite: Deploy machine learning models on mobile and edge devices,” [Online]. Available: <https://www.tensorflow.org/lite>. [Accessed: 31-Aug-2026].
29. Google Firebase, “Firebase Realtime Database,” [Online]. Available: <https://firebase.google.com/docs/database>. [Accessed: 31-Aug-2026].
30. Google Firebase, “Understand Firebase Realtime Database Security Rules,” [Online]. Available: <https://firebase.google.com/docs/database/security>. [Accessed: 31-Aug-2026].
31. W3C, “WebRTC: Real-Time Communication in Browsers,” W3C Recommendation, Mar. 2025. [Online]. Available: <https://www.w3.org/TR/webrtc/>.

32. A. Keränen, C. Holmberg, and J. Rosenberg, “Interactive Connectivity Establishment (ICE): A protocol for Network Address Translator traversal,” RFC 8445, IETF, Jul. 2018, doi: 10.17487/RFC8445.
33. E. Rescorla, “WebRTC security architecture,” RFC 8827, IETF, Jan. 2021, doi: 10.17487/RFC8827.
34. Bluetooth SIG, “The Bluetooth Low Energy primer,” 2024. [Online]. Available: <https://www.bluetooth.com/bluetooth-le-primer/>.
35. Bluetooth SIG, “Bluetooth Core Specification,” [Online]. Available: <https://www.bluetooth.com/specifications/specs/core-specification/>. [Accessed: 31-Aug-2026].
36. SimpleFOC Community, “Arduino SimpleFOC library documentation,” [Online]. Available: <https://docs.simplefoc.com/>. [Accessed: 31-Aug-2026].
37. J. R. Hendershot and T. J. E. Miller, *Design of Brushless Permanent-Magnet Machines*. Venice, FL, USA: Motor Design Books, 2010.
38. Google, “CameraX architecture,” Android Developers. [Online]. Available: [Android Developers – CameraX architecture](#). [Accessed: 31-Aug-2026].
39. Google, “Bluetooth permissions,” Android Developers. [Online]. Available: <https://developer.android.com/develop/connectivity/bluetooth/bt-permissions>. [Accessed: 31-Aug-2026].
40. Groq, “API reference: Chat completions,” [Online]. Available: <https://console.groq.com/docs/api-reference>. [Accessed: 31-Aug-2026].
41. National Institute of Standards and Technology, “Artificial Intelligence Risk Management Framework (AI RMF 1.0),” NIST AI 100-1, Jan. 2023, doi: 10.6028/NIST.AI.100-1.
42. OWASP Foundation, “OWASP Mobile Application Security Verification Standard,” [Online]. Available: <https://mas.owasp.org/MASVS/>. [Accessed: 31-Aug-2026].
43. ISO/IEC, *ISO/IEC 25010:2023 Systems and software engineering—SQuaRE—Product quality model*, 2023.

44. P. Lewis et al., “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 9459–9474.

PHỤ LỤC

Phụ lục A – Đặc tả use case

UC01 – Đăng ký và đăng nhập

Tác nhân	Người chăm sóc/người quản trị thiết bị robot
Mục tiêu	Tạo hoặc sử dụng tài khoản Firebase để nhận UID và truy cập dữ liệu chăm sóc tương ứng.
Tiền điều kiện	Thiết bị có Internet, ứng dụng trở đúng Firebase project.
Luồng chính	(1) Nhập email/mật khẩu; (2) ứng dụng gửi Firebase Auth; (3) nhận phiên và UID; (4) tải Users/uid; (5) chuyển màn hình chính.
Ngoại lệ	Email sai định dạng, mật khẩu sai, tài khoản chưa xác minh, mất mạng hoặc profile chưa tồn tại.
Hậu điều kiện	Phiên hợp lệ được duy trì; listener dữ liệu chỉ dùng UID được xác thực.
Yêu cầu liên quan	FR01, FR02, NFR05.

UC02 – Quản lý hồ sơ chăm sóc

Tác nhân	Người chăm sóc
Mục tiêu	Cập nhật thông tin giúp robot xưng hô và hỗ trợ phù hợp.
Tiền điều kiện	Đã đăng nhập và có quyền trên profile.
Luồng chính	Mở Profile; sửa họ tên, tên robot, đại từ, người liên hệ, quan hệ, tình trạng, ghi chú, sở thích; xác nhận; ghi Users/uid; RobotApp nhận thay đổi.
Ngoại lệ	Trường bắt buộc rỗng, số điện thoại sai, mất mạng, write bị Rules từ chối.
Hậu điều kiện	Profile mới được dùng khi tạo prompt/lời nhắc; thay đổi được ghi log nếu triển khai audit.
Yêu cầu liên quan	FR02, NFR05, NFR06.

UC03 – Quản lý lịch nhắc

Tác nhân	Người chăm sóc
Mục tiêu	Tạo, sửa, bật/tắt hoặc xóa lịch thuốc/sinh hoạt.
Tiền điều kiện	Đăng nhập; thời gian và ngày lập hợp lệ.
Luồng chính	Chọn thêm lịch; nhập tiêu đề, ghi chú, thời gian, ngày; lưu dưới Schedules/uid/id; danh sách cập nhật; RobotApp nhận lịch active.
Ngoại lệ	Lịch trùng, múi giờ thay đổi, thiết bị khởi động lại, mất mạng, thời gian quá khứ.
Hậu điều kiện	Lịch định nghĩa được lưu riêng với từng lần thực hiện.
Yêu cầu liên quan	FR03, FR04, NFR10.

UC04 – Nhận và xác nhận lời nhắc

Tác nhân	Người cao tuổi; RobotApp
Mục tiêu	Phát lời nhắc dễ hiểu và lưu phản hồi.
Tiền điều kiện	Lịch active tới hạn; RobotApp hoạt động; TTS sẵn sàng.
Luồng chính	ReminderService tạo log delivered; phát sự kiện; chatbot phát TTS; nghe phản hồi; cập nhật confirmed/completed và confirmedAt.
Ngoại lệ	Người dùng đang gọi, TTS/STT bận, không phản hồi, lịch bị tắt đồng thời, dịch vụ LLM lỗi.
Hậu điều kiện	Một log phản ánh kết quả; lịch gốc không bị thay đổi ngoài quy tắc lặp.
Yêu cầu liên quan	FR04, FR05, FR19.

UC05 – Hội thoại trợ lý

Tác nhân	Người cao tuổi
Mục tiêu	Trao đổi bằng giọng nói và có câu trả lời theo hồ sơ.
Tiền điều kiện	Micro, STT, Internet, Groq và TTS khả dụng.
Luồng chính	Lắng nghe; STT tạo text; thêm history/prompt; gọi stream; ghép theo câu; TTS phát; trở về listening.

Ngoại lệ	Không nghe rõ, API timeout/rate limit, nội dung trống, TTS lỗi, người dùng ngắt.
Hậu điều kiện	History giữ tối đa số message cấu hình; không lưu secret trong log.
Yêu cầu liên quan	FR05, NFR04, NFR06.

UC06 – Theo dõi và tự bám người mục tiêu

Tác nhân	Người chăm sóc; RobotApp; ESP32
Mục tiêu	Robot duy trì khoảng cách/hướng tương đối với ID0.
Tiền điều kiện	Camera, model, BLE và cờ follow khả dụng; vùng di chuyển an toàn.
Luồng chính	Bật follow; pose detection; tracker chọn/duy trì ID0; sinh x, y, ts ; RobotControlManager ánh xạ; ESP32 điều khiển bánh.
Ngoại lệ	Nhiều người, mất ID0, vật cản, thiếu sáng, BLE mất, caregiver giành manual control.
Hậu điều kiện	Khi không đủ điều kiện, lệnh dừng; không tự đổi mục tiêu trong cảnh mờ hồ.
Yêu cầu liên quan	FR06–FR09, FR17–FR20.

UC07 – Phát hiện và xác nhận té ngã

Tác nhân	RobotApp; người cao tuổi; người chăm sóc
Mục tiêu	Phát hiện sự cố, hỏi xác nhận và chuyển cảnh báo có thể can thiệp.
Tiền điều kiện	Fall detection bật; ID0 và model sẵn sàng.
Luồng chính	Pose-tracking–LSTM phát raw fall; hậu kiểm; dừng robot; tạo một alert pending; hỏi TTS; phân loại phản hồi; cập nhật safe/danger.
Ngoại lệ	Mất bbox, bbox bị cắt, API lỗi, STT lỗi, không phản hồi, call đang tồn tại.

Hậu điều kiện	Safe: đóng sự cố; danger/no-response: gọi caregiver với emergencyReason; không phát lập đến khi phục hồi.
Yêu cầu liên quan	FR10–FR14, NFR01, NFR12.

UC08 – Gọi video

Tác nhân	Người cao tuổi/RobotApp; người chăm sóc/CaregiverApp
Mục tiêu	Thiết lập âm thanh, hình ảnh hai chiều.
Tiền điều kiện	Internet, camera/mic, signaling RTDB và STUN/TURN sẵn sàng.
Luồng chính	Caller ghi calling/role, offer; callee đọc và ghi answer; hai bên trao đổi ICE; connected; truyền media; ended và cleanup.
Ngoại lệ	Từ chối, timeout, mất mạng, permission thiếu, camera bận, TURN lỗi.
Hậu điều kiện	Listener, PeerConnection, capturer và renderer được giải phóng; camera trả về tracking; call history được ghi.
Yêu cầu liên quan	FR14, FR15, NFR03, NFR06.

UC09 – Điều khiển joystick

Tác nhân	Người chăm sóc; CaregiverApp; RobotApp; ESP32
Mục tiêu	Lái robot từ xa trong cuộc gọi.
Tiền điều kiện	Call/control screen hoạt động, Firebase và BLE kết nối, khu vực an toàn.
Luồng chính	Chạm joystick; tắt auto-follow; chuẩn hóa x, y ; gửi khoảng 20 Hz với timestamp; RobotApp lọc/ánh xạ; firmware chấp hành; thả để STOP.
Ngoại lệ	Mất touch, fragment pause, timestamp cũ, lệnh không hữu hạn, RTDB/BLE mất.
Hậu điều kiện	Khi thao tác kết thúc hoặc lỗi, target motor về 0.
Yêu cầu liên quan	FR16–FR20, NFR01–NFR03.

UC10 – Xem nhật ký

Tác nhân	Người chăm sóc
Mục tiêu	Xem các hoạt động nhắc, cảnh báo và cuộc gọi theo thời gian.
Tiền điều kiện	Đăng nhập và có quyền đọc dữ liệu.
Luồng chính	Chọn loại diary; query log; sắp xếp theo timestamp; hiển thị trạng thái, nội dung, phản hồi và lý do.
Ngoại lệ	Không có dữ liệu, schema cũ, offline, quyền bị từ chối.
Hậu điều kiện	Không sửa bản ghi khi chỉ xem; query được giới hạn để tránh tải toàn bộ lịch sử.
Yêu cầu liên quan	FR19, NFR05, NFR10.

Phụ lục B – Hợp đồng dữ liệu và giao thức

B.1. Lệnh điều khiển RTDB

```

1 {
2   "robot_control": {
3     "x": -0.65,
4     "y": 0.10,
5     "ts": 1788381000000
6   }
7 }
```

Listing 4: Ví dụ lệnh điều khiển hợp lệ

Ràng buộc: x, y phải là số hữu hạn trong $[-1, 1]$ sau clipping; ts là epoch millisecond; tại RobotApp $now - ts \leq 2000$ ms và $ts - now \leq 5000$ ms. Lệnh STOP là $x = 0, y = 0$ với timestamp mới. Không xóa node thay cho STOP vì listener có thể diễn giải trường thiếu theo giá trị mặc định không mong muốn.

B.2. Signaling WebRTC

```

1 webrtc_signal: {
2   status: "calling" | "connected" | "ended",
3   callerRole: "robot" | "caregiver",
4   emergencyReason: "fall" | null,
```

```

5 offer: { type: "offer", sdp: "..." },
6 answer: { type: "answer", sdp: "..." },
7 robotCandidates: {
8   id: { candidate: "...", sdpMid: "0", sdpMLineIndex: 0 }
9 },
10 caregiverCandidates: {
11   id: { candidate: "...", sdpMid: "0", sdpMLineIndex: 0 }
12 }
13 }

```

Listing 5: Cấu trúc signaling khái quát

Mỗi cuộc gọi tương lai nên có callId, robotId, caller UID, callee UID, createdAt, expiresAt và state transition được rules kiểm soát. Candidate/SDP phải được xóa khi kết thúc để tránh client mới đọc dữ liệu cũ.

B.3. Bản ghi cảnh báo

```

1 fall_alerts/{alertId}: {
2   timestamp: Long,
3   status: "pending" | "safe" | "danger" | "no_response",
4   elderlyResponse: String,
5   verificationReason: String,
6   evidenceScore: Int,
7   visibleKeypoints: Int,
8   bboxFullyInside: Boolean,
9   upperBodyYRatio: Number
10 }

```

Listing 6: Các trường cảnh báo té ngã

B.4. BLE GATT

Phụ lục C – Danh mục module mã nguồn

Bảng 6.11: Thông số BLE triển khai

Device name	OhmniRobot
Service UUID	4FAFC201-1FB5-459E-8FCC-C5C9C331914B
Characteristic UUID	BEB5483E-36E1-4688-B7F5-EA07361B26A8
Property	Write without response
Payload	L<left>R<right>\n, ASCII/UTF-8, dấu chấm thập phân
Android limit	[−20, 20] mỗi motor
Firmware limit	[−40, 40] rad/s; deadzone 0,5/3,0
Watchdog	Android 1.200 ms; firmware 800 ms

Bảng 6.12: Module chính của RobotApp

Tệp/lớp	Trách nhiệm
MainActivity	Điều hướng, vòng đời fragment/call và listener lời mời
tracking_fragment	CameraX/WebRTC frame, AI, ID0, fall và auto-follow
BoundingBoxOverlay	Vẽ vùng bao, keypoint, ID và trạng thái
SortTracker	Ghép detection-track, bảo vệ ID0, tích hợp Re-ID
KalmanBoxTracker	Dự báo/cập nhật trạng thái vùng bao
HungarianAlgorithm	Tối ưu phép gán từ ma trận chi phí
ReIdModule	Histogram vùng thân và khôi phục track
FallDetectionQueue	Bộ đệm cửa sổ 15 frame, 69 feature
LstmFallDetector	Interpreter và máy trạng thái fall/lying/recovery
FallVerificationEngine	Hậu kiểm bằng chứng, khóa sự cố
TrackingService/ ReminderEvent	Chuyển sự kiện giữa tracking/reminder và chat-bot
chatbot_fragment	UI giọng nói, xử lý reminder/fall/call intent
SpeechRecognizer Manager	Đóng gói Android SpeechRecognizer
AndroidTtsService	Đóng gói TextToSpeech và callback
GroqChatService	Chat streaming và các tác vụ phân loại
ReminderService	Theo dõi lịch và phát sự kiện nhắc
video_call_fragment	PeerConnection, signaling, media và camera sharing

Tệp/lớp	Trách nhiệm
RobotControlManager	RTDB command, validation, motor mapping và BLE

Bảng 6.13: Module chính của CaregiverApp

Tệp/lớp	Trách nhiệm
Login/SignUpActivity	Firestore Authentication và tạo profile
MainActivity	Điều hướng chính và khởi động incoming-call service
ProfileFragment	Đọc/cập nhật Users/uid
tracking_fragment	Cờ follow/fall và STOP khi tắt
Schedule/ AddScheduleFragment	CRUD lịch nhắc
ScheduleAdapter/ ScheduleItem	Danh sách và trạng thái active
Diary*Fragment/ Adapter	Reminder log, fall alert và call history
IncomingCallService	Listener nền và notification cuộc gọi
IncomingCallActivity	UI nhận/từ chối call toàn màn hình
video_call_fragment	WebRTC caller/callee, joystick và call log
chatbot_fragment	Placeholder trong snapshot hiện tại

Phụ lục D – Manifest artifact AI

Mỗi model phát hành nên đi kèm manifest theo mẫu:

```

1 model_id: pose-320-fp16-v1
2 source_weight: yolo26n-pose.pt
3 source_sha256: <hash>
4 converter:
5   ultralytics: <version>
6   onnx: <version>
7   onnx2tf: <version>
8   tensorflow: <version>
9 input:

```

```

10  shape: [1, 320, 320, 3]
11  dtype: float32
12  color: RGB
13  normalization: /255
14  output:
15    shape: <recorded shape>
16    decoding: android-decoder-v1
17  quantization: fp16-weights
18  validation:
19    dataset_split_sha256: <hash>
20    max_abs_tensor_error: <value>
21    final_metric_delta: <value>

```

Listing 7: Mẫu manifest model có thể tải lập

Đối với LSTM, manifest phải ghi thứ tự 69 feature, cách normalize keypoint, class order của output, sequence length, stride, threshold và hysteresis. Nếu thiếu một trong các thông tin này, model có thể nạp thành công nhưng cho kết quả sai ngữ nghĩa.

Phụ lục E – Quy trình build và triển khai

E.1. Hai ứng dụng Android

Yêu cầu JDK 11 trở lên, Android SDK phù hợp compile SDK 36 và tệp cấu hình Firebase của từng applicationId. Các lệnh chuẩn:

```

1  cd DATN_RobotApp_Smartphone_v2
2  gradlew.bat testDebugUnitTest
3  gradlew.bat assembleDebug
4
5  cd DATN_CaregiverApp_Smartphone_v2
6  gradlew.bat testDebugUnitTest
7  gradlew.bat assembleDebug

```

Listing 8: Build và unit test Android trên Windows

Không đưa local.properties, API key hoặc credential TURN vào Git. Build release cần signing key bảo mật, minify/resource shrinking và kiểm thử lại reflection/native library.

E.2. Firmware

Trong Arduino IDE, chọn ESP32 Dev Module; cài SimpleFOC phiên bản tương thích và NimBLE-Arduino; mở `robot_ble_control.ino`; kiểm tra pin; compile/upload; mở Serial 115200. Lần đầu initFOC phải nâng bánh khỏi sàn. Kiểm tra encoder quay đúng chiều và chỉ thử setpoint nhỏ trước.

E.3. Dịch vụ

Thiết lập Firebase Auth email/password, RTDB và Rules. Cấu hình STUN/TURN trên cả hai ứng dụng. Cấp Groq key qua cơ chế bí mật. Tạo một tài khoản thử, profile, lịch và cặp thiết bị; xóa SDP/ICE cũ trước test. Đảm bảo notification/full-screen intent được cấp trên caregiver.

Phụ lục F – Phiếu ghi kết quả kiểm thử

Test ID	
Commit/model hash	
Ngày, người thực hiện	
Thiết bị/Android	
Mạng/BLE/khoảng cách	
Tiền điều kiện	
Các bước	
Kết quả mong đợi	
Kết quả quan sát	
Metric/log/video	
Pass/Fail/Blocked	
Mã lỗi và ghi chú	

Phụ lục G – Checklist an toàn trình diễn

- Bánh xe được nâng khỏi sàn khi kiểm tra dấu/FOC lần đầu.

Bảng 6.15: Mẫu tổng hợp benchmark AI

Cấu hình	Median (ms)	P95 (ms)	FPS hiệu dụng	Drop frame (%)
CPU float32				
GPU graph				
FP16				

Bảng 6.16: Mẫu tổng hợp phát hiện té ngã theo sự cố

Cấu hình	Precision	Recall	F1	Cảnh báo giả/giờ
Pose/luật				
Pose + LSTM				
Pose + LSTM + hậu kiểm				

Bảng 6.17: Mẫu tổng hợp độ trễ và an toàn điều khiển

Kịch bản	Median	P95	Max/ghi chú
Caregiver tới Robo-tApp			
RobotApp tới BLE callback			
Thời gian dừng khi mất RTDB			
Thời gian dừng khi mất BLE			

- Khu vực chạy phẳng, không có bậc thang, trẻ nhỏ, vật nuôi hoặc vật cản dễ vỡ.
- Pin, dây, driver và động cơ không quá nhiệt; có cầu chì/bảo vệ phù hợp.
- Có người giữ công tắc ngắt nguồn; STOP đã được thử trước auto-follow.
- Không dùng người cao tuổi thật để dựng lại té ngã; có đệm và giám sát.
- Không trình chiếu API key, token, email/số điện thoại thật hoặc dữ liệu sức khỏe.
- Kiểm tra RTDB Rules và dùng tài khoản/dữ liệu demo.
- Kiểm tra Internet, TURN, notification và quyền camera/micro/BLE.
- Lưu commit/model checksum và phương án khôi phục bản APK/firmware ổn định.
- Ghi log, nhưng tắt/xóa log nhạy cảm sau trình diễn.